VCS

K19G

2026-10-11

Table of contents

Introduction **58**

VCS **59**
Who this book is for . . . 59
What you will master . . . 59
How to read this so it sticks . . . 60
The companion kit . . . 60
Conventions . . . 60
Acknowledgements . . . 60

Git Foundations **61**

Mental Model **62**

Theory **63**

Theory: Snapshots, Names, and a Graph **64**
Invariants . . . 64
Why this model . . . 64
What can break . . . 64
How to read this chapter . . . 65

The Version Control Landscape **66**

The Version Control Landscape **67**
Why you need a history . . . 67
Three generations . . . 67
Why Git . . . 68
What a forge is . . . 68
What Git is not . . . 68
Next . . . 68

The Directed Acyclic Graph **69**

The Directed Acyclic Graph **70**
Snapshots with parents . . . 70
A branch is a name . . . 70
A second line of work . . . 70
HEAD . . . 71
Look at the graph . . . 71

The Three-Tree Model 72

The Three-Tree Model 73
Three snapshots, one project . . . 73
Why the index exists . . . 73
`status` is the three-tree report . . . 74
`reset` in one sentence . . . 74
Check it . . . 74

Object Types 75

Object Types 76
Content-addressed storage . . . 76
Four types . . . 76
Packfiles are compression . . . 77
Integrity . . . 77
What this buys a server . . . 77

References: Branches, Tags, and HEAD 78

References: Branches, Tags, and HEAD 79
A ref is a name for a hash . . . 79
Branches move, tags mostly do not . . . 79
HEAD is usually a pointer to a pointer . . . 80
Remote-tracking names . . . 80
Plumbing vs porcelain . . . 80

The Distributed Model 81

The Distributed Model 82
Every clone is a full copy . . . 82
Implications . . . 82
What a hosting platform must speak . . . 83

Detached HEAD 84

Detached HEAD 85
Outline . . . 85

Revisions and Ranges 86

Revisions and Ranges 87
Outline . . . 87

Install and Config 88

Theory 89

Theory: Identity and Config are Data, Not Login **90**
Invariants . . . 90
Why layers . . . 90
What can break . . . 90
How to read this chapter . . . 91

Installing Git **92**

Installing Git **93**
Check what you already have . . . 93
Install from the distro . . . 93
Line endings and the filesystem . . . 94
What else you need for Parts I–II . . . 94
Confirm . . . 94

Config Layers **95**

Config Layers **96**
Three layers . . . 96
Set the global layer once . . . 96
Repo-only overrides . . . 97
`includeIf` for home vs work . . . 97
See the result . . . 98

Identity, Email, and Essential Settings **99**

Identity, Email, and Essential Settings **100**
Identity is metadata, not a login . . . 100
Check what will be recorded . . . 100
Signing is later . . . 101
Author vs committer . . . 101
Essential settings, one list . . . 101

SSH vs HTTPS **102**

SSH vs HTTPS **103**
Two ways to talk to a remote . . . 103
SSH keys . . . 103
HTTPS and tokens . . . 104
Pick one default . . . 105

Editor, Pager, and Difftool **106**

Editor, Pager, and Difftool **107**
Outline . . . 107

Aliases **108**

Aliases **109**
Outline . . . 109

Updating Git on Linux **110**

Updating Git on Linux **111**
Outline . . . 111

First Repository **112**

Theory **113**

Theory: The Add/Commit Cycle is the Product **114**
Invariants . . . 114
Why the index exists . . . 114
What can break . . . 114
How to read this chapter . . . 115

Initializing a Repository **116**

Initializing a Repository **117**
A directory becomes a repository . . . 117
What `git init` wrote . . . 118
`init` vs `clone` . . . 118
Already have files? . . . 118
`init` in the wrong place . . . 118

The Add/Commit Cycle **119**

The Add/Commit Cycle **120**
The loop . . . 120
One file, one commit . . . 120
A second commit . . . 121
What `-m` is doing . . . 121
`git add` shapes . . . 121
`commit -a` . . . 122
After the cycle . . . 122

status, log, and diff **123**

`git status`, `git log`, `git diff` **124**
`git status` — where you are . . . 124
`git log` — the DAG as a list . . . 124
`git diff` — the patch . . . 125
DiffTool . . . 125
A one-minute drill . . . 125

Undoing Mistakes **126**

Undoing Mistakes **127**
Three common “undo”s . . . 127
Discard unstaged work . . . 127

Unstage, keep the edit . . . 128
Amend the last commit . . . 128
Revert is a new commit . . . 128
`rm` and `mv` . . . 129
If you are lost . . . 129

Hands-On: Build a Tiny Project **130**

Hands-On: Build a Tiny Project **131**
Setup . . . 131
Commit 1 — README . . . 131
Commit 2 — a tiny module . . . 132
Read it . . . 132
Break it on purpose . . . 132
Stage, unstage, restage . . . 132
Amend the message . . . 133
Ignore a secret-shaped file . . . 133
Done when . . . 133

.gitignore in Depth **134**

.gitignore in Depth **135**
Outline . . . 135

Interactive Add **136**

Interactive Add **137**
Outline . . . 137

git rm and git mv **138**

git rm and git mv **139**
Outline . . . 139

What Not to Commit **140**

What Not to Commit **141**
Outline . . . 141

Inspecting History **142**

Theory **143**

Theory: History is a Query over the DAG **144**
Invariants . . . 144
Why this matters on a forge . . . 144
What can break . . . 144
How to read this chapter . . . 144

Advanced git log **145**

Advanced git log **146**
Outline 146

Two-Dot and Three-Dot Ranges **147**

Two-Dot and Three-Dot Ranges **148**
Outline 148

git show and –stat **149**

git show and –stat **150**
Outline 150

git blame **151**

git blame **152**
Outline 152

Pickaxe and –follow **153**

Pickaxe and –follow **154**
Outline 154

git shortlog **155**

git shortlog **156**
Outline 156

Hands-On: History Archaeology **157**

Hands-On: History Archaeology **158**
Outline 158

Ignore, Attributes, and LFS **159**

Theory **160**

Theory: The Index is a Filter; LFS is a Pointer **161**
Invariants 161
Why Git is a bad backup of videos 161
What can break 161
How to read this chapter 161

gitignore Patterns **162**

gitignore Patterns **163**
Outline 163

gitattributes **164**

gitattributes **165**
Outline . . . 165

export-ignore and git archive **166**

export-ignore and git archive **167**
Outline . . . 167

Git LFS **168**

Git LFS **169**
Outline . . . 169

Hands-On: Ignore and LFS **170**

Hands-On: Ignore and LFS **171**
Outline . . . 171

Everyday Workflows **172**

Commits and History **173**

Theory **174**

Theory: A Commit is Immutable; Messages are for Humans and Machines **175**
Invariants . . . 175
Why messages matter . . . 175
What can break . . . 175
How to read this chapter . . . 176

Commit Anatomy **177**

Commit Anatomy **178**
A commit is a snapshot plus metadata . . . 178
Author and committer . . . 178
Short names . . . 179
The tree under it . . . 179
What `git show` adds . . . 179

Conventional Commits **180**

Commit Messages That Matter: Conventional Commits **181**
Shape of a message . . . 181
Types this book uses . . . 181
Why bother . . . 182
What to put in the body . . . 182
Trailers . . . 182

A local alias . . . 182

Amending a Commit Message **183**

Amending and Fixing an Incorrect Commit Message **184**
The last commit only . . . 184
When this is safe . . . 184
Older than HEAD . . . 185
Recover if you amended by mistake . . . 185
Identity in an amended commit . . . 185

Unstaging a File **186**

Unstaging an Added File **187**
The index is the next commit . . . 187
Restore vs reset . . . 187
Unstage everything . . . 188
Untracked files . . . 188
Partial unstage . . . 188
Check before you commit . . . 188

Hands-On: Amend and Rewrite **189**

Hands-On: Amend and Rewrite a Commit **190**
1. A sloppy commit . . . 190
2. Fix the subject . . . 190
3. Forgot a file . . . 190
4. Staged too much, then unstaged . . . 191
5. Confirm the graph . . . 191

Commit Templates **192**

Commit Templates **193**
Outline . . . 193

Empty and Split Commits **194**

Empty and Split Commits **195**
Outline . . . 195

Trailers and mailmap **196**

Trailers and mailmap **197**
Outline . . . 197

Reading a Commit with git show **198**

Reading a Commit with git show **199**
Outline . . . 199

Reset, Restore, and Checkout **200**

Theory **201**

Theory: Reset Moves Names; Restore Copies Bytes; Revert Adds a Commit **202**
Invariants . . . 202
Why `checkout` split . . . 202
What can break . . . 202
How to read this chapter . . . 203

Soft, Mixed, and Hard Reset **204**

Soft, Mixed, and Hard Reset **205**
One commit, three trees . . . 205
`--soft` moves the name . . . 205
`--mixed` moves the name and the index . . . 206
`--hard` moves all three . . . 206
Remove the scratch commit . . . 207
Where this is safe . . . 207

restore vs reset vs revert **208**

Restore, Reset, and Revert **209**
Three audiences . . . 209
Amend and soft reset . . . 209
Revert goes forward . . . 210
A test you can apply . . . 210

Resetting Paths **211**

Resetting Paths **212**
A path means the branch stays . . . 212
Put the file back, then drop the commit . . . 213
Whole tree versus one path . . . 213

switch, restore, and checkout **214**

switch, restore, and checkout **215**
Two commands, two jobs . . . 215
Switch branches . . . 215
A path is not a branch . . . 216
Local edits that would be overwritten . . . 216
Detached HEAD . . . 216

Hands-On: Reset Lab **218**

Hands-On: Reset Lab **219**
Setup . . . 219
Soft . . . 219
Mixed . . . 220

Hard, and an untracked file . . . 220
Recover with the reflog . . . 220
Leave the repo as you found it . . . 220
Done when . . . 221

Branching and Merging **222**

Theory **223**

Theory: A Branch is a Name; a Merge is a Node with Two Parents **224**
Invariants . . . 224
Why merge commits exist . . . 224
What can break . . . 224
How to read this chapter . . . 225

Creating and Switching Branches **226**

Branch to Create a Safe Working Area **227**
A branch is a name . . . 227
Name them so later pages stay readable . . . 227
Work on the branch . . . 227
List and see . . . 228
Delete . . . 228
Rename . . . 228
Stuck with a dirty working tree . . . 228

Merge Types **229**

Merge Types: Fast-Forward, Recursive, and Octopus **230**
Fast-forward . . . 230
Recursive (two parents) . . . 230
Octopus (three or more parents) . . . 231
See the parents . . . 231
Abort . . . 231

Three-Way Merge **232**

Three-Way Merge Explained **233**
The three trees . . . 233
See the base . . . 233
Stages in the index during a conflict . . . 234
Why this matters for a forge . . . 234
`zdiff3` markers . . . 234

Merge Conflicts **235**

Merge Conflicts: In Git and in Teams **236**
What a conflict is . . . 236
The loop . . . 236

Abort vs continue . . . 237
Binary files and deletes . . . 237
Teams . . . 237
After the merge . . . 237

Fast-Forward for Clean History **238**

Creating a Fast-Forward Merge for a Clean History **239**
When a fast-forward is possible . . . 239
If it fails . . . 239
See it . . . 239
`first-parent` . . . 240

Hands-On: A Real Merge Conflict **241**

Hands-On: Simulate a Real Merge Conflict **242**
Setup . . . 242
Branch A — JSON ping . . . 242
Branch B — string change on main . . . 242
Merge and conflict . . . 243
Done when . . . 243

mergetool **244**

mergetool **245**
Outline . . . 245

ours, theirs, and Renames **246**

ours, theirs, and Renames **247**
Outline . . . 247

Orphan Branches **248**

Orphan Branches **249**
Outline . . . 249

Comparing Branches **250**

Comparing Branches **251**
Outline . . . 251

rerere **252**

rerere **253**
Outline . . . 253

Git 2.56 for Daily Work **254**

Git 2.56 for Daily Work: `add --resolved`, `branch --delete-merged`, and Friends **255**
Thesis . . . 255

Mental model . . . 255
Worked examples (Cases) . . . 256
Case 1: Stage what you resolved, not what you happened to touch . . . 257
Case 2: Set topics up the way `--delete-merged` expects . . . 258
Case 3: Preview, then sweep . . . 259
Case 4: A bisect that cleans up after itself . . . 262
Case 5: Drop one commit without an interactive rebase . . . 263
What you get for free . . . 264
The trap . . . 265
The boring rule . . . 266
Try this . . . 266
Sources . . . 267

Rebasing **268**

Theory **269**

Theory: Rebase Copies Commits; It Does Not Slide Them **270**
Invariants . . . 270
Why teams fight about it . . . 270
What can break . . . 270
How to read this chapter . . . 271

Why Rebase **272**

Why Rebase? Clean History vs Merge Commits **273**
Replay commits on a new parent . . . 273
What you gain . . . 273
What you pay . . . 273
`rebase` vs `merge` in one line . . . 274

Interactive Rebase **275**

Interactive Rebase: Squash, Reorder, Fixup, Polish **276**
Open the todo list . . . 276
Verbs you will use . . . 276
Linux without the editor (labs) . . . 277
Stop and amend (`edit`) . . . 277
Abort . . . 277
Range . . . 277

Rebase Conflicts **278**

Rebase Conflicts: How They Differ from Merge Conflicts **279**
One commit at a time . . . 279
The loop . . . 279
Ours / theirs are swapped from merge . . . 280
`rerere` . . . 280

After success . . . 280

Pull with Rebase **281**

Pull with Rebase When Your Branch Is Out of Date **282**
`pull` is fetch plus integrate . . . 282
When this is the right shape . . . 283
When to merge instead . . . 283
Set it for this branch only . . . 283
After a rebase pull . . . 283

Rebase vs Merge **284**

The Debate: Rebase vs Merge **285**
Same trees, different graphs . . . 285
A default that scales . . . 285
`rerere`, tests, Linux . . . 286
You can mix . . . 286

Hands-On: Clean Up a Messy History **287**

Hands-On: Clean Up a Messy History **288**
Setup . . . 288
Three messy commits . . . 288
Squash with GNU sed (Linux) . . . 288
Optional: rebase onto main . . . 289
Cleanup . . . 289
Done when . . . 289

rebase –onto **290**

rebase –onto **291**
Outline . . . 291

autosquash and fixup **292**

autosquash and fixup **293**
Outline . . . 293

rebase –exec **294**

rebase –exec **295**
Outline . . . 295

update-refs and keep-base **296**

update-refs and keep-base **297**
Thesis . . . 297
Mental model . . . 297

Worked examples (Cases) . . . 298
Case 1: The stale stack (what goes wrong without the flag) . . . 299
Case 2: `--update-refs` moves the whole stack . . . 300
Case 3: Fix the bottom commit from the top, without moving the base . . . 301
Case 4: Make it the default . . . 302
Case 5: Publish the moved stack in one atomic push . . . 303
Case 6: The bottom branch was squash-merged . . . 303
The trap . . . 304
The boring rule . . . 305
Try this . . . 305
Sources . . . 305

Cherry-Pick and Revert **307**

Theory **308**

Theory: Cherry-Pick and Revert are Patches with New Hashes **309**
Invariants . . . 309
Why not reset on production . . . 309
What can break . . . 309
How to read this chapter . . . 309

Cherry-Pick **310**

Cherry-Pick: Moving Commits Surgically **311**
Copy the patch, new hash . . . 311
Several commits . . . 311
Conflicts . . . 311
Empty pick . . . 312
What cherry-pick is for . . . 312

Hotfixes and Backports **313**

Deploying Hotfixes Without Merging Features **314**
The situation . . . 314
Release branch . . . 314
Backport the other direction . . . 314
Label it in the message . . . 315
After the pick . . . 315

Revert and Unmerge **316**

`git revert` and Unmerging Changes **317**
Revert writes history forward . . . 317
Several commits . . . 317
Revert a merge commit . . . 317
Revert vs reset vs restore . . . 318
Un-merge without `-m` . . . 318

Production Emergency Scenarios 319

Real Production Emergency Scenarios 320
1. Bad commit is `HEAD` on `main`, already pulled by others . . . 320
2. Bad commit is in the middle of `main` . . . 320
3. Fix exists on a feature branch, production cannot wait . . . 321
4. You committed a secret . . . 321
5. Merge to `main` was wrong, merge commit is `HEAD`, not pushed . . . 321
6. You are not sure what `main` is . . . 321

Ranges and -x 322

Ranges and -x 323
Outline . . . 323

Empty Picks and –skip 324

Empty Picks and –skip 325
Outline . . . 325

Remotes 326

Theory 327

Theory: A Remote is Another Repository Plus a Nickname 328
Invariants . . . 328
Why PRs are not Git objects . . . 328
What can break . . . 328
How to read this chapter . . . 328

Fetch, Pull, and Push 329

Remote Mechanics: Fetch, Pull, Push 330
A remote is a named URL . . . 330
Fetch . . . 330
Push . . . 331
Pull . . . 331
Tracking . . . 331
Delete a remote branch . . . 331

Multiple Remotes 332

Understanding Multiple Remotes 333
Extra remotes are normal . . . 333
Push to both . . . 333
Independence preview . . . 334
Rename and remove . . . 334
`origin` on a USB disk . . . 334

Force Push **335**

Force Push Dangers **336**
Why push refuses . . . 336
`--force-with-lease` . . . 336
`--force` . . . 336
`main` . . . 336
Recover if you force-pushed by mistake . . . 337
Lease plus fetch . . . 337

Pull Requests and Code Review **338**

Pull Requests, Code Review, and Handling Conflicts in Teams **339**
Git has no pull request object . . . 339
Local shape . . . 339
Conflicts in a PR . . . 340
Review comments . . . 340
Hands-on next . . . 340

Hands-On: Two Clones **341**

Hands-On: Fork, Clone, Branch, Commit, Push **342**
Origin (bare) . . . 342
Your clone . . . 342
A second clone (teammate or your own review checkout) . . . 343
Feature on A, fetch on B . . . 343
Done when . . . 343

Tracking and Upstream **345**

Tracking and Upstream **346**
Outline . . . 346

Refspecs **347**

Refspecs **348**
Outline . . . 348

git bundle **349**

git bundle **350**
Outline . . . 350

Prune and Rename **351**

Prune and Rename **352**
Outline . . . 352

Forks vs Clones **353**

Forks vs Clones **354**
Outline . . . 354

Stash, Tags, and Cleanup **355**

Theory **356**

Theory: Stash is Commits; a Tag is a Name Meant to Stay Put **357**
Invariants . . . 357
Why stash vs a branch . . . 357
What can break . . . 357
How to read this chapter . . . 358

Stash **359**

Stash Mechanics, Including Patch Stashes **360**
A stash is a commit Git keeps on the side . . . 360
Include untracked . . . 360
Patch stash (`-p`) . . . 361
Named and multiple . . . 361
Conflicts on apply . . . 361
Stash vs commit . . . 361

Tags **362**

Tags: Lightweight vs Annotated **363**
Two kinds . . . 363
List and show . . . 363
Push . . . 363
Checkout . . . 364
Delete . . . 364

Cleanup Commands **365**

Cleanup Commands **366**
Merged branches . . . 366
Stashes . . . 366
Untracked files: `git clean` . . . 366
Garbage collection . . . 367
`git fsck` (peek) . . . 367

Hands-On: Stash and Switch **368**

Hands-On: Stash, Work on Another Branch, Apply **369**
Park work . . . 369
Other work . . . 369
Restore the stash . . . 369
Tag this point . . . 370
Done when . . . 370

stash branch 371

stash branch 372
Outline . . . 372

git describe 373

git describe 374
Outline . . . 374

git archive 375

git archive 376
Outline . . . 376

Hooks 377

Theory 378

Theory: Hooks are Local Policy; the Server is Real Policy 379
Invariants . . . 379
Why both . . . 379
What can break . . . 379
How to read this chapter . . . 380

Hook Types 381

Hook Types: Pre-Commit, Commit-Msg, Pre-Push 382
A hook is a program Git runs . . . 382
Client hooks you will write . . . 383
Server hooks (later) . . . 383
Environment . . . 383

Writing a Hook 384

Writing Your First Hook 385
`pre-commit` . . . 385
`commit-msg` . . . 385
`pre-push` . . . 386
`--no-verify` . . . 386

Hook Templates and CI 387

Hook Templates: Sharing Hooks Across Repos 388
`.git/hooks` is local . . . 388
`core.hooksPath` . . . 388
`init.templateDir` . . . 388
CI is the hook you cannot skip . . . 389
Frameworks . . . 389

Hands-On: A Pre-Commit Hook **390**

Hands-On: Create a Pre-Commit Hook **391**
Fail on purpose . . . 391
Pass . . . 391

prepare-commit-msg **393**

prepare-commit-msg **394**
Outline . . . 394

post-checkout and post-merge **395**

post-checkout and post-merge **396**
Outline . . . 396

–no-verify **397**

–no-verify **398**
Outline . . . 398

Submodules and Worktrees **399**

Theory **400**

Theory: Submodules Pin a Hash; Worktrees Share an Object Store **401**
Invariants . . . 401
Why not always submodule . . . 401
What can break . . . 401
How to read this chapter . . . 402

Submodules **403**

Submodule Mechanics: Add, Update, Clone **404**
A submodule is a nested repository plus a gitlink . . . 404
Clone a superproject . . . 405
Update the pin . . . 405
Pitfalls . . . 405

Worktrees **406**

Git Worktrees: The Modern Alternative **407**
One object store, several working trees . . . 407
Why not `git clone` again . . . 407
Linux workflow . . . 407
Detached review . . . 408
Hooks and `GIT_DIR` . . . 408

When to Use Which **409**

When to Use Which **410**
Worktree . . . 410
Submodule . . . 410
Copy the files . . . 410
Monorepo . . . 410
Extra clone . . . 410
Extra remote . . . 410
Rule of thumb . . . 411

Hands-On: Submodule and Worktree **412**

Hands-On: A Repo with a Submodule and a Worktree **413**
Child library . . . 413
Parent with submodule . . . 413
Worktree of the parent . . . 413
Cleanup . . . 414
Done when . . . 414

foreach and sync **415**

foreach and sync **416**
Outline . . . 416

deinit and absorbgitdirs **417**

deinit and absorbgitdirs **418**
Outline . . . 418

Worktree Locks and prune **419**

Worktree Locks and prune **420**
Outline . . . 420

Worktree Lifecycle for Parallel Work: add, prune, repair **421**

Worktree Lifecycle for Parallel Work: `add`, `prune`, `repair` **422**
Thesis . . . 422
Mental model . . . 422
Worked examples (Cases) . . . 423
Case 1: One worktree per line of work . . . 423
Case 2: `rm -rf` leaves a ghost . . . 424
Case 3: Lock what lives somewhere unreliable . . . 425
Case 4: `remove` protects your work, `mv` breaks the link, `repair` fixes it . . . 425
Case 5: Relative links for repositories that move . . . 426
Case 6: A throwaway build that cannot leak . . . 427
The trap . . . 427
The boring rule . . . 428
Try this . . . 428
Sources . . . 429

Branching Workflows **430**

Theory **431**

Theory: A Workflow is a Policy on Names, Not a Git Feature **432**
Invariants . . . 432
Why write it down . . . 432
What can break . . . 432
How to read this chapter . . . 432

Git Flow **433**

Git Flow: The Classic Model **434**
The picture . . . 434
Commands (shape, not a plugin) . . . 434
What it costs . . . 435
What it buys . . . 435

GitHub Flow and Trunk-Based **436**

GitHub Flow and Trunk-Based Development **437**
GitHub Flow (as a Git topology) . . . 437
Trunk-based . . . 437
Compare . . . 438

Feature Flags **439**

Feature Flags and Deployment **440**
A flag is a runtime switch . . . 440
What Git still does . . . 440
Cleanup . . . 441
GitOps preview . . . 441

Monorepos vs Polyrepos **442**

Monorepos vs Polyrepos **443**
Polyrepo (this book's default so far) . . . 443
Monorepo . . . 443
Trade-offs . . . 443
Linux tooling . . . 444
This book . . . 444

Choosing a Strategy **445**

Choosing a Strategy **446**
A default for this book's desk . . . 446
When to change it . . . 446
What Git will not decide . . . 446

The Forking Workflow **448**

The Forking Workflow **449**
Outline . . . 449

Stacked Changes **450**

Stacked Changes **451**
Outline . . . 451

Release Tags and Changelogs **452**

Release Tags and Changelogs **453**
Outline . . . 453

Power Skills **454**

Internals **455**

Theory **456**

Theory: Content Addressing is the Whole Trick **457**
Invariants . . . 457
What can break . . . 457
How to read this chapter . . . 457

The Object Store **458**

The Object Store **459**
Outline . . . 459

Content Addressing — SHA-1, SHA-256, and the Git 3.0 Switch **460**

Content Addressing — SHA-1, SHA-256, and the Git 3.0 switch **461**
Mental model . . . 461
Worked examples . . . 462
Case 1: hash an object by hand . . . 462
Case 2: what a SHA-256 repository is . . . 462
Case 3: what does not cross between formats . . . 463
Case 4: tooling that hard-codes SHA-1 . . . 464
Case 5: previewing Git 3.0 defaults, and pinning yours . . . 465
Case 6: the interoperability work (`compatObjectFormat`) . . . 466
Case 7: migrating an existing repository today . . . 467
Cheat sheet . . . 468
Safety . . . 468
Hands-on checklist . . . 468
Pitfalls . . . 469
Sources . . . 469

Packfiles and git log **470**

Packfiles and git log **471**
Outline . . . 471

Internals That Affect Servers **472**

Internals That Affect Servers **473**
Outline . . . 473

Hands-On: git cat-file **474**

Hands-On: git cat-file **475**
Outline . . . 475

Reflog **476**

Theory **477**

Theory: The Reflog is a Local Journal of Names **478**
Invariants . . . 478
What can break . . . 478
How to read this chapter . . . 478

How the Reflog Works **479**

How the Reflog Works **480**
Outline . . . 480

Recover Deleted Branches **481**

Recover Deleted Branches **482**
Outline . . . 482

The 30-Day Safety Window **483**

The 30-Day Safety Window **484**
Outline . . . 484

Recovery Scenarios **485**

Recovery Scenarios **486**
Outline . . . 486

Bisect **487**

Theory **488**

Theory: Bisect is Binary Search on First-Parent History **489**
Invariants . . . 489
What can break . . . 489

How to read this chapter . . . 489

Binary Search for Bugs **490**

Binary Search for Bugs **491**
Outline . . . 491

Automate Bisect with Tests **492**

Automate Bisect with Tests **493**
Outline . . . 493

2 AM Production Debugging **494**

2 AM Production Debugging **495**
Outline . . . 495

Hands-On: git bisect **496**

Hands-On: git bisect **497**
Outline . . . 497

Performance and Scale **498**

Theory **499**

Theory: Scale is Reachability and Pack Shape **500**
Invariants . . . 500
What can break . . . 500
How to read this chapter . . . 500

Large Repositories **501**

Large Repositories **502**
Outline . . . 502

Shallow, Sparse, and Partial Clones **503**

Shallow, Sparse, and Partial Clones **504**
Outline . . . 504

Performance Tuning **505**

Performance Tuning **506**
Outline . . . 506

git maintenance **507**

git maintenance **508**
Mental model . . . 508

Tasks that matter at scale . . . 508
Worked examples . . . 509
Case 1: One-shot tasks on a lab clone . . . 509
Case 2: Enable incremental strategy and register . . . 510
Case 3: `git maintenance start` on Linux . . . 510
Case 4: Prefetch vs a normal fetch . . . 510
Case 5: Manual vs scheduled, and locks . . . 510
Strategy cheat sheet . . . 511
Safety . . . 511
Hands-on checklist . . . 511
Pitfalls . . . 512
Sources . . . 512

Git Security **513**

Theory **514**

Theory: Signatures Bind a Key to a Hash; History Still Contains Secrets **515**
Invariants . . . 515
What can break . . . 515
How to read this chapter . . . 515

Signed Commits and Tags **516**

Signed Commits and Tags **517**
What a signature covers . . . 517
SSH signing (boring default) . . . 517
One-time setup . . . 518
Sign a commit and prove it locally . . . 518
Allowed signers (local trust) . . . 518
Sign an annotated release tag . . . 519
OpenPGP signing (when policy requires it) . . . 519
Selective signing and overrides . . . 519
Push and forge Verified . . . 520
Safety . . . 520
Hands-on checklist . . . 520
Pitfalls . . . 521
Sources . . . 521

Secrets in History **522**

Secrets in History **523**
Outline . . . 523

Audit Trails **524**

Audit Trails **525**
Outline . . . 525

Security Scanning **526**

Security Scanning **527**
Outline 527

Hands-On: Remove a Secret **528**

Hands-On: Remove a Secret **529**
Outline 529

Troubleshooting 530

Theory **531**

Theory: fsck Walks the Graph; Repair is Restore Reachability **532**
Invariants 532
What can break 532
How to read this chapter 532

fsck and Repair **533**

fsck and Repair **534**
Outline 534

Common Error Patterns **535**

Common Error Patterns **536**
Outline 536

Recovery Procedures **537**

Recovery Procedures **538**
Outline 538

Hands-On: Corrupted Repo **539**

Hands-On: Corrupted Repo **540**
Outline 540

Hosting Platforms 541

GitHub Monoculture 542

Theory **543**

Theory: Default is Gravity; Lock-in is Extra Objects Outside Git **544**
Invariants 544
What can break 544

How to read this chapter . . . 544

How GitHub Became Default **545**

How GitHub Became Default **546**
Outline . . . 546

Risks of Platform Lock-in **547**

Risks of Platform Lock-in **548**
Outline . . . 548

Regulatory and Geopolitical **549**

Regulatory and Geopolitical **550**
Outline . . . 550

What Beyond GitHub Means **551**

What Beyond GitHub Means **552**
Outline . . . 552

A Framework for This Part **553**

A Framework for This Part **554**
Outline . . . 554

Platform Architecture **555**

Theory **556**

Theory: A Forge is Git Plus Metadata, Identity, and Jobs **557**
Invariants . . . 557
What can break . . . 557
How to read this chapter . . . 557

Git Alone vs Git Hosting **558**

Git Alone vs Git Hosting **559**
Outline . . . 559

Core Platform Components **560**

Core Platform Components **561**
Outline . . . 561

The Protocol Stack **562**

The Protocol Stack **563**
Outline . . . 563

Storage Architectures **564**

Storage Architectures **565**
Outline . . . 565

Authn, Authz, and Identity **566**

Authn, Authz, and Identity **567**
Outline . . . 567

Extensibility **568**

Extensibility **569**
Outline . . . 569

Monolith vs Microservices **570**

Monolith vs Microservices **571**
Outline . . . 571

Operational Complexity **572**

Operational Complexity **573**
Outline . . . 573

GitLab **574**

Theory **575**

Theory: GitLab Sells a Full SDLC on a (Mostly) Monolith **576**
Invariants . . . 576
What can break . . . 576
How to read this chapter . . . 576

History and Philosophy **577**

History and Philosophy **578**
Outline . . . 578

GitLab Architecture **579**

GitLab Architecture **580**
Outline . . . 580

GitLab Feature Set **581**

GitLab Feature Set **582**
Outline . . . 582

Operations and Licensing **583**

Operations and Licensing **584**
Outline . . . 584

Hands-On: GitLab **585**

Hands-On: GitLab **586**
Outline . . . 586

When GitLab Is the Right Choice **587**

When GitLab Is the Right Choice **588**
Outline . . . 588

Gitea and Forgejo **589**

Theory **590**

Theory: A Single Binary is an Architectural Statement **591**
Invariants . . . 591
What can break . . . 591
How to read this chapter . . . 591

Origins and Philosophy **592**

Origins and Philosophy **593**
Two forks, one shape . . . 593
What you get on the first afternoon . . . 593
What this shape refuses to be . . . 594

The Forgejo Fork **595**

The Forgejo Fork **596**
Who holds the name . . . 596
Soft fork, then hard fork . . . 596
What actually changed . . . 596

Gitea vs Forgejo vs GitLab **598**

Gitea vs Forgejo vs GitLab **599**
Same afternoon, different contracts . . . 599
What is not a difference . . . 599
How to read the license row . . . 600

Deployment and Scaling **601**

Deployment and Scaling **602**
One directory is the whole server . . . 602
Where it stops being small . . . 602
Upgrades . . . 603

Hands-On: Gitea and Forgejo **604**

Hands-On: Gitea and Forgejo **605**
Forgejo . . . 605
Push desk-api . . . 606
Tear it down . . . 606
The Gitea file . . . 606
Done when . . . 607

When Gitea or Forgejo Fits **608**

When Gitea or Forgejo Fits **609**
Choose it when . . . 609
Choose something else when . . . 609
A decision you can write down . . . 610

Bitbucket **611**

Theory **612**

Theory: Bitbucket is Git Hosting Inside an Issue Database **613**
Invariants . . . 613
What can break . . . 613
How to read this chapter . . . 613

Cloud vs Data Center **614**

Cloud vs Data Center **615**
Outline . . . 615

Bitbucket Architecture **616**

Bitbucket Architecture **617**
Outline . . . 617

Jira and Atlassian **618**

Jira and Atlassian **619**
Outline . . . 619

Enterprise Features **620**

Enterprise Features **621**
Outline . . . 621

Bitbucket Compared **622**

Bitbucket Compared **623**
Outline . . . 623

Hands-On: Bitbucket Cloud **624**

Hands-On: Bitbucket Cloud **625**
Outline . . . 625

When Bitbucket Fits **626**

When Bitbucket Fits **627**
Outline . . . 627

Gerrit **628**

Theory **629**

Theory: The Change is the Object, Not the Branch **630**
Invariants . . . 630
What can break . . . 630
How to read this chapter . . . 630

What Gerrit Is **631**

What Gerrit Is **632**
Outline . . . 632

Change-Ids, Drafts, Approvals **633**

Change-Ids, Drafts, Approvals **634**
Outline . . . 634

Gerrit and CI **635**

Gerrit and CI **636**
Outline . . . 636

Gerrit Operations **637**

Gerrit Operations **638**
Outline . . . 638

Gerrit in Practice **639**

Gerrit in Practice **640**
Outline . . . 640

Hands-On: Gerrit **641**

Hands-On: Gerrit **642**
Outline . . . 642

When Gerrit Is the Right Choice **643**

When Gerrit Is the Right Choice **644**
Outline . . . 644

SourceHut 645

Theory **646**

Theory: Tools in a UNIX Pipeline, Not a Platform Product **647**
Invariants . . . 647
What can break . . . 647
How to read this chapter . . . 647

SourceHut Philosophy **648**

SourceHut Philosophy **649**
Outline . . . 649

The sr.ht Suite **650**

The sr.ht Suite **651**
Outline . . . 651

SourceHut Architecture **652**

SourceHut Architecture **653**
Outline . . . 653

SourceHut Compared **654**

SourceHut Compared **655**
Outline . . . 655

Hands-On: SourceHut **656**

Hands-On: SourceHut **657**
Outline . . . 657

When SourceHut Makes Sense **658**

When SourceHut Makes Sense **659**
Outline . . . 659

Niche Platforms 660

Theory **661**

Theory: Niche is a Different Default, Not a Toy **662**
Invariants . . . 662
What can break . . . 662

How to read this chapter . . . 662

Codeberg **663**

Codeberg **664**
Outline . . . 664

GitBucket **665**

GitBucket **666**
Outline . . . 666

Gogs **667**

Gogs **668**
Outline . . . 668

Phorge and Arcanist **669**

Phorge and Arcanist **670**
Outline . . . 670

GitHub Enterprise Server **671**

GitHub Enterprise Server **672**
Outline . . . 672

Summary of Niche Platforms **673**

Summary of Niche Platforms **674**
Outline . . . 674

Operating a Platform 675

Identity 676

Theory **677**

Theory: Authn is Who; Authz is What a Ref May Do **678**
Invariants . . . 678
What can break . . . 678
How to read this chapter . . . 678

Password, SSH, OAuth, SAML **679**

Password, SSH, OAuth, SAML **680**
Outline . . . 680

LDAP and Directory Services **681**

LDAP and Directory Services **682**
Outline . . . 682

RBAC and Fine-Grained Permissions **683**

RBAC and Fine-Grained Permissions **684**
Outline . . . 684

SSH Keys at Scale **685**

SSH Keys at Scale **686**
Outline . . . 686

MFA and Hardening **687**

MFA and Hardening **688**
Outline . . . 688

Governance **689**

Theory **690**

Theory: Governance is a Server-Side Predicate on Refs **691**
Invariants . . . 691
What can break . . . 691
How to read this chapter . . . 691

Enforcement Implications **692**

Enforcement Implications **693**
Outline . . . 693

Protecting Branches **694**

Protecting Branches **695**
Outline . . . 695

Monorepo vs Polyrepo (Operator) **696**

Monorepo vs Polyrepo (Operator) **697**
Outline . . . 697

Governance Automation **698**

Governance Automation **699**
Outline . . . 699

CI/CD **700**

Theory **701**

Theory: CI is a Function from Git Events to Jobs **702**
Invariants . . . 702
What can break . . . 702
How to read this chapter . . . 702

Webhooks **703**

Webhooks **704**
Outline . . . 704

Native CI **705**

Native CI **706**
Outline . . . 706

External CI **707**

External CI **708**
Outline . . . 708

Pipelines and Runners **709**

Pipelines and Runners **710**
Outline . . . 710

GitOps Preview **711**

GitOps Preview **712**
Outline . . . 712

Packages **713**

Theory **714**

Theory: A Registry is an Object Store with Language Semantics **715**
Invariants . . . 715
What can break . . . 715
How to read this chapter . . . 715

Container Registries **716**

Container Registries **717**
Outline . . . 717

Language Package Managers **718**

Language Package Managers **719**
Outline . . . 719

Retention and CI Integration **720**

Retention and CI Integration **721**
Outline . . . 721

Self-Hosted Registry Architecture **722**

Self-Hosted Registry Architecture **723**
Outline . . . 723

Security **724**

Theory **725**

Theory: Supply Chain is Trust in What You Did Not Write **726**
Invariants . . . 726
What can break . . . 726
How to read this chapter . . . 726

Signed Commits on the Server **727**

Signed Commits on the Server **728**
Outline . . . 728

Dependency Scanning **729**

Dependency Scanning **730**
Outline . . . 730

SBOM **731**

SBOM **732**
Outline . . . 732

Secrets Detection **733**

Secrets Detection **734**
Outline . . . 734

Supply Chain Attacks **735**

Supply Chain Attacks **736**
Outline . . . 736

Platform-Level Hardening **737**

Platform-Level Hardening **738**
Outline . . . 738

Federation 739

Theory **740**

Theory: Git is Already Distributed; Forges are Not **741**
Invariants . . . 741
What can break . . . 741
How to read this chapter . . . 741

Git as a Decentralization Primitive **742**

Git as a Decentralization Primitive **743**
Outline . . . 743

Federated Hosting **744**

Federated Hosting **745**
Outline . . . 745

Are We Ready? **746**

Are We Ready? **747**
Outline . . . 747

Storage and Scale 748

Theory **749**

Theory: Git Storage is Packs Plus a Database of Metadata **750**
Invariants . . . 750
What can break . . . 750
How to read this chapter . . . 750

Repository Growth **751**

Repository Growth **752**
Outline . . . 752

Storage Options **753**

Storage Options **754**
Outline . . . 754

Clone and Fetch Performance **755**

Clone and Fetch Performance **756**
Outline . . . 756

Database Scaling **757**

Database Scaling **758**
Outline . . . 758

CDN and Geo-Distributed Hosting **759**

CDN and Geo-Distributed Hosting **760**
Outline . . . 760

Benchmarks **761**

Benchmarks **762**
Outline . . . 762

HA and DR 763

Theory **764**

Theory: Backup is Another Clone Plus Metadata Dump **765**
Invariants . . . 765
What can break . . . 765
How to read this chapter . . . 765

Backup Strategies **766**

Backup Strategies **767**
Outline . . . 767

Point-in-Time Recovery **768**

Point-in-Time Recovery **769**
Outline . . . 769

Geo-Redundant Deployments **770**

Geo-Redundant Deployments **771**
Outline . . . 771

Disaster Recovery Planning **772**

Disaster Recovery Planning **773**
Outline . . . 773

Monitoring Repository Health **774**

Monitoring Repository Health **775**
Outline . . . 775

Observability 776

Theory **777**

Theory: If You Cannot See Ref Movement, You Cannot Audit **778**
Invariants . . . 778
What can break . . . 778
How to read this chapter . . . 778

Operational Metrics **779**

Operational Metrics **780**
Outline . . . 780

Log Aggregation **781**

Log Aggregation **782**
Outline . . . 782

Audit Logging **783**

Audit Logging **784**
Outline . . . 784

Tracing User Actions **785**

Tracing User Actions **786**
Outline . . . 786

Compliance Frameworks **787**

Compliance Frameworks **788**
Outline . . . 788

Security Monitoring **789**

Security Monitoring **790**
Outline . . . 790

Enterprise and Air-Gap 791

Theory **792**

Theory: Air-Gap is a Closed Universe of Bytes **793**
Invariants . . . 793
What can break . . . 793

How to read this chapter . . . 793

Air-Gapped Deployments **794**

Air-Gapped Deployments **795**
Outline . . . 795

Multi-Cloud and Hybrid **796**

Multi-Cloud and Hybrid **797**
Outline . . . 797

Data Residency **798**

Data Residency **799**
Outline . . . 799

Support and SLAs **800**

Support and SLAs **801**
Outline . . . 801

Capacity and TCO **802**

Capacity and TCO **803**
Outline . . . 803

Migration and Future 804

Theory **805**

Theory: Migration is Mapping Names, Then Objects, Then Metadata **806**
Invariants . . . 806
What can break . . . 806
How to read this chapter . . . 806

Migration Strategies **807**

Migration Strategies **808**
Outline . . . 808

Preserving History **809**

Preserving History **810**
Outline . . . 810

Interoperability **811**

Interoperability **812**
Outline . . . 812

Emerging Trends 813

Emerging Trends 814
Outline . . . 814

A Decision Framework 815

A Decision Framework 816
Outline . . . 816

The Future of Code Hosting 817

The Future of Code Hosting 818
Outline . . . 818

jj (Jujutsu): Working Copy as a Commit 819

jj (Jujutsu): Working Copy as a Commit 820
Learning goals . . . 820
Thesis . . . 820
Mental model . . . 821
Case 0 — install and version . . . 821
Case 1 — colocated repo: Git and jj share one working tree . . . 821
Case 2 — describe, then `jj new` when the change is "done enough" . . . 822
Case 3 — keep editing, then squash (or edit) . . . 823
Case 4 — undo is not reflog archaeology . . . 823
Interop with Git remotes . . . 823
The trap . . . 824
The boring rule . . . 824
Try this . . . 824
Sources . . . 825

GitHub Actions 826

Fundamentals 827

Theory 828

Theory: A Workflow is a Graph of Jobs Triggered by Git Events 829
Invariants . . . 829
What can break . . . 829
How to read this chapter . . . 829

Automate Tasks with Actions 830

Automate Tasks with Actions 831
Outline . . . 831

Workflow Anatomy **832**

Workflow Anatomy **833**
Outline 833

Runtime and Contexts **834**

Runtime and Contexts **835**
Outline 835

Authoring and Debugging **836**

Authoring and Debugging **837**
Outline 837

Building and Validating Code **838**

Building and Validating Code **839**
Outline 839

Custom Actions and Releases **840**

Theory **841**

Theory: Composite, JS, and Container Actions are Functions **842**
Invariants 842
What can break 842
How to read this chapter 842

Building Custom Actions **843**

Building Custom Actions **844**
Outline 844

Reuse, Matrix, Cache, Secrets **845**

Reuse, Matrix, Cache, Secrets **846**
Outline 846

Releasing Software **847**

Releasing Software **848**
Outline 848

Actions Security Notes **849**

Actions Security Notes **850**
Outline 850

GitOps **851**

What Is GitOps **852**

Theory **853**

Theory: Desired State Lives in Git; the Cluster Converges **854**
Invariants . . . 854
What can break . . . 854
How to read this chapter . . . 854

DevOps Pain Points **855**

DevOps Pain Points **856**
Outline . . . 856

The Four Pillars of GitOps **857**

The Four Pillars of GitOps **858**
Outline . . . 858

Deploy Faster, Recover Faster **859**

Deploy Faster, Recover Faster **860**
Outline . . . 860

Push vs Pull **861**

Push vs Pull **862**
Outline . . . 862

Environment Repositories **863**

Environment Repositories **864**
Outline . . . 864

Trunk vs Pull Requests **865**

Trunk vs Pull Requests **866**
Outline . . . 866

GitOps vs Other Ops **867**

GitOps vs Other Ops **868**
Outline . . . 868

When to Use GitOps **869**

When to Use GitOps **870**
Outline . . . 870

Flux Quickstart **871**

Theory **872**

Theory: Flux is a Set of Controllers that Watch Git **873**
Invariants . . . 873
What can break . . . 873
How to read this chapter . . . 873

Application and Environment Repos **874**

Application and Environment Repos **875**
Outline . . . 875

CD Pipeline with Actions **876**

CD Pipeline with Actions **877**
Outline . . . 877

Flux Operator on Kubernetes **878**

Flux Operator on Kubernetes **879**
Outline . . . 879

The Future of GitOps **880**

The Future of GitOps **881**
Outline . . . 881

Argo CD **882**

Concepts and Architecture **883**

Theory **884**

Theory: Reconciliation is Compare Desired to Actual, Forever **885**
Invariants . . . 885
What can break . . . 885
How to read this chapter . . . 885

Where Argo CD Came From **886**

Where Argo CD Came From **887**
Outline . . . 887

Reconciliation and Immutable Infrastructure **888**

Reconciliation and Immutable Infrastructure **889**
Outline . . . 889

The Reconciliation Loop **890**

The Reconciliation Loop **891**
Outline . . . 891

Argo CD Components **892**

Argo CD Components **893**
Outline . . . 893

Applications and Projects **894**

Applications and Projects **895**
Outline . . . 895

From Git to the Cluster **896**

From Git to the Cluster **897**
Outline . . . 897

Desired vs Actual State **898**

Desired vs Actual State **899**
Outline . . . 899

Install and Lab **900**

Theory **901**

Theory: The Control Plane is Just Another Workload **902**
Invariants . . . 902
What can break . . . 902
How to read this chapter . . . 902

Lab Shape **903**

Lab Shape **904**
Outline . . . 904

K3s and kubectl **905**

K3s and kubectl **906**
Outline . . . 906

Install Methods **907**

Install Methods **908**
Outline . . . 908

UI, CLI, and Clusters **909**

UI, CLI, and Clusters **910**
Outline . . . 910

Applications and Sync 911

Theory **912**

Theory: Sync Policy is When the Loop May Write **913**
Invariants . . . 913
What can break . . . 913
How to read this chapter . . . 913

Your First Application **914**

Your First Application **915**
Outline . . . 915

Sync Policy **916**

Sync Policy **917**
Outline . . . 917

Pruning **918**

Pruning **919**
Outline . . . 919

Retries and Sync Options **920**

Retries and Sync Options **921**
Outline . . . 921

Namespace, Labels, History **922**

Namespace, Labels, History **923**
Outline . . . 923

Recap Exercise **924**

Recap Exercise **925**
Outline . . . 925

Clusters and Diff 926

Theory **927**

Theory: Diff Algorithms Decide What "Out of Sync" Means **928**
Invariants . . . 928
What can break . . . 928

How to read this chapter . . . 928

Cluster Registration **929**

Cluster Registration **930**
Outline . . . 930

Polling vs Webhooks **931**

Polling vs Webhooks **932**
Outline . . . 932

Diff Algorithms **933**

Diff Algorithms **934**
Outline . . . 934

Application Health **935**

Application Health **936**
Outline . . . 936

Hooks and Waves **937**

Theory **938**

Theory: A Sync is a Phased Transaction with Optional Jobs **939**
Invariants . . . 939
What can break . . . 939
How to read this chapter . . . 939

Sync Phases **940**

Sync Phases **941**
Outline . . . 941

Names and Sync Waves **942**

Names and Sync Waves **943**
Outline . . . 943

When a Hook Fails **944**

When a Hook Fails **945**
Outline . . . 945

Hook Anti-Patterns **946**

Hook Anti-Patterns **947**
Outline . . . 947

Users and RBAC 948

Theory 949

Theory: Local Users are Passwords; RBAC is Casbin Strings 950
Invariants . . . 950
What can break . . . 950
How to read this chapter . . . 950

Creating a User 951

Creating a User 952
Outline . . . 952

The p Line and the g Line 953

The p Line and the g Line 954
Outline . . . 954

Verifying Grants 955

Verifying Grants 956
Outline . . . 956

Helm 957

Theory 958

Theory: Helm is a Function from Chart+Values to YAML 959
Invariants . . . 959
What can break . . . 959
How to read this chapter . . . 959

Helm vs Plain Resources 960

Helm vs Plain Resources 961
Outline . . . 961

Charts and Repositories 962

Charts and Repositories 963
Outline . . . 963

Deploying Charts and Values 964

Deploying Charts and Values 965
Outline . . . 965

Helm Hooks vs Argo CD Hooks 966

Helm Hooks vs Argo CD Hooks **967**
Outline . . . 967

ApplicationSets and Patterns 968

Theory **969**

Theory: ApplicationSets are Generators of Applications **970**
Invariants . . . 970
What can break . . . 970
How to read this chapter . . . 970

ApplicationSets **971**

ApplicationSets **972**
Outline . . . 972

App of Apps **973**

App of Apps **974**
Outline . . . 974

Multi-Branch CI/CD **975**

Multi-Branch CI/CD **976**
Outline . . . 976

Build to Application Update **977**

Build to Application Update **978**
Outline . . . 978

Notifications, Secrets, and Blue-Green 979

Theory **980**

Theory: Notifications Observe the Loop; Secrets Must Not Live as Plain Desired State **981**
Invariants . . . 981
What can break . . . 981
How to read this chapter . . . 981

Slack Notifications **982**

Slack Notifications **983**
Outline . . . 983

Secret Management **984**

Secret Management **985**
Outline . . . 985

Blue-Green with Argo CD **986**

Blue-Green with Argo CD **987**
Outline . . . 987

Argo vs Alternatives **988**

Theory **989**

Theory: Engines Differ in UX and Ecosystem, Not in Pull-vs-Push Physics **990**
Invariants . . . 990
What can break . . . 990
How to read this chapter . . . 990

Argo CD vs the Alternatives **991**

Argo CD vs the Alternatives **992**
Outline . . . 992

Choosing a GitOps Engine **993**

Choosing a GitOps Engine **994**
Outline . . . 994

Argo Rollouts **995**

Introducing Rollouts **996**

Theory **997**

Theory: Progressive Delivery is Reducing Blast Radius Over Time **998**
Invariants . . . 998
What can break . . . 998
How to read this chapter . . . 998

Beyond RollingUpdate **999**

Beyond RollingUpdate **1000**
Outline . . . 1000

Canary and Blue-Green Fundamentals **1001**

Canary and Blue-Green Fundamentals **1002**
Outline . . . 1002

Gradual Releases and Risk **1003**

Gradual Releases and Risk **1004**
Outline 1004

Resources and Lifecycle **1005**

Theory **1006**

Theory: A Rollout Owns Stable and Canary ReplicaSets **1007**
Invariants 1007
What can break 1007
How to read this chapter 1007

Rollout vs Deployment **1008**

Rollout vs Deployment **1009**
Outline 1009

Stable and Canary Versions **1010**

Stable and Canary Versions **1011**
Outline 1011

AnalysisTemplates and Experiments **1012**

AnalysisTemplates and Experiments **1013**
Outline 1013

Rollouts Lab **1014**

Theory **1015**

Theory: The Lab is a Tiny Cluster that Still Reconciles from Git **1016**
Invariants 1016
What can break 1016
How to read this chapter 1016

Rollouts Lab Stack **1017**

Rollouts Lab Stack **1018**
Outline 1018

Manifests and Verify **1019**

Manifests and Verify **1020**
Outline 1020

First Canary **1021**

Theory **1022**

Theory: A Step is a Promise About Traffic or Replicas, Then a Pause **1023**
Invariants . . . 1023
What can break . . . 1023
How to read this chapter . . . 1023

Canary Strategy and Steps **1024**

Canary Strategy and Steps **1025**
Outline . . . 1025

Promote, Abort, Inspect **1026**

Promote, Abort, Inspect **1027**
Outline . . . 1027

Istio Canaries **1028**

Theory **1029**

Theory: Traffic Split is a Mesh API, Not a Replica Count **1030**
Invariants . . . 1030
What can break . . . 1030
How to read this chapter . . . 1030

Istio Traffic Management **1031**

Istio Traffic Management **1032**
Outline . . . 1032

Subsets and Weighted Traffic **1033**

Subsets and Weighted Traffic **1034**
Outline . . . 1034

Replica-Based vs Traffic-Based **1035**

Replica-Based vs Traffic-Based **1036**
Outline . . . 1036

Analysis and Prometheus **1037**

Theory **1038**

Theory: Analysis is a Predicate on Metrics Over a Window **1039**
Invariants . . . 1039

What can break . . . 1039
How to read this chapter . . . 1039

AnalysisTemplate and AnalysisRun **1040**

AnalysisTemplate and AnalysisRun **1041**
Outline . . . 1041

Success, Failure, Aborts **1042**

Success, Failure, Aborts **1043**
Outline . . . 1043

Inline vs Background Analysis **1044**

Inline vs Background Analysis **1045**
Outline . . . 1045

Experiments 1046

Theory **1047**

Theory: An Experiment is a Bounded Extra Version **1048**
Invariants . . . 1048
What can break . . . 1048
How to read this chapter . . . 1048

Running Versions Side by Side **1049**

Running Versions Side by Side **1050**
Outline . . . 1050

Analysis During Experiments **1051**

Analysis During Experiments **1052**
Outline . . . 1052

Experiments vs AnalysisTemplates **1053**

Experiments vs AnalysisTemplates **1054**
Outline . . . 1054

HPA and Canary 1055

Theory **1056**

Theory: HPA Owns Replicas; Canary Steps Also Own Replicas **1057**
Invariants . . . 1057
What can break . . . 1057

How to read this chapter . . . 1057

HPA with Replica-Based Canaries **1058**

HPA with Replica-Based Canaries **1059**
Outline . . . 1059

Metrics Server and Canary Scaling **1060**

Metrics Server and Canary Scaling **1061**
Outline . . . 1061

setCanaryScale **1062**

setCanaryScale **1063**
Outline . . . 1063

Blue-Green 1064

Theory **1065**

Theory: Blue-Green is Two Stacks and One Cutover **1066**
Invariants . . . 1066
What can break . . . 1066
How to read this chapter . . . 1066

Active and Preview Services **1067**

Active and Preview Services **1068**
Outline . . . 1068

Promotion Analysis **1069**

Promotion Analysis **1070**
Outline . . . 1070

Promote and Rollback **1071**

Promote and Rollback **1072**
Outline . . . 1072

Rollout Notifications 1073

Theory **1074**

Theory: Notifications are Observers of Rollout Events **1075**
Invariants . . . 1075
What can break . . . 1075
How to read this chapter . . . 1075

Notification Architecture **1076**

Notification Architecture **1077**
Outline . . . 1077

Triggers, Templates, Slack **1078**

Triggers, Templates, Slack **1079**
Outline . . . 1079

Troubleshooting Notifications **1080**

Troubleshooting Notifications **1081**
Outline . . . 1081

Migrating to Rollouts 1082

Theory **1083**

Theory: workloadRef Lets a Rollout Adopt a Deployment **1084**
Invariants . . . 1084
What can break . . . 1084
How to read this chapter . . . 1084

Migration and workloadRef **1085**

Migration and workloadRef **1086**
Outline . . . 1086

Direct Conversion **1087**

Direct Conversion **1088**
Outline . . . 1088

Canary Migration **1089**

Canary Migration **1090**
Outline . . . 1090

StatefulSet Limitations **1091**

StatefulSet Limitations **1092**
Outline . . . 1092

Capstone 1093

Theory **1094**

Theory: One Loop from Commit to Careful Traffic **1095**
Invariants . . . 1095
What can break . . . 1095
How to read this chapter . . . 1095

A Working Mental Model **1096**

A Working Mental Model **1097**
Outline . . . 1097

Larger Environments **1098**

Larger Environments **1099**
Outline . . . 1099

Appendices **1100**

Understanding Questions and Answers **1101**

Understanding Questions and Answers **1102**
Outline . . . 1102

Git Competency Checklists **1103**

Git Competency Checklists **1104**
Outline . . . 1104

Real-World Scenario Playbook **1105**

Real-World Scenario Playbook **1106**
Outline . . . 1106

Lab Environment Reference **1107**

Lab Environment Reference **1108**
Outline . . . 1108

Command Quick Reference and Glossary **1109**

Command Quick Reference and Glossary **1110**
Outline . . . 1110

Introduction

VCS

Architectures, Platforms, Operations, and GitOps

A practical path from your first commit to running a forge and shipping with GitOps. You start as a Git user, then you install and compare hosting platforms, then you operate them, then you let Git drive the cluster.

```
VCS
   Git foundations      mental model → install → first repository
   Everyday workflows   commits, branches, rebase, remotes, hooks
   Power skills         internals, reflog, bisect, repair
   Hosting platforms    GitHub lock-in, GitLab, Forgejo, Gerrit, …
   Operating a forge    identity, CI, storage, HA, air-gap
   GitHub Actions       workflows as a product
   GitOps               Flux, Argo CD, Argo Rollouts
```

The sidebar is the full TOC (chapters 1–66 plus appendices). **Every chapter starts with Theory** (invariants, a diagram, what can break). **Parts I–II** have prose for the original pages and for chapter 7 (reset, restore, and checkout). The other newer core sub-pages are still stubs. Later parts are stubs until a writing pass fills them, except chapter 25 (Gitea and Forgejo), which is prose. The map lives in `content/_planning/SYLLABUS.md`.

Who this book is for

Software engineers, DevOps, platform, and SRE readers on **Linux**. You can start with no Git. Terms are defined on first use. Depth ramps: user → installer → admin → operator → GitOps practitioner.

Commands, paths, and package names assume a POSIX shell on Linux (`bash` is fine). Examples use `/home/…`, `~/.gitconfig`, and distro packages (`pacman`, `apt`, `dnf`). The Git commands themselves are the same on other Unix systems; this book does not walk macOS or Windows.

What you will master

- Git's object model, everyday commands, and recovery
- How a Git hosting platform is built, and how GitLab, Forgejo, Bitbucket, Gerrit, SourceHut, and GitHub differ
- Identity, CI, packages, supply-chain controls, backups, and air-gap
- GitHub Actions as one CI product
- GitOps with Flux and Argo CD, then progressive delivery with Argo Rollouts

How to read this so it sticks

Work front to back in Part I. Type every command in a throwaway directory. Later parts assume you can commit, branch, and push. Platform labs reuse the same three desk repositories: `desk-web`, `desk-api`, `desk-infra`.

The companion kit

Required for Parts I–II: Git 2.45 or newer on Linux (examples checked against 2.55), a terminal, and a text editor (`nvim` is the default in config examples).

Required from the GitOps parts onward: Docker, kind or K3s, `kubectl`, and Helm. Install those when you reach Part VII. You do not need a cluster to finish Git foundations.

Conventions

Commands are complete. Run them from the directory the page names. Output is representative; hashes and timestamps on your machine will differ.

Heredocs create files without opening an editor:

```
cat > README.md <<'EOF'
# desk-api
EOF
```

The quoted `'EOF'` keeps `$variables` and backticks literal. Drop the quotes only when you want the shell to expand them.

A **repository** (repo) is a directory Git is tracking. A **forge** is a Git hosting platform: the website and services around repositories (issues, pull requests, CI, packages). A **remote** is another copy of a repository you fetch from and push to.

Acknowledgements

K19G. Built as part of the books library.

Git Foundations

Mental Model

Theory

Theory: Snapshots, Names, and a Graph

This chapter is the model the rest of the book assumes. Commands later only add objects and move names.

Invariants

1. A **commit** names a complete tree, not a patch. Unchanged files reuse blob hashes.
2. History is a **DAG**: each commit points at parent(s). Nothing in the object store is rewritten in place.
3. A **branch** is a movable name for one commit. `HEAD` is a name for "where I am," usually a pointer to a branch name.
4. Everyday work uses **three trees**: working directory, index, `HEAD`. Status is the report of how they differ.
5. **Distribution** means every clone is a repository. A forge is extra software around that fact.

```
names (refs)                    objects (content-addressed)
HEAD → main → C                 C  commit  → tree T → blobs
              ↑                 B  commit
              B ← A             A  commit
```

If the name moves and the objects stay, you have not lost history; you have lost a pointer. Reflog (chapter 17) is the local journal of pointer motion.

Why this model

Centralized systems made the server the only full history. Git inverts that: the laptop is complete. Cheap branches follow from "a branch is a file containing a hash." Integrity follows from hashing content. Hosting platforms, CI, and GitOps cannot escape these invariants; they only add names, ACLs, and jobs.

What can break

- Treating Git like a file-sync tool (last write wins, no merge).
- Treating GitHub as Git (issues, Actions, and PRs are forge objects).
- Committing on a **detached HEAD** and switching away without a branch name.
- Assuming `git pull` is a primitive; it is fetch plus integrate.

How to read this chapter

Landscape is *why a VCS*. DAG, three trees, objects, refs, and distribution are *how Git thinks*. Detached HEAD and revision ranges are the first notations you will type. Install and the first repository apply this model on Linux.

The Version Control Landscape

The Version Control Landscape

After this page you can say what version control is for, why Git is the usual tool, and what a forge is.

Why you need a history

Without version control, a desk project grows copies:

```
desk-api/
   app.py
   app_backup.py
   app_final.py
   app_final_v2.py
   app_really_final_fixed.py
```

You lose which file is current, who changed the login line, and any way to undo Tuesday without also undoing Wednesday. Two people editing `app.py` overwrite each other.

A **version control system** (VCS) keeps one tree of files, a history of snapshots, and a way to merge parallel work.

Three generations

Generation	Idea	Examples
Local	History on one disk	SCCS, RCS
Centralized	One server is the repository	CVS, Subversion, Perforce
Distributed	Every clone is a full repository	Git, Mercurial

Local tools recorded deltas next to the files. Fine for one person on one machine.

Centralized tools put the real history on a server. You check out a working copy. Committing needs the network. If the server dies, history you do not have locally is gone.

Distributed tools give every clone the whole history. `git commit` is a local operation. A **remote** is another copy you push to when you choose. That is why Git works on a train, and why rewriting commits you already shared is a people problem.

Why Git

Git won the ecosystem. The properties that matter for this book:

- A commit is a **snapshot** of the whole tree, not a pile of patches as the primary model.
- A **branch** is a cheap name for one commit. Merging and rebasing are graph operations.
- Object names are **hashes** of content. Bit flips show up as missing or mismatched objects.
- Log, diff, commit, branch, and merge all work without a network.
- Hosting platforms speak Git remotes. Learning Git is not loyalty to one website.

Git vs Subversion, the comparison that still comes up:

	Git	Subversion
Architecture	Distributed	Centralized
Offline commit	Yes	No
Branch	A name on a commit	A copy in the tree
Typical remote	Any number	One canonical server

Mercurial is also distributed. Git is what forges, CI systems, and GitOps engines speak.

What a forge is

A **repository** is a directory Git is tracking (the `.git` folder plus your files).

A **forge** is a Git hosting platform: the website and services around repositories. Issues, pull requests, CI, packages, and user accounts live on the forge. GitHub, GitLab, Forgejo, Bitbucket, Gerrit, and SourceHut are forges. This book treats Git first, then the platforms.

Git is the tool. The forge is the product you log into. GitOps (later) is the idea that the same Git history should drive what runs in production.

What Git is not

- Not a backup by itself. A single laptop clone can die; you still need remotes and backups.
- Not a file-sync tool. Dropbox overwrites. Git records snapshots and asks you to merge.
- Not GitHub. You can use Git for years with a forge you run yourself, or with no forge at all.

Next

The next pages are how Git thinks: a graph of commits, three trees on disk, four object types, names for those objects, and what "distributed" means for a server.

The Directed Acyclic Graph

The Directed Acyclic Graph

After this page you can draw a tiny Git history as a graph and say what a branch is.

Snapshots with parents

A **commit** is a snapshot of the whole project at one instant, plus metadata (author, message, time) and a pointer to the commit that came before it — its **parent**.

Two commits in a row:

```
A  ←  B
```

`B`'s parent is `A`. Time flows left to right in this book's diagrams; the parent pointer points backward, toward history.

A chain of commits is a **directed acyclic graph** (DAG): directed because each commit points at parents, acyclic because a commit cannot be its own ancestor.

A branch is a name

A **branch** is a movable name for one commit. `main` is a file that contains a hash. When you commit, Git writes a new snapshot whose parent is the current tip, then slides the name forward.

```
A  ←  B  ←  C     main
```

`main` now names `C`. `A` and `B` are still there. Nothing was overwritten.

A second line of work

Start a branch named `feature` while `main` is at `C`, then add two commits:

```
A  ←  B  ←  C          main
            ↑
            D  ←  E    feature
```

`C` has two children. Git is fine with that. A **merge commit** has two parents:

```
A  ←  B  ←  C  ←  M     main
            ↑     ↑
            D  ←  E     feature
```

`M` ties the two lines together. **Rebase** (chapter 9) copies `D` and `E` onto a new parent instead. New hashes, similar diffs. You will do that after first-repository muscle memory is in place.

HEAD

HEAD is "where I am." Most of the time it points at a branch name, and the branch name points at a commit:

```
HEAD → refs/heads/main → C
```

Commands that "move HEAD" are moving that name, or (when HEAD is **detached**) pointing HEAD at a raw commit hash. Detached HEAD shows up when you check out a tag or a hash. New commits on a detached HEAD are easy to lose when you switch away. Create a branch before you leave, or find them later with the reflog (chapter 17).

```
git log --oneline --graph --all
git rev-parse HEAD main
git status
```

`git status` says `HEAD detached at …` when it is.

Look at the graph

Once you have a repository (next chapter's neighbor, chapter 3):

```
git log --oneline --graph --decorate --all
```

`--decorate` prints the names (`main`, `HEAD`, tags) next to the commits they point at. The graph is the truth. Commands add objects and move names. If you hold the graph, `reset`, `rebase`, and `reflog` stop feeling like magic.

The Three-Tree Model

The Three-Tree Model

After this page you can name Git's three snapshots of the same project and say which command copies between them.

Three snapshots, one project

Everyday Git is three versions of the same tree:

Name	On disk	Role
Working directory	your files	What you edit
Index (staging area)	`.git/index`	What the *next* commit will contain
HEAD commit	an object in `.git/objects`	Last committed snapshot on this branch

```
working directory  --git add-->  index  --git commit-->  HEAD
                 <--git restore--      <--git restore --staged / git reset--

 ~/desk/desk-api/              .git/index                .git/objects
 README.md, src/…              next commit               last commit (HEAD)
```

- `git add` copies working-tree bytes into the index (and writes **blob** objects).
- `git commit` writes a **tree** from the index, then a **commit** pointing at that tree and at the current HEAD, then slides the branch name forward.
- `git restore <file>` copies from the index (or a commit) onto the working tree.
- `git restore --staged <file>` copies from HEAD into the index, leaving your working-tree edit in place.

There is no fourth "GitHub tree." A remote is another repository's names plus objects you have fetched.

Why the index exists

The index lets the next commit be a chosen subset of what is on disk. You can edit five files and commit two.

```
echo "ok" >> README.md
echo "wip" >> notes.txt
git add README.md
git status
# README.md is staged; notes.txt is not
```

`git commit -a` copies **already tracked** files from the working tree into the index and commits. New files still need `git add`. Prefer explicit `git add` until that distinction is boring.

`status` is the three-tree report

```
git status
```

Git compares:

- working tree vs index → "Changes not staged for commit"
- index vs HEAD → "Changes to be committed"
- untracked files → paths the index does not know

When all three match, `status` is clean. That is the happy state after a commit.

`reset` in one sentence

`git reset` moves HEAD and optionally the index and working tree. Soft / mixed / hard are which of the three trees follow. Chapter 7 (reset) and chapter 17 (reflog) use this on purpose. For now, prefer `git restore` to throw away an unstaged edit, and `git restore --staged` to unstage.

Check it

```
git status
git diff              # working tree vs index
git diff --staged     # index vs HEAD
```

If `git diff` is empty and `git diff --staged` is not, the next commit already contains the change. If both are empty and `status` still lists a file, it is untracked — `git add` it.

Object Types

Object Types

After this page you can name Git's four object types and say what a hash is identifying.

Content-addressed storage

Git stores objects in `.git/objects`. The name of an object is a **hash** of its content. Change one byte of a file → new blob, new tree, new commit. Unchanged files keep the same blob hash across commits, which is why a one-line edit is cheap.

Historically the hash is SHA-1. Repositories can use SHA-256. Commands look the same. This book prints short hashes (`a1b2c3d`) the way `git log --oneline` does.

Four types

Type	Payload
blob	file bytes (no filename)
tree	name → object (a blob or a nested tree)
commit	a tree + parent commit(s) + author + message
tag	annotated pointer + message + optional signature

Filenames live in **trees**, not in blobs. Rename a file: new tree, same blob.

```
commit C
  tree T
    README.md  → blob B1
    src/       → tree T2
      main.py  → blob B2
  parent → commit B
  author Ada Desk <ada@example.com>
  message "Add ping"
```

A **lightweight tag** is only a name (a ref). An **annotated tag** is a real tag object. Chapter 12 covers tagging in daily use.

Packfiles are compression

Git also stores **packfiles**: many objects packed together with deltas, for disk. That is compression, not the data model. You still think in snapshots. Chapter 16 opens the object store with `git cat-file`. You can peek now:

```
git cat-file -t HEAD          # usually "commit"
git cat-file -p HEAD          # the commit payload
git rev-parse HEAD^{tree}     # the tree that commit names
```

`-p` pretty-prints. `-t` prints the type.

Integrity

Because names are hashes, a silent bit flip in a blob does not match the name the tree recorded. `git fsck` (chapter 21) walks the graph and reports missing or malformed objects. Forges and GitOps engines rely on this: the commit you pulled is the tree the author hashed.

What this buys a server

A Git hosting platform is, at minimum, a store of these objects plus names (branches, tags) plus auth. Chapter 23 is that architecture. The object model you just learned is the constraint the server cannot wish away: clones fetch objects and update names.

References: Branches, Tags, and HEAD

References: Branches, Tags, and HEAD

After this page you can say what a ref is, where it lives, and how HEAD relates to a branch.

A ref is a name for a hash

A **ref** (reference) is a name that points at a commit (or, for annotated tags, at a tag object). Git stores them as files under `.git/refs/` and, when there are many, in `.git/packed-refs`.

Ref	Typical path	Meaning
Branch	`.git/refs/heads/main`	Movable name for a line of work
Tag	`.git/refs/tags/v1.0`	Name meant to stay put
Remote-tracking	`.git/refs/remotes/origin/main`	Last-seen tip of a branch on a remote
HEAD	`.git/HEAD`	Where you are

```
git rev-parse main
git rev-parse HEAD
cat .git/HEAD
# ref: refs/heads/main
```

`git rev-parse` turns a name into a hash. Use it when a diagram and the disk disagree in your head.

Branches move, tags mostly do not

```
git branch                  # local branches
git branch feature          # new name at current HEAD
git switch feature          # HEAD now points at feature
git switch -c hotfix        # create and switch
```

`git switch` is the modern command for changing branches (`git checkout` still works; this book uses `switch` for branches and `restore` for files).

A tag is a bookmark:

```
git tag v0.1                # lightweight: a name at HEAD
git tag -a v0.1 -m "first"  # annotated: a tag object
```

Moving a published tag surprises people who already fetched it. Moving a branch is normal.

HEAD is usually a pointer to a pointer

```
.git/HEAD          →  ref: refs/heads/main
.git/refs/heads/main →  a1b2c3d4….
```

Commit while on `main`: Git writes a new commit whose parent is `a1b2c3d4…`, then overwrites `refs/heads/main` with the new hash. HEAD did not change its text; the branch it names did.

Detached HEAD: `.git/HEAD` contains a raw hash instead of `ref: refs/heads/…`. You get there with `git switch --detach`, or by checking out a tag. To keep work from a detached HEAD:

```
git switch -c keep-this
```

That creates a branch at the current commit and attaches HEAD to it.

Remote-tracking names

After you add a remote named `origin` and fetch (chapter 11):

```
refs/remotes/origin/main
```

That is *your* last photograph of `main` on that remote. It updates on fetch. You do not commit onto it. Your local `main` is a separate name you push.

Plumbing vs porcelain

`git branch`, `git switch`, `git tag` are **porcelain** (human commands). `git update-ref` and `git symbolic-ref` are **plumbing** (the same operations, no sugar). This book stays on porcelain until internals (chapter 16). Knowing refs are files is enough to understand why creating a branch is cheap: Git writes a 41-byte file.

The Distributed Model

The Distributed Model

After this page you can explain why every clone is a repository, and what that implies for a hosting server.

Every clone is a full copy

When you `git clone`, you get:

- every commit reachable from the branches and tags you fetched
- a working tree checked out at one branch (usually `main`)
- a remote named `origin` pointing at the URL you cloned

There is no "thin client." `git log`, `git diff`, `git commit`, `git branch`, and `git merge` talk to `.git/` on your disk. The network is for **fetch** (download objects and update remote-tracking names) and **push** (upload objects and ask the other side to move a name).

```
your laptop                          a forge

  .git/objects        fetch/push     .git/objects
  refs/heads/main   ←       →   refs/heads/main
  refs/remotes/                    (plus issues,
    origin/main                     CI, packages)
```

Both sides are Git repositories. The forge adds a website, accounts, and extra services. Git itself does not need them.

Implications

Offline work is real. You can commit on a plane. Push when you have a network.

Several remotes are normal. `origin` is a convention, not a law. You can push the same `desk-api` history to a laptop USB clone, a Forgejo server, and GitHub. Chapter 22 uses that as an independence move.

The server does not own history uniquely. If the forge vanishes, every up-to-date clone still has the Git objects. What you lose are issues, pull-request comments, CI logs, and packages — the forge's extras. That split is the rest of this book.

Rewriting shared history is a social operation. `git commit --amend` and rebase change hashes. On your laptop before anyone fetched, that is fine. After others based work on those hashes, you are asking them to reset. Chapter 9 is the skill; this page is the reason it matters.

Fetch is not pull. `git fetch` updates `origin/main` on disk. `git pull` is fetch plus a merge or rebase into your current branch. Chapter 11 separates them. Prefer fetch until pull's extra step is a choice you can name.

What a hosting platform must speak

A Git server has to:

1. Store objects and refs.
2. Speak the Git protocol over SSH and/or HTTP (smart protocol: the client and server negotiate which objects to send).
3. Decide who may move which refs (authorization).
4. Run **hooks** on receive (the server-side programs that can reject a push).

Everything else — pull requests, CI, registries, SAML — is forge software on top of that. Chapter 23 draws the boxes. Chapter 3 stays on your laptop: `git init`, add, commit.

Detached HEAD

Detached HEAD

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- What detached HEAD is, how you get there, how to keep the work.

> **Note**
>
> Core Git depth (Linux host). See `content/_planning/SYLLABUS.md`.

Revisions and Ranges

Revisions and Ranges

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- `HEAD~` vs `HEAD^`, `A..B` vs `A...B`, `@{u}`, reflog syntax preview.

> **Note**
>
> Core Git depth (Linux host). See `content/_planning/SYLLABUS.md`.

Install and Config

Theory

Theory: Identity and Config are Data, Not Login

Git does not have accounts. It has **key-value config** and **author strings baked into commits**. A forge later *matches* those strings to a user.

Invariants

1. Config is three layers: system (`/etc/gitconfig`), global (`~/.gitconfig`), local (`.git/config`). **Later overrides earlier.**
2. `user.name` and `user.email` are copied into every commit object. Changing config does not rewrite old commits.
3. Authentication to a **remote** (SSH key or HTTPS token) is separate from commit identity.
4. `init.defaultBranch` only affects **new** repositories.

```
/etc/gitconfig          machine
~/.gitconfig            you
.git/config             this repo     ← wins

user.email    commit.author     (history)
SSH key       git@host          (transport)
```

Why layers

One person, two emails (home and work) is a local or `includeIf` problem, not a second Git install. One laptop, several forges, is several keys in `~/.ssh/config`, not several `user.name`s.

What can break

- Commits with `$USER@$HOSTNAME` because identity was never set (`user.useConfigOnly` refuses that).
- Pushing with HTTPS using an account password; most forges want a **token**.
- Setting `core.autocrlf` on Linux and fighting LF files.
- Assuming `core.editor` is optional and then hanging in `vim` on `git commit` with no `-m`.

How to read this chapter

Install is PATH and version. Layers are which file won (`--show-origin`). Identity is what history will say. SSH vs HTTPS is how a remote will know you. Editor, aliases, and updates are the Linux daily driver.

Installing Git

Installing Git

After this page you have a current `git` on your `PATH` on Linux and can print its version.

Check what you already have

```
git --version
which -a git
```

This book wants **2.45 or newer** on **Linux**. Examples were checked against **2.55**. You need `git switch`, `git restore`, and (later) SSH commit signing.

Most desktop distros already ship a new enough Git. If `which git` prints `/usr/bin/git` and the version is 2.45+, you are done.

Install from the distro

Arch Linux and Omarchy

```
pacman -Q git
# missing or too old:
sudo pacman -S git
```

Debian and Ubuntu

```
sudo apt update
sudo apt install git
```

Fedora

```
sudo dnf install git
```

Enterprise images sometimes freeze Git at 2.25. If `git --version` is older than 2.45, use your distro's backports or a current tarball from git-scm.com so `switch` and `restore` exist.

Line endings and the filesystem

Linux stores **LF** line endings and executable bits. Leave `core.autocrlf` unset. Leave `core.fileMode` at the default (`true`) so `chmod +x` is recorded. Those defaults are why hooks and scripts in later chapters work with `chmod +x hook.sh`.

What else you need for Parts I–II

Tool	Role
A Linux terminal	Where you run `git` (`bash` or another POSIX shell)
A text editor	`nvim` in this book's config examples; any editor is fine

Docker, Kubernetes, and a forge account wait until later parts.

Set `core.editor` once you know which editor you type in (next page). Until then, `git commit -m "message"` avoids opening one.

Confirm

```
git --version
which git
echo "$HOME"
```

`$HOME` should be `/home/<you>`. Then set identity and the three config layers on the next page.

Config Layers

Config Layers

After this page you can say which file a Git setting comes from, and set the handful of keys this book assumes.

Three layers

Layer	Flag	Typical file	Who
System	`--system`	`/etc/gitconfig`	every user on the machine
Global	`--global`	`~/.gitconfig`	you, every repo
Local	`--local` (default)	`.git/config`	this repo only

Later keys **override** earlier ones. Work email in a company repo is a local `user.email`, not a second global identity you forget to switch.

```
wins

      .git/config          local    this repo
      ~/.gitconfig         global   this user
      /etc/gitconfig       system   this machine

reads bottom → top; last value for a key wins
```

```
git config --list --show-origin --show-scope
git config --show-origin user.email
```

`--show-origin` prints the file that won. `--unset` removes a key at the layer you name:

```
git config --global --unset core.editor
```

Set the global layer once

```
git config --global init.defaultBranch main
git config --global core.editor "nvim"
git config --global pull.ff only
git config --global rerere.enabled true
```

```
git config --global diff.algorithm histogram
git config --global merge.conflictstyle zdiff3
```

Key	Why
`init.defaultBranch main`	`git init` creates `main`, not `master`
`core.editor`	`git commit` without `-m` opens this editor (`code --wait`, `vim`, `nvim`)
`pull.ff only`	`git pull` fast-forwards or fails; you merge or rebase on purpose
`rerere.enabled`	reuse recorded conflict resolutions
`diff.algorithm histogram`	better diffs on refactored files
`merge.conflictstyle zdiff3`	conflict markers include the common ancestor

`pull.ff only` means a pull that would create a merge commit fails instead. That is safer than the old default. Chapter 11 is remotes; this setting is safe to apply now.

Leave `core.fileMode` at the default so Git records executable bits. Set it to `false` only on a filesystem that cannot store those bits.

Repo-only overrides

```
cd desk-api
git config user.email "ada@work.example"
# writes .git/config
```

Inspect:

```
git config --local --list --show-origin
```

`includeIf` for home vs work

```
# ~/.gitconfig
[user]
    name = Ada Desk
    email = ada@example.com

[includeIf "gitdir:~/work/"]
    path = ~/.gitconfig-work
```

```
# ~/.gitconfig-work
[user]
```

```
    email = ada@work.example
```

Any repo under `~/work/` picks up the work email without a per-repo command. Paths in `gitdir:` should be absolute or start with `~/`.

See the result

```
git config --list --show-origin --show-scope | less
```

Identity (`user.name`, `user.email`) is the next page. Those two keys go into every commit.

Identity, Email, and Essential Settings

Identity, Email, and Essential Settings

After this page every commit you make records a name and email you chose.

Identity is metadata, not a login

`user.name` and `user.email` are written **into every commit**. They are not a password and not a forge account. Forges *match* the email to an account when they display history. A typo produces "unknown author" on the website. `git commit` still succeeds.

```
~/.gitconfig
  user.name  = Ada Desk
  user.email = ada@example.com


      commit object
        tree    …
        parent  …
        author  Ada Desk <ada@example.com>  1774… +0530
        committer Ada Desk <ada@example.com> …
        message …
```

```
git config --global user.name "Ada Desk"
git config --global user.email "ada@example.com"
```

Use the email the forge should associate with you. For a first laptop, any address you control is fine. For work, the company address, often via `includeIf` from the previous page.

Refuse to commit if identity is missing, instead of guessing `$USER@$HOSTNAME`:

```
git config --global user.useConfigOnly true
```

Check what will be recorded

```
git config user.name
git config user.email
git config --show-origin user.email
```

After you have a commit (chapter 3):

```
git log -1 --format=full
```

`Author:` is `user.name <user.email>` from the time of the commit. Changing `~/.gitconfig` later does not rewrite old commits. Chapter 6 is amending; chapter 20 is signing.

Signing is later

Signed commits (SSH or GPG) prove the commit came from a key you hold. Set identity here first. Signing is chapter 20.

Author vs committer

A commit has two people fields: **author** (who wrote the patch) and **committer** (who made the commit object). They match until you cherry-pick, rebase, or apply a patch from someone else. `git log` shows the author by default. You do not need to set `GIT_COMMITTER_NAME` for Part I.

Essential settings, one list

If you jumped here from install, this is the whole global block this book expects:

```
git config --global user.name "Ada Desk"
git config --global user.email "ada@example.com"
git config --global user.useConfigOnly true
git config --global init.defaultBranch main
git config --global core.editor "nvim"
git config --global pull.ff only
```

Replace the name, email, and editor with yours. Then pick SSH or HTTPS on the next page.

SSH vs HTTPS

SSH vs HTTPS

After this page you can authenticate to a forge over SSH or HTTPS and know which URL form you are looking at.

Two ways to talk to a remote

	SSH	HTTPS
URL	`git@host:owner/repo.git`	`https://host/owner/repo.git`
Secret	Private key on disk	Password, personal access token, or credential helper
Typical use	Daily push/fetch on a machine you own	CI, quick clone, hosts that block SSH

Both speak the Git smart protocol. The difference is how the server decides who you are. Part I does not require a forge. Set this up before chapter 11 so the first push is not a scavenger hunt.



SSH keys

Generate an **ed25519** key if you do not have one:

```
ssh-keygen -t ed25519 -C "ada@example.com" -f ~/.ssh/id_ed25519
```

Press Enter for no passphrase, or set one (then `ssh-agent` remembers it for the session):

```
eval "$(ssh-agent -s)"
ssh-add ~/.ssh/id_ed25519
```

Public key (this is what you paste into the forge):

```
cat ~/.ssh/id_ed25519.pub
```

The private file `~/.ssh/id_ed25519` stays on your machine. Never commit it. Never paste it into a ticket.

On the forge: account settings → SSH keys → paste the `.pub` line. GitHub, GitLab, and Forgejo all have this screen. The label can be `laptop`.

Test (GitHub as an example host; use your forge's hostname):

```
ssh -T git@github.com
```

A greeting that includes your username means the key was accepted. A password prompt means the server did not see the key (`ssh -v` shows which files it tried).

`~/.ssh/config` for several hosts:

```
Host github.com
  HostName github.com
  User git
  IdentityFile ~/.ssh/id_ed25519

Host forgejo.home
  HostName 192.168.68.200
  User git
  Port 2222
  IdentityFile ~/.ssh/id_ed25519
```

A later Forgejo lab uses a port like 2222. The `Host` alias lets you write `git@forgejo.home:ada/desk-api.git`.

HTTPS and tokens

```
git clone https://github.com/example/desk-api.git
```

Pushing over HTTPS asks for a username and a **personal access token** (not your account password on most forges). Store it in a credential helper:

```
git config --global credential.helper libsecret   # GNOME Keyring / libsecret, if installed
# memory-only fallback (forgets at reboot or after a timeout):
git config --global credential.helper cache
```

CI systems often use HTTPS with a job-scoped token. Humans on a laptop usually have an easier time with SSH.

Pick one default

For this book's desk repos on a machine you own: **SSH**. Clone and remote URLs will look like:

```
git@host:owner/desk-api.git
```

You can change a remote later (chapter 11):

```
git remote -v
git remote set-url origin git@host:owner/desk-api.git
```

Chapter 3 stays local: `git init` does not need SSH or HTTPS.

Editor, Pager, and Difftool

Editor, Pager, and Difftool

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- `core.editor`, `core.pager`, `less`, `nvimdiff` / `vimdiff` on Linux.

> **Note**
>
> Core Git depth (Linux host). See `content/_planning/SYLLABUS.md`.

Aliases

Aliases

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- `git config alias.*`, shell aliases vs Git aliases, useful starter set.

> **Note**
>
> Core Git depth (Linux host). See `content/_planning/SYLLABUS.md`.

Updating Git on Linux

Updating Git on Linux

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Distro packages vs building from source; checking `git --version`.

> **Note**
>
> Core Git depth (Linux host). See `content/_planning/SYLLABUS.md`.

First Repository

Theory

Theory: The Add/Commit Cycle is the Product

A repository is useful the moment you can take bytes on disk, choose a subset, and name a snapshot. Everything else (branch, rebase, push) is names and copies of that operation.

Invariants

1. `git init` creates `.git/` and a symbolic `HEAD`. The first **commit** creates `refs/heads/main`.
2. `git add` copies working-tree bytes into the **index** (and writes blobs). It does not create a commit.
3. `git commit` writes a tree from the index, then a commit, then slides the current branch name.
4. `git status` is three comparisons: working vs index, index vs `HEAD`, untracked vs index.
5. Undo of **uncommitted** work is `restore`. Undo of the **last unpublished** commit is `--amend`. Undo of **published** history is `revert` (later).

```
edit  →  add  →  commit
 WD       index    HEAD / main

status reports which of the three disagree
```

Why the index exists

Without an index, every commit would be "whatever is on disk." The index is how you commit two of five edited files, or one hunk of a file (`add -p`). That is also why `.gitignore` exists: untracked paths never enter the index unless you force them.

What can break

- `git add .` of `.env` and secrets (ignore first).
- `git commit -a` on a **new** file (it only auto-stages tracked paths).
- `git init` in `$HOME` and then every subdirectory looking like a repo.
- Amending after someone else fetched that hash.

How to read this chapter

Init, add/commit, status/log/diff, and undo are the loop. The hands-on page is that loop on `desk-api`. Ignore, interactive add, `rm`/`mv`, and "what not to commit" are the same loop with sharper edges.

Initializing a Repository

Initializing a Repository

After this page you have an empty Git repository on `main` and you know what `git init` created.

A directory becomes a repository

```
mkdir -p ~/desk/desk-api
cd ~/desk/desk-api
git init
```

Output looks like:

```
Initialized empty Git repository in /home/ada/desk/desk-api/.git/
```

If you set `init.defaultBranch main` (chapter 2), the empty branch is named `main`. Confirm:

```
git status
```

```
On branch main

No commits yet

nothing to commit (create/copy files and use "git add" to track)
```

`~/desk/desk-api` is the **working tree**. `~/desk/desk-api/.git/` is the **repository** (objects, refs, config). You edit files next to `.git`, not inside it.

```
~/desk/desk-api/                 working tree (you edit these)
   README.md
   src/
   .git/                        repository
       HEAD                     ref: refs/heads/main
       config
       objects/                 empty until first commit
       refs/heads/              main appears after first commit
```

This book's recurring desk is three repos: `desk-web`, `desk-api`, `desk-infra`. Part I uses `desk-api` as the tiny project. Create the other two the same way when a later chapter asks.

What `git init` wrote

```
ls -a .git
```

You will see at least `config`, `HEAD`, `objects/`, `refs/`. `HEAD` currently says `ref: refs/heads/main` even though `refs/heads/main` does not exist yet. The first commit creates that file.

```
cat .git/HEAD
git rev-parse --git-dir
# .git
```

`--git-dir` is useful when you are in a subdirectory later.

`init` vs `clone`

Command	When
`git init`	You are starting a project that has no Git history yet
`git clone <url>`	You are copying an existing repository, including its history

`clone` is chapter 11. For a brand-new desk project, `init` is the right verb.

Already have files?

`git init` in a directory that already has source files is fine. Git does not add them until you `git add`. The next page is that loop.

`init` in the wrong place

If you run `git init` in your home directory, every `git status` from a subdirectory without its own `.git` will try to use `~/.git`. Remove a mistaken repo with:

```
# only if you meant to throw it away and there are no commits you need
rm -rf .git
```

That deletes Git's data for that directory, not your files. The first-commit hands-on page builds `desk-api` from scratch so you can see a clean `init`.

The Add/Commit Cycle

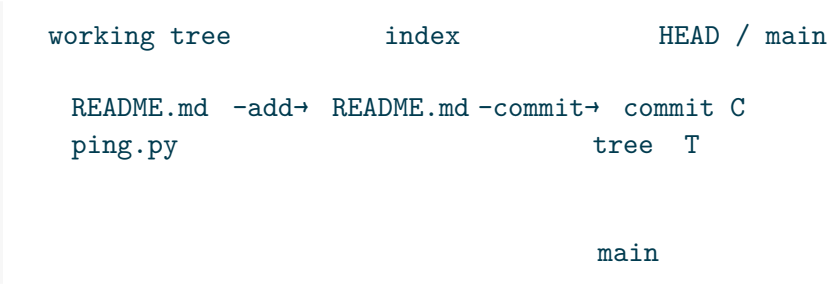
The Add/Commit Cycle

After this page you can take a file from the working tree to a commit on `main`.

The loop

```
edit files  →  git add  →  git commit  →  git status (clean)
```

That is the whole product: copy chosen files into the index, snapshot the index, name the snapshot with a message, slide `main` forward.



One file, one commit

From an initialized `desk-api` (previous page):

```
cd ~/desk/desk-api

cat > README.md <<'EOF'
# desk-api

HTTP API for the desk.
EOF

git add README.md
git status
git commit -m "Add README"
git log --oneline
```

`git add` stages `README.md`. `git commit -m` writes the commit without opening an editor. `git log --oneline` prints one line: a short hash and the message.

`status` is now clean. Working tree, index, and HEAD match.

A second commit

```
mkdir -p src
cat > src/ping.py <<'EOF'
def ping():
    return "ok"
EOF

git add src/ping.py
git commit -m "Add ping"
git log --oneline
```

Two commits. `src/` exists in the tree object because Git stored a tree named `src` pointing at the `ping.py` blob. You did not `git add` the directory separately; adding the file is enough.

What `-m` is doing

Every commit needs a message. `-m "Add README"` is the short form. Without `-m`, Git opens `core.editor`. Multi-paragraph messages:

```
git commit -m "Add ping" -m "Returns a fixed ok string so the health check has a function to
```

The first `-m` is the subject. The second is the body. Chapter 6 is Conventional Commits (`feat:`, `fix:`). For Part I, a short imperative sentence is enough: **Add ping**, **Fix empty name**, **Document env vars**.

`git add` shapes

```
git add README.md              # one file
git add src/ping.py src/app.py # several
git add src                    # a directory, recursively
git add .                      # everything from here down, still respects .gitignore
git add -p README.md           # hunk by hunk (interactive)
```

`.gitignore` is a file of patterns Git will leave untracked. Create it when you have build output or secrets to skip:

```
cat > .gitignore <<'EOF'
__pycache__/
.env
EOF
```

```
git add .gitignore
git commit -m "Ignore bytecode and env files"
```

Patterns are matched from the file's directory downward. `/build` at the repo root is only `build/` at the root; `build/` is every directory named `build`.

`commit -a`

```
git commit -a -m "Touch tracked files"
```

Stages **already tracked** files and commits. New files still need `git add`. Prefer explicit `git add` until that is automatic in your head.

After the cycle

```
git status
git log --oneline
```

Clean status, two or three commits. Next page: how to *read* what you just did (`status`, `log`, `diff`).

status, log, and diff

git status, git log, git diff

After this page you can read the three-tree report, walk history, and see a patch before you commit it.

`git status` — where you are

```
cd ~/desk/desk-api
git status
git status -sb          # short: branch + one letter per file
```

`-sb` prints `## main` and lines like `M README.md` (modified) or `?? notes.txt` (untracked). Use it when the long form is noise.

`git status` is the three-tree report from chapter 1: working tree vs index vs HEAD.

```
 git diff              git diff --staged         git log
 working vs index      index vs HEAD             HEAD chain

 working tree          index                     HEAD
                                        A ← B ← C  main
   extra               staged
```

`git log` — the DAG as a list

```
git log
git log --oneline
git log --oneline --graph --decorate --all
git log -3                          # last three
git log --oneline -- README.md      # commits that touched this path
git log -p -1                       # last commit plus its patch
```

`--oneline` is the default view in this book. `--graph --decorate --all` is the view that matches the mental-model diagrams.

Useful format:

```
git log --format='%h %an %s'
```

`%h` short hash, `%an` author name, `%s` subject. `git help log` lists more placeholders. You will use `--format` in hooks and CI later; for now, `--oneline` is enough.

`git diff` — the patch

```
git diff                # working tree vs index (unstaged)
git diff --staged       # index vs HEAD (what the next commit will contain)
git diff HEAD           # working tree vs HEAD (staged and unstaged together)
git diff HEAD~1 HEAD    # last commit vs its parent
git diff main feature   # two branches (once you have them, chapter 8)
```

Empty `git diff` and non-empty `git diff --staged` means everything is already in the index. Empty both, plus an untracked file in `status`, means you still need `git add`.

DiffTool

For a side-by-side view in the terminal:

```
git config --global diff.tool nvimdiff    # or vimdiff, meld
git difftool README.md
git difftool --staged
```

`difftool` opens each changed file in the tool you named. `mergetool` is the sibling for conflicts (chapter 8). Stay on `git diff` until you want the split view. The patch is the same data.

A one-minute drill

```
echo "note" >> README.md
git status
git diff
git add README.md
git diff                # empty
git diff --staged       # the note
git restore --staged README.md
git restore README.md   # discard the note
git status              # clean
```

`restore` is the next page, on purpose. Here you only need to *see* the three diffs change as the file moves.

Undoing Mistakes

Undoing Mistakes

After this page you can discard an unstaged edit, unstage a file, and rewrite the last commit **before anyone else has it**.

Three common "undo"s

You did	You want	Command
Edited a tracked file, not staged	Throw the edit away	`git restore <file>`
`git add` too soon	Keep the edit, leave it out of the next commit	`git restore --staged <file>`
Last commit message (or last commit contents) is wrong, not pushed	Replace that commit	`git commit --amend`

These are safe on a laptop branch nobody has fetched. After `git push`, amending rewrites a hash other people may have. Chapter 9 and chapter 11 cover that case. Chapter 17 (reflog) is the safety net when you went too far.

```
restore <file>          restore --staged        commit --amend
working ← index         index ← HEAD            A ← B ← C
                                                A ← B ← C'   (C gone from main)
```

Discard unstaged work

```
cd ~/desk/desk-api
echo "oops" >> README.md
git restore README.md
```

The working-tree file matches the index again. The "oops" line is gone. Git cannot get it back from Git; your editor's undo might.

Restore one file from HEAD even if it was staged:

```
git restore --source=HEAD --worktree --staged README.md
```

That is "make this file look like the last commit, in both index and working tree."

Unstage, keep the edit

```
echo "keep me" >> README.md
git add README.md
git restore --staged README.md
git status
```

The line is still in the file. It is not in the index. The next `git commit` will not include it until you `git add` again.

Older docs say `git reset HEAD README.md` for the same unstage. This book uses `git restore --staged`.

Amend the last commit

You just committed and noticed a typo in the message:

```
git commit --amend -m "Add ping helper"
```

You forgot a file:

```
git add src/ping.py
git commit --amend --no-edit
```

`--no-edit` keeps the old message. `--amend` replaces the last commit with a new one (new hash) whose parent is the same as before. `git log` no longer shows the old hash as `main`. The old commit is still in the reflog for about 30 days (chapter 17).

Amend only when:

- it is the latest commit on this branch, and
- you have not pushed it, or you are sure no one else built on it.

Revert is a new commit

`git revert <hash>` adds a commit that undoes a previous one. Use it on history other people already have. Chapter 10 is revert and cherry-pick. For a local last-commit mistake, amend. For an old published mistake, revert.

`rm` and `mv`

```
git rm src/old.py              # stop tracking and delete the file
git rm --cached secrets.env    # stop tracking, keep the file on disk
git mv src/ping.py src/health.py
git status
git commit -m "Rename ping to health"
```

`git mv` is `mv` plus `git add` of the rename. Git detects renames in diffs even if you `mv` and `add` by hand; `git mv` is the tidy form.

If you are lost

```
git status
git log --oneline -5
```

Status tells you which of the three trees disagree. Log tells you where `main` points. The hands-on page next builds a tiny `desk-api` from scratch so you can practice the whole loop, including one amend.

Hands-On: Build a Tiny Project

Hands-On: Build a Tiny Project

Build `desk-api` from nothing: init, two commits, a bad edit, a restore, an amend, a clean log.

```
init → add/commit README → add/commit ping
     → restore an "oops" → amend the last message
     → leave .env untracked

A ← B ← C     main     status clean
```

Setup

Use a throwaway directory. If `~/desk/desk-api` already exists from earlier pages, either work there or pick `~/desk/desk-api-lab`.

```
mkdir -p ~/desk/desk-api
cd ~/desk/desk-api
git init
git status
```

Expect `On branch main` and `No commits yet`.

Commit 1 — README

```
cat > README.md <<'EOF'
# desk-api

HTTP API for the desk.
EOF

git add README.md
git commit -m "Add README"
git log --oneline
```

One line of history.

Commit 2 — a tiny module

```
mkdir -p src
cat > src/ping.py <<'EOF'
def ping():
    return "ok"
EOF

cat > .gitignore <<'EOF'
__pycache__/
.env
EOF

git add src/ping.py .gitignore
git status
git commit -m "Add ping and gitignore"
```

Read it

```
git status
git log --oneline --graph --decorate
git show --stat HEAD
```

`git show` prints the last commit's metadata and patch. `--stat` is the short file list.

Break it on purpose

```
echo "oops" >> README.md
git status
git diff
git restore README.md
git status
```

Clean again. The "oops" line is gone.

Stage, unstage, restage

```
echo "Health: GET /ping" >> README.md
git add README.md
git diff --staged
git restore --staged README.md
```

```
git diff
git add README.md
git commit -m "Document /ping"
```

Amend the message

That last message is vague. Replace it (still local, unpushed):

```
git commit --amend -m "Document the ping endpoint in README"
git log --oneline
```

Three commits. The amended hash is new. `git status` is clean.

Ignore a secret-shaped file

```
echo 'TOKEN=dev' > .env
git status
```

`.env` should be untracked (your `.gitignore` lists it). If it shows as untracked, the ignore pattern is working. If Git wants to add it, the `.gitignore` commit is missing — add the file and commit.

Leave `.env` untracked. Do not `git add .env`.

Done when

```
git status
git log --oneline
```

- Status: clean, on `main`
- Log: three commits (README, ping, document endpoint)
- `.env` exists on disk and is untracked

You now have the loop this book's later Git chapters assume: init, add, commit, status, log, diff, restore, amend.

Part II starts with commit anatomy and Conventional Commits. Remotes and forges wait until you can do this drill without looking at the page.

.gitignore in Depth

.gitignore in Depth

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Patterns, negation, `/` vs glob, `.git/info/exclude`, global excludes.

> **Note**
>
> Core Git depth (Linux host). See `content/_planning/SYLLABUS.md`.

Interactive Add

Interactive Add

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- `git add -p`, hunks, splitting hunks, what staging a hunk means.

> **Note**
>
> Core Git depth (Linux host). See `content/_planning/SYLLABUS.md`.

git rm and git mv

git rm and git mv

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Stop tracking, keep on disk, rename detection.

> **Note**
>
> Core Git depth (Linux host). See `content/_planning/SYLLABUS.md`.

What Not to Commit

What Not to Commit

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Secrets, build artifacts, `.env`, binaries; first look at LFS.

> **Note**
>
> Core Git depth (Linux host). See `content/_planning/SYLLABUS.md`.

Inspecting History

Theory

Theory: History is a Query over the DAG

`git log` is not a file. It is a **walk** of commit objects from one or more tips, filtered by paths, dates, pickaxe, and ranges. Blame is a walk of *lines* through that DAG.

Invariants

1. Reachability: `git log B` shows commits reachable from `B` by following parents.
2. **Two-dot** `A..B` = reachable from `B`, not from `A` (what a feature branch adds).
3. **Three-dot** `A...B` = reachable from `A` or `B`, not from the merge base (symmetric difference). A pull-request diff is often `git diff A...B` (merge-base to `B`).
4. `git blame` attributes each line of a file to the commit that last touched it; `-C` also hunts copies.
5. Pickaxe `-S`/`-G` filters commits by whether a string's count or a regex's hunk changed.

```
A ← B ← C ← G     main
     ↑
     D ← E        feat

main..feat     =  D, E
main...feat    =  G, D, E     (both sides since split)
```

Why this matters on a forge

A PR "files changed" list is this query, computed on the server. Reviewers who can run `git log main..HEAD` and `git diff main...HEAD` on Linux are not trapped in the UI.

What can break

- Reading `A..B` as "between A and B inclusive on a line" (it is set difference of reachability).
- `git blame` on a file that was reformatted in one giant commit (whitespace; `-w`).
- `--follow` only works when you name **one** path.

How to read this chapter

Log flags, ranges, show, blame, pickaxe, shortlog, then a lab that finds a line's origin in `desk-api`.

Advanced git log

Advanced git log

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- `--oneline --graph --decorate --all`, `--format`, `--since`, path limiting.

> **Note**
>
> Core Git depth (Linux host). See `content/_planning/SYLLABUS.md`.

Two-Dot and Three-Dot Ranges

Two-Dot and Three-Dot Ranges

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- `A..B` vs `A...B`; what a pull request actually diffs.

> **Note**
>
> Core Git depth (Linux host). See `content/_planning/SYLLABUS.md`.

git show and –stat

git show and –stat

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- `git show`, `--stat`, `--name-only`, `--name-status`.

> **Note**
>
> Core Git depth (Linux host). See `content/_planning/SYLLABUS.md`.

git blame

git blame

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- `git blame`, `-L`, `-C` copy detection, ignoring whitespace.

> **Note**
>
> Core Git depth (Linux host). See `content/_planning/SYLLABUS.md`.

Pickaxe and –follow

Pickaxe and –follow

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- `-S`, `-G`, `--follow` across renames.

> **Note**
>
> Core Git depth (Linux host). See `content/_planning/SYLLABUS.md`.

git shortlog

git shortlog

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- `git shortlog`, contributor summaries.

> **Note**
>
> Core Git depth (Linux host). See `content/_planning/SYLLABUS.md`.

Hands-On: History Archaeology

Hands-On: History Archaeology

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Find who changed a line and when a string entered `desk-api`.

> **Note**
>
> Core Git depth (Linux host). See `content/_planning/SYLLABUS.md`.

Ignore, Attributes, and LFS

Theory

Theory: The Index is a Filter; LFS is a Pointer

Git will store any blob you add. Ignore rules and attributes are **policy on what enters the index and how it is treated**. LFS replaces a large blob with a tiny pointer blob and puts the real bytes elsewhere.

Invariants

1. `.gitignore` applies to **untracked** files. A tracked file stays tracked until `git rm --cached`.
2. Precedence: command-line exclude → `.git/info/exclude` → `.gitignore` in-tree → global excludes file.
3. `gitattributes` attach behavior to paths: text vs binary, `eol`, `diff`, `merge`, `filter` (clean/smudge).
4. Git LFS stores a **pointer file** (text, few hundred bytes) in Git, and the payload in an LFS object store the forge must provide.

```
git add huge.bin     without LFS     blob of huge.bin in .git/objects
git add huge.bin     with LFS        blob of pointer  version https://git-lfs...
                                     payload    LFS store (not in packfiles)
```

Why Git is a bad backup of videos

Every clone downloads every version of every blob (unless partial clone / LFS / sparse). A 200 MB binary edited ten times is ~2 GB of packfile. LFS and "do not commit build output" are the same theory: Git's object store is for source, not for arbitrary bytes.

What can break

- Ignoring `.env` after it was committed (must `rm --cached`).
- CRLF filters on a Linux-only repo.
- Pushing LFS pointers to a forge with no LFS storage; clones get pointers, not files.

How to read this chapter

Patterns, attributes, archive, LFS, then a lab. Platform chapters later must store LFS if you use it.

gitignore Patterns

gitignore Patterns

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Per-repo, global, and `.git/info/exclude`; precedence.

> **Note**
>
> Core Git depth (Linux host). See `content/_planning/SYLLABUS.md`.

gitattributes

gitattributes

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Text/binary, `eol`, `ident`, `filter`, `diff`, `merge` drivers.

> **Note**
>
> Core Git depth (Linux host). See `content/_planning/SYLLABUS.md`.

export-ignore and git archive

export-ignore and git archive

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- `export-ignore`, `git archive` for tarballs.

> **Note**
>
> Core Git depth (Linux host). See `content/_planning/SYLLABUS.md`.

Git LFS

Git LFS

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- When blobs are too big; pointers; `git lfs install` on Linux; what a forge must store.

> **Note**
>
> Core Git depth (Linux host). See `content/_planning/SYLLABUS.md`.

Hands-On: Ignore and LFS

Hands-On: Ignore and LFS

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Ignore `.env` and `__pycache__`; optional LFS track of a sample binary.

> **Note**
>
> Core Git depth (Linux host). See `content/_planning/SYLLABUS.md`.

Everyday Workflows

Commits and History

Theory

Theory: A Commit is Immutable; Messages are for Humans and Machines

Once written, a commit object is byte-for-byte fixed. `--amend` and rebase do not edit it; they write a **new** object and slide a name. That is why hashes change and why shared history is a social problem.

Invariants

1. Commit hash = hash of (tree, parents, author, committer, message). Change any field → new hash.
2. Author and committer can differ (rebase, cherry-pick).
3. Conventional Commits are a **convention**, not a Git feature. Hooks and forges can enforce them.
4. Unstaging moves a path from index toward `HEAD`; it does not delete working-tree bytes.

```
amend

A ← B ← C     main     C has message "stuff"
A ← B ← C'    main     C' new object; C remains in reflog
```

Why messages matter

`git log --grep '^fix'` only works if subjects are structured. Changelog generators and `feat!:` breaking-change scanners parse the same text. The object store does not care; your future self and CI do.

What can break

- Amending a commit already on `origin/main`.
- Empty subject or `WIP` on a branch you intended to merge.
- Trailers with the wrong spelling (`Co-authored-by` is magic for some forges).

How to read this chapter

Anatomy, Conventional Commits, amend, unstage, lab; then templates, split/empty, trailers/mailmap, `git show`.

Commit Anatomy

Commit Anatomy

After this page you can read a commit object and say what each field is for.

A commit is a snapshot plus metadata

```
cd ~/desk/desk-api
git cat-file -p HEAD
```

You will see something like:

```
tree 4b825dc642cb6eb9a060e54bf8d69288fbee4904
parent 9fceb02d0ae598e95dc970b74767f19372d61af8
author Ada Desk <ada@example.com> 1774000000 +0530
committer Ada Desk <ada@example.com> 1774000000 +0530

Document the ping endpoint in README
```

Field	Meaning
`tree`	Hash of the directory snapshot this commit names
`parent`	Previous commit (omitted on the first commit; two lines on a merge)
`author`	Who wrote the patch, plus timestamp
`committer`	Who created this commit object, plus timestamp
blank line, then body	The message

The **name** of the commit is the hash of this object. Change the message, the tree, or a parent → new hash. That is why amend and rebase rewrite history: they write new objects.

Author and committer

They match until you rebase, cherry-pick, or apply someone else's patch. `git log` shows the **author** by default. `git log --format=fuller` shows both.

```
git log -1 --format=fuller
```

Short names

```
git rev-parse HEAD
git rev-parse HEAD~1          # first parent, one step back
git rev-parse HEAD^           # same as HEAD~1 on a non-merge
git rev-parse --short HEAD    # abbreviated, unique in this repo
```

On a merge commit, `HEAD^1` is the first parent (the branch you were on) and `HEAD^2` is the branch you merged in. `HEAD~n` walks first-parent only. Chapter 10 uses that when reverting a merge.

The tree under it

```
git ls-tree -r HEAD
```

Each line is a mode, type, hash, and path. Blobs have no filename inside them; the tree supplies the name. Two commits that share an unchanged `src/ping.py` share the same blob hash.

What `git show` adds

```
git show HEAD
git show --stat HEAD
git show --name-only HEAD
```

`git show` pretty-prints the commit and, by default, the diff against its first parent. That diff is computed; Git stored the tree, not the patch.

Next page: how to write the message so later `git log` and automation can parse it.

Conventional Commits

Commit Messages That Matter: Conventional Commits

After this page you can write a subject line other people and tools can parse.

Shape of a message

```
type(scope): short summary

Optional body: why, not what. Wrap at 72 characters.

Optional footer:
Fixes: #12
BREAKING CHANGE: /ping now returns JSON
```

The **subject** is one line, imperative, about 50 characters, no trailing period. Git itself does not enforce this. Forges, changelog generators, and `git log --grep` benefit when you do.

Types this book uses

Type	When
`feat`	A user-visible capability
`fix`	A bug
`docs`	README, comments, this book's desk docs
`chore`	Tooling, ignore files, no user-facing change
`refactor`	Same behavior, different code
`test`	Tests only
`ci`	Pipeline files (later parts)

`scope` is optional: `feat(api):`, `fix(web):`. For `desk-api` a scope of `ping` or `auth` is enough.

Examples that match the desk:

```
feat(ping): return JSON from /ping
fix(ping): treat empty name as 400
docs: document GET /ping in README
```

```
chore: ignore .env and __pycache__
```

Why bother

```
git log --oneline --grep '^fix'
git log --oneline --grep '^feat(ping)'
```

Later, platform governance (chapter 31) can require this shape in a server-side hook. A client-side `commit-msg` hook (chapter 13) can reject a bad subject before the object exists.

What to put in the body

The subject is *what*. The body is *why* and any context `git log` should keep:

```
git commit -m "fix(ping): treat empty name as 400" -m "The handler returned 200 and an empty
```

Skip the body when the subject is the whole story.

Trailers

```
Signed-off-by: Ada Desk <ada@example.com>
Co-authored-by: Sam Desk <sam@example.com>
Fixes: #18
```

`git interpret-trailers` reads these. Signed-off-by is a project convention (Linux kernel, some forges). Use it when the project asks. Co-authored-by is how forges attribute a second author on a squash.

A local alias

```
git config --global alias.cm '!f() { git commit -m "$1"; }; f'
```

Aliases are optional. The skill is the subject line, not the shortcut.

Amending a Commit Message

Amending and Fixing an Incorrect Commit Message

After this page you can replace the last commit's message, and you know when that is safe.

The last commit only

```
cd ~/desk/desk-api
git log -1 --format='%s'
git commit --amend -m "docs: document GET /ping in README"
git log -1 --format='%h %s'
```

`--amend` writes a **new** commit object with the same tree and parent (unless you also staged files) and slides the branch name to it. The old hash disappears from `git log`. It is still in `git reflog` for about 30 days (chapter 17).

```
before                          after
A ← B ← C   main                A ← B ← C'  main

                message "stuff"       C still in reflog
```

Keep the old message and only restage files:

```
git add src/ping.py
git commit --amend --no-edit
```

Open the editor with the old message loaded:

```
git commit --amend
```

`core.editor` (chapter 2) is what opens. On Linux with `nvim`, you edit, save, quit, and Git takes the buffer.

When this is safe

Safe when **all** of these hold:

- it is `HEAD` (the latest commit on this branch)

- you have not pushed, or you are the only clone
- nobody else based work on the old hash

After a push, amending means a force-push (chapter 11). On `main` shared with a team, prefer a new commit: `git commit -m "docs: fix typo in README"`.

Older than HEAD

`--amend` cannot retitle commit `HEAD~3` by itself. Interactive rebase can (chapter 9, `reword`). For now, live with old subjects or wait for rebase.

Recover if you amended by mistake

```
git reflog
git reset --soft HEAD@{1}
```

`--soft` moves `HEAD` back and leaves the index and working tree as they are. Chapter 7 is reset in full. The hash you just left is in the reflog (chapter 17). This one-liner is enough if you just amended and panicked.

Identity in an amended commit

Amend rewrites the **committer** date by default. The author stays. To keep both timestamps:

```
git commit --amend --no-edit --date="$(git log -1 --format=%aD)"
```

Rarely needed. Know that `git log` after an amend shows "now" as the committer time.

Unstaging a File

Unstaging an Added File

After this page you can take a path out of the index and leave the working-tree edit in place.

The index is the next commit

You `git add` too much. The file should stay on disk. It should not be in the next snapshot.

```
cd ~/desk/desk-api
echo "scratch" >> README.md
echo "debug" >> src/ping.py
git add README.md src/ping.py
git status
git restore --staged src/ping.py
git status
```

`README.md` is still staged. `src/ping.py` is modified, not staged. `git commit` will only record the README line.

```
after git add -A                after restore --staged ping.py

working   index   HEAD          working   index   HEAD
ping.py   ping.py  (old)        ping.py   (old)   (old)
README    README   (old)        README    README  (old)
                                  still on disk, not in next commit
```

Restore vs reset

Command	Index	Working tree
`git restore --staged <file>`	Matches HEAD	Unchanged (your edit stays)
`git restore <file>`	Unchanged	Matches the index (edit discarded)
`git restore --source=HEAD --staged --worktree <file>`	Matches HEAD	Matches HEAD

This book uses `git restore`. Older guides use `git reset HEAD <file>` for unstage; same index effect.

Unstage everything

```
git restore --staged .
git status
```

All paths return to "modified" or "untracked." Nothing is deleted.

Untracked files

`git add` of a new file copies it into the index. `git restore --staged` on that path leaves it **untracked** (the index no longer knows it). The file remains on disk. `git clean` would delete untracked files; that is chapter 12. Leave `git clean` alone until you can name what it removes.

Partial unstage

```
git restore --staged -p src/ping.py
```

Git offers hunks. `n` keeps the hunk staged, `y` unstages it. Same interactive idea as `git add -p`.

Check before you commit

```
git diff --staged
git diff
```

`--staged` is the next commit. The other is leftover work. Both should match what you intend. The hands-on page next amends a real `desk-api` commit so this is muscle memory.

Hands-On: Amend and Rewrite

Hands-On: Amend and Rewrite a Commit

Practice on `~/desk/desk-api`. If that tree is missing, run chapter 3's tiny-project page first.

```
commit "stuff"     A ← B ← C
amend subject      A ← B ← C'     C in reflog only
amend add file     A ← B ← C''    tree now includes version.py
```

1. A sloppy commit

```
cd ~/desk/desk-api

echo "Returns a fixed string." >> README.md
git add README.md
git commit -m "stuff"
git log -1 --oneline
```

2. Fix the subject

```
git commit --amend -m "docs: note that ping returns a fixed string"
git log -1 --format='%h %s'
```

The hash changed. The tree is the same if you staged nothing else.

3. Forgot a file

```
cat > src/version.py <<'EOF'
VERSION = "0.1.0"
EOF
git add src/version.py
git commit --amend --no-edit
git show --name-only --oneline HEAD
```

`version.py` is now in the same commit as the docs line.

4. Staged too much, then unstaged

```
echo "WIP" >> src/ping.py
echo "also" >> README.md
git add -A
git restore --staged src/ping.py
git diff --staged
git commit --amend --no-edit
git restore src/ping.py
git status
```

The WIP line is gone. README's "also" is in HEAD if it was staged. Status is clean.

5. Confirm the graph

```
git log --oneline -5
git status
```

You should be on `main`, clean, with a conventional subject on `HEAD`. Next chapter: branches, so this `main` history can grow a second line of work.

Commit Templates

Commit Templates

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- `commit.template`, `prepare-commit-msg` preview.

> **Note**
>
> Core Git depth (Linux host). See `content/_planning/SYLLABUS.md`.

Empty and Split Commits

Empty and Split Commits

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- `--allow-empty`; splitting a commit with reset + add -p.

> **Note**
>
> Core Git depth (Linux host). See `content/_planning/SYLLABUS.md`.

Trailers and mailmap

Trailers and mailmap

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- `Signed-off-by`, `Co-authored-by`, `git interpret-trailers`, `.mailmap`.

> **Note**
>
> Core Git depth (Linux host). See `content/_planning/SYLLABUS.md`.

Reading a Commit with git show

Reading a Commit with git show

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- `git show`, `--pretty=fuller`, `--format=raw`.

> **Note**
>
> Core Git depth (Linux host). See `content/_planning/SYLLABUS.md`.

Reset, Restore, and Checkout

Theory

Theory: Reset Moves Names; Restore Copies Bytes; Revert Adds a Commit

Three different operations get called "undo." They act on different trees and different audiences.

Invariants

Tool	What it moves	History
`git restore`	Working tree and/or index, from a source	Names unchanged
`git reset` (no path)	`HEAD` (and maybe index, working tree)	Branch name slides; old commit may become unreachable
`git reset` (with path)	Index (mixed) for those paths	`HEAD` stays
`git revert`	Nothing; writes a new commit	History goes forward

Soft / mixed / hard is **which of the three trees follow `HEAD`** when you reset a commit:

```
             HEAD     index          worktree
  --soft     moves    stays          stays
  --mixed    moves    matches HEAD   stays          (default)
  --hard     moves    matches HEAD   matches HEAD
```

Why `checkout` split

Old `git checkout` meant both "switch branch" and "restore file." Git 2.23+ splits them: `switch` for branches, `restore` for files. This book uses the split so a command's name is its theory.

What can break

- `reset --hard` on a shared `main`.
- Confusing mixed reset (unstage all) with hard (throw away tracked work).
- Expecting `--hard` or `restore` to delete untracked files. They stay. Removing them is `git clean`.

How to read this chapter

Three modes, the comparison table as practice, path reset, switch/restore/checkout, then a lab that walks one commit through soft, mixed, and hard on Linux.

Soft, Mixed, and Hard Reset

Soft, Mixed, and Hard Reset

After this page you can predict `git status` after `--soft`, `--mixed`, and `--hard`, and you can put the branch name back with `ORIG_HEAD`.

One commit, three trees

Start from a clean `main` on `~/desk/desk-api` (chapter 3). This page adds one scratch commit, tries each mode, and removes that commit at the end.

```
cd ~/desk/desk-api
git switch main
git status
```

Ignored files such as `.env` are fine. Commit or restore any other uncommitted work before you continue.

```
echo "scratch" >> README.md
git add README.md
git commit -m "lab: reset modes"
```

```
 A ← B ← S     main
 HEAD, index, and worktree all name S
```

`S` is the scratch commit. Each mode below starts from `S`. Git records the old tip in `ORIG_HEAD` whenever a reset moves the branch, so `git reset --hard ORIG_HEAD` returns to `S`.

`--soft` moves the name

```
git reset --soft HEAD~1
git status
```

```
 HEAD → B
 index still S       "Changes to be committed"
 worktree still S
```

`main` now names `B`. The scratch line is staged. `git log` no longer shows `lab: reset modes`. The commit object stays in the reflog (chapter 17).

```
git reset --hard ORIG_HEAD
git log --oneline -1
```

You are back on the scratch commit. The line is in `README.md`, and status is clean.

`--mixed` moves the name and the index

`--mixed` is the default. `git reset HEAD~1` and `git reset --mixed HEAD~1` are the same command.

```
git reset HEAD~1
git status
git diff
```

Git prints `Unstaged changes after reset:`.

```
HEAD → B
index matches B      the scratch line is not staged
worktree still S     "Changes not staged for commit"
```

The patch is in the working tree. The next commit includes it only after `git add`.

```
git reset --hard ORIG_HEAD
```

`--hard` moves all three

```
git reset --hard HEAD~1
git status
grep scratch README.md || echo "scratch line is gone"
```

```
HEAD, index, and worktree all match B
tracked files match B
```

The line is gone from disk. Untracked files are still on disk. `--hard` rewrites tracked files to the commit you named and leaves untracked and ignored files alone.

```
git reset --hard ORIG_HEAD
```

The scratch commit is the branch tip again, and the line is back. `ORIG_HEAD` is the immediate undo. After other commands have moved `HEAD`, the reflog is the record (chapter 17). The lab at the end of this chapter recovers with `HEAD@{1}` on purpose.

Remove the scratch commit

```
git reset --hard HEAD~1
git status
git log --oneline -5
```

`lab: reset modes` is off the log. `README.md` has no scratch line. The object remains in the reflog until it expires.

Where this is safe

Use these modes on a local branch nobody else has fetched. On a `main` other people already have, moving the name and pushing is a force-push (chapter 11). A published undo is `git revert` (chapter 10). The next page is how to choose restore, reset, or revert.

restore vs reset vs revert

Restore, Reset, and Revert

After this page you can pick the undo that matches what is wrong and who else has the commit.

Three audiences

```
uncommitted                 unpublished                  published
working tree or index       only this clone has the hash someone else may have it
restore                     reset or amend               revert
```

Situation	Command	Branch tip
Edited a tracked file, not staged	`git restore <file>`	stays
Staged too soon	`git restore --staged <file>`	stays
Last local commit should be redone	`git reset --soft HEAD~1`, then commit	parent; changes stay staged
Last local commit should be an unstaged edit	`git reset HEAD~1`	parent; changes stay in the worktree
Last local commit and its files should disappear	`git reset --hard HEAD~1`	parent; tracked files match the parent
Last local commit needs a new message or one more file	`git commit --amend`	new hash, same parent (chapter 6)
A shared commit should be undone	`git revert <hash>`	moves forward (chapter 10)

`restore` copies bytes. The branch name stays. `reset` with no path slides the branch name. `revert` adds a commit.

Amend and soft reset

Both replace an unpublished tip.

Amend keeps one commit at the tip: same parent, new hash. You are still one commit ahead of the parent. Chapter 6 is amend.

Soft reset takes the tip off the branch and leaves that tree staged. You can drop a file, split the change, or commit it again with an ordinary message. The previous page is that mode.

Revert goes forward

`git revert` writes a new commit whose patch is the inverse of an older one. The old hash stays reachable. Use it when the commit is on a shared `main`, or when you are unsure whether anyone fetched it. Chapter 10 includes un-merging with `git revert -m 1`.

```
published:   A ← B ← C ← R      R undoes C; C is still reachable
local only:  A ← B              reset moved main off C; C is in the reflog
```

A test you can apply

Ask where the hash lives.

- In the working tree or the index only: `restore`.
- On this branch, and you have not pushed it (or there is no remote yet): `reset` or `amend`.
- On a forge, or a teammate has pulled: `revert`.

Chapter 11 is the force-push that follows a reset of something you already pushed. On shared `main`, revert.

Resetting Paths

Resetting Paths

After this page you can change one path in the index while the branch name stays put.

A path means the branch stays

With no path, `git reset` moves `HEAD`. With a path, `HEAD` stays and Git copies that path from a commit into the index only.

```
cd ~/desk/desk-api
git switch main
git status
BEFORE=$(git rev-parse --short HEAD)
echo "path lab" >> README.md
git add README.md
git commit -m "lab: path reset"
git reset HEAD~1 -- README.md
git rev-parse --short HEAD
echo "parent was $BEFORE"
git status
git diff -- README.md
git diff --staged -- README.md
```

The short hash is the commit you just made, `lab: path reset`. It is not `$BEFORE`. Status lists `README.md` in two places:

- **Changes to be committed.** The index holds the parent's blob.
- **Changes not staged for commit.** The working tree still has the `path lab` line.

That is one edit, seen from both sides. `git diff` shows the line added. `git diff --staged` shows the line removed. Git also prints `Unstaged changes after reset:`.

```
HEAD stays on "lab: path reset"
index README     ← parent blob
worktree README  ← still has "path lab"
```

Older notes unstage with `git reset HEAD -- README.md`. This book unstages with `git restore --staged` (chapters 3 and 6). The path form is how you aim the index at an older blob, and how you read a script that still uses it.

Put the file back, then drop the commit

`HEAD` is still the scratch commit. Copy that commit's `README.md` into the index and the working tree:

```
git restore --source=HEAD --staged --worktree -- README.md
git status
```

Status is clean. The log still shows `lab: path reset`.

```
git reset --hard HEAD~1
git log --oneline -3
```

The scratch commit is off `main`, and `README.md` has no `path lab` line. The path reset did not do that. The last command is a branch reset from the first page of this chapter.

Whole tree versus one path

```
git reset HEAD~1
    main slides to the parent

git reset HEAD~1 -- README.md
    main stays; the index path changes

git restore --source=HEAD~1 --staged --worktree -- README.md
    main stays; index and worktree take the old blob
```

The third command is how you make a file look like an older commit and then record that with a new commit. The older commit stays in history. `switch` and `restore` on the next page are the same split: names versus bytes.

switch, restore, and checkout

switch, restore, and checkout

After this page you can change branches with `git switch` and files with `git restore`, and you can translate an old `git checkout` line.

Two commands, two jobs

Until Git 2.23, `git checkout` did both jobs. It still does, including in Git 2.55. Checking out a branch moves `HEAD`. Checking out a path copies bytes. One word for both is how a file gets overwritten when the goal was to change branches.

`git switch` moves `HEAD` and refuses a path. `git restore` copies bytes and leaves `HEAD` where it is.

Older command	Command in this book
`git checkout main`	`git switch main`
`git checkout -b feat/x`	`git switch -c feat/x`
`git checkout -- README.md`	`git restore README.md`
`git checkout HEAD -- README.md`	`git restore --source=HEAD --staged --worktree -- README.md`
`git checkout <hash>`	`git switch --detach <hash>`

Scripts and old notes still say `checkout`. Translate them with the table. Type `switch` and `restore` yourself.

Switch branches

```
cd ~/desk/desk-api
git switch main
git switch -c feat/switch-lab
git switch main
git switch -
git branch --show-current
```

`git switch -` uses `@{-1}`, the checkout you were on before this one. The last command prints `feat/switch-lab`.

```
git switch main
git branch -d feat/switch-lab
```

`-d` deletes a branch whose tip is already reachable from `HEAD`. You are on `main`, and `feat/switch-lab` names that same commit, so the delete succeeds.

A path is not a branch

```
git switch README.md
```

Git stops with `fatal: invalid reference: README.md`. The file is unchanged.

Local edits that would be overwritten

```
git switch -c feat/switch-lab
echo "other side" >> README.md
git add README.md
git commit -m "lab: other side"
git switch main
echo "main side" >> README.md
git switch feat/switch-lab
```

Git aborts. The message still says the files would be overwritten by checkout. `README.md` on `main` still ends with `main side`. `feat/switch-lab` still has `other side`.

```
git restore README.md
git branch -D feat/switch-lab
git status
```

`-D` deletes a branch even when `main` does not contain its commits. Use it here to discard the throwaway `lab: other side` commit. When you need to keep the uncommitted line and still change branches, stash it (chapter 12).

Detached HEAD

```
git switch --detach HEAD
git switch main
```

`--detach` points `HEAD` at a commit and at no branch. Chapter 1 is the trap: a commit made there needs a new branch before you leave it. From a detached `HEAD` with no new commit, `git switch main` attaches you again.

The lab next walks one scratch commit through soft, mixed, and hard, then brings it back from the reflog.

Hands-On: Reset Lab

Hands-On: Reset Lab

Walk one commit through soft, mixed, and hard on `desk-api`. Lose the line on purpose, and take it back from the reflog.

Setup

```
cd ~/desk/desk-api
git switch main
git status
START=$(git rev-parse HEAD)
echo "$START"
```

Status is clean, aside from ignored files such as `.env`. If `git switch main` refuses, restore or commit the work left from the previous page.

```
echo "lab line" >> README.md
git add README.md
git commit -m "lab: scratch line"
```

Soft

```
git reset --soft HEAD~1
git status
git diff --staged -- README.md
```

The line is under "Changes to be committed". `git log --oneline` does not show `lab: scratch line`.

```
git commit -m "lab: scratch line"
```

Same tree, new hash. The next mode starts from a commit again.

Mixed

```
git reset HEAD~1
git status
git diff -- README.md
```

Git prints `Unstaged changes after reset:`. The line is in the file and not in the index.

```
git add README.md
git commit -m "lab: scratch line"
```

Hard, and an untracked file

```
echo "not tracked" > notes.tmp
git reset --hard HEAD~1
git status
grep "lab line" README.md || echo "line gone"
ls notes.tmp
```

Tracked files match the commit in `$START`. `notes.tmp` is still in the directory. `--hard` left it there.

Recover with the reflog

```
git reflog -5
```

The first line is this reset (`HEAD@{0}`). The next line (`HEAD@{1}`) is `commit: lab: scratch line`. `HEAD@{1}` means "where `HEAD` was one move ago." That is a different question from `HEAD~1`, the parent of the current commit. Chapter 17 is that difference in full.

```
git reset --hard HEAD@{1}
grep "lab line" README.md
git log --oneline -1
```

The line is back. `main` names the scratch commit again.

Leave the repo as you found it

```
git reset --hard "$START"
rm -f notes.tmp
git status
git log --oneline -5
```

`$START` is the tip from the start of the lab, so this last reset also drops `lab: scratch line`.

Done when

- `git status` is clean (an ignored `.env` is fine)
- `notes.tmp` is gone
- the log has no `lab: scratch line`
- `README.md` has no `lab line`
- you saw that line staged, then unstaged, then missing, then restored from `HEAD@{1}`

Chapter 8 creates branches from this same `desk-api`. A reset of shared `main` waits; merging and rebasing come first.

Branching and Merging

Theory

Theory: A Branch is a Name; a Merge is a Node with Two Parents

Creating a branch does not copy files. It writes a ref. Merging either **slides** that ref (fast-forward) or **writes a commit with two parents** (a join).

Invariants

1. `git switch -c feat` = write `refs/heads/feat` at current `HEAD`, point `HEAD` at it.
2. Fast-forward is legal iff the current tip is an ancestor of the other tip.
3. A three-way merge diffs **base→ours** and **base→theirs** and applies both to the base. The merge base is the unique best common ancestor (usually).
4. Conflict markers are unmerged index stages (1 base, 2 ours, 3 theirs) spilled into the working tree.
5. During a merge, `--ours` is `HEAD`. During a rebase, `--ours` is the **upstream**. That swap is the theory, not a bug.

```
fast-forward          three-way merge

A ← B ← C   main      A ← B ← C ← M   main
          ← D ← E           ↑     ↑
                            D ← E
```

Why merge commits exist

They record *that two lines of work met*, and they keep both sides' hashes stable. Rebase prefers a straight first-parent chain and pays with new hashes. Neither changes the fact that Git stores snapshots.

What can break

- Merging with uncommitted work that overlaps the incoming tree.
- Resolving markers but forgetting `git add` (merge stays open).
- `git merge --abort` after the merge commit exists (too late).

How to read this chapter

Create/switch, merge types, three-way, conflicts, fast-forward, lab; then mergetool, ours/theirs, orphan, comparing branches, rerere.

Creating and Switching Branches

Branch to Create a Safe Working Area

After this page you can create, switch, list, and delete branches on Linux without touching `main`.

A branch is a name

```
cd ~/desk/desk-api
git branch
git switch -c feat/ping-json
git branch
```

`git switch -c` creates `feat/ping-json` at current `HEAD` and points `HEAD` at it. `git branch` (no args) lists local names; `*` is where HEAD is.

```
git switch main
git switch feat/ping-json
```

`git switch` is the modern command. `git checkout` still works; this book uses `switch` for branches.

Name them so later pages stay readable

```
feat/…     a capability
fix/…      a bug
chore/…    tooling
```

Slashes are fine on Linux. The name is a file under `.git/refs/heads/` (or a line in `packed-refs`). `feat/ping-json` is the path `refs/heads/feat/ping-json`.

Work on the branch

```
git switch feat/ping-json
# edit, add, commit as in chapter 3
git log --oneline --graph --all --decorate
```

`main` stays where you left it. Commits on `feat/ping-json` are extra nodes in the DAG.

```
A ← B ← C                 main

            D ← E         feat/ping-json   HEAD
```

List and see

```
git branch -v                 # last commit on each
git branch --show-current
git log --oneline main..HEAD  # commits on this branch that main does not have
```

`A..B` means reachable from `B`, not from `A`.

Delete

```
git switch main
git branch -d feat/ping-json     # refuses if not merged
git branch -D feat/ping-json     # delete anyway
```

`-d` is the default you want. `-D` is when you meant to throw the line of work away.

Rename

```
git branch -m feat/ping-json feat/ping
```

Renames the current branch if you omit the old name. After you publish a branch, renaming locally does not rename it on the remote; chapter 11.

Stuck with a dirty working tree

Git refuses to switch if the switch would overwrite uncommitted edits. Commit, stash (chapter 12), or restore first.

```
git status
git switch main
```

Merge Types

Merge Types: Fast-Forward, Recursive, and Octopus

After this page you can name the three merge shapes Git actually writes.

Fast-forward

`main` is an ancestor of `feat/ping-json`. Merging can just slide `main` forward. No new commit.

```
A ← B ← C                 main
          ← D ← E         feat/ping-json

# after git switch main && git merge feat/ping-json

A ← B ← C ← D ← E         main, feat/ping-json
```

```
git switch main
git merge feat/ping-json
```

`git log` shows a straight line. `git merge --no-ff` forces a merge commit even when a fast-forward is possible (Git Flow likes that; trunk-based usually does not).

Recursive (two parents)

The branches diverged. Git writes a **merge commit** with two parents.

```
A ← B ← C ← M             main
     ↑      ↑
     D ← E                feat/ping-json
```

The default strategy for two heads is `ort` (the current "recursive" replacement). You rarely pass `-s`. Git finds a **merge base** (common ancestor), diffs both sides against it, and applies both diffs to the base.

```
git switch main
git merge feat/ping-json -m "merge feat/ping-json"
git log --oneline --graph --decorate
```

Octopus (three or more parents)

```
git merge topic-a topic-b topic-c
```

One merge commit with several parents. Git only does this when there are **no** overlapping conflicts. If two topics touch the same hunk, octopus refuses; merge them pairwise instead. You will almost never need octopus on `desk-api`. Know the name so `git log --graph` with a commit that has three parents is not a mystery.

```
A ← B ← C ← M          main     M has three parents
     ↑   ↑   ↑
     D   E   F          three topic tips
```

See the parents

```
git cat-file -p HEAD
git rev-parse HEAD^1 HEAD^2
```

`HEAD^1` is the first parent (the branch you had checked out). `HEAD^2` is what you merged in.

Abort

```
git merge --abort
```

Returns the three trees to the pre-merge state while a merge is in progress (`MERGE_HEAD` exists). After the merge commit exists, abort is too late; `git reset` (chapter 7) or `git revert -m 1` (chapter 10).

Three-Way Merge

Three-Way Merge Explained

After this page you can draw the three inputs Git uses when two branches diverged.

The three trees

Call them:

Name	What
Base	Merge base: latest common ancestor of both tips
Ours	The branch you have checked out (`HEAD`)
Theirs	The branch you named in `git merge`

Git diffs base→ours and base→theirs, then applies both diffs to the base. That is a **three-way merge**. A two-way merge (ours vs theirs only) cannot tell "we both added the same line" from "they added a line we never had."

```
          C  base     (merge-base main feat)
         / \
        /   \
   G    E            ours = main, theirs = feat
    \   /
     \ /
      M  result      two-parent merge commit
```

See the base

```
cd ~/desk/desk-api
git merge-base main feat/ping-json
git log --oneline --graph main feat/ping-json
```

`merge-base` prints one hash. `git log --graph` should make that node obvious.

Stages in the index during a conflict

When both sides changed the same hunk, Git records three stages for that path in the index:

Stage	Content
1	base
2	ours
3	theirs

```
git ls-files -u
```

`git checkout --ours path` / `--theirs path` copies stage 2 or 3 into the working tree. You still `git add` after you edit. The next page is the full conflict loop.

Why this matters for a forge

A pull request / merge request is this same three-way merge, run on the server when you click Merge. The conflict you see in the UI is stages 2 and 3 plus markers. Learning it locally means the UI is not a new theory.

`zdiff3` markers

With `merge.conflictstyle zdiff3` (chapter 2) a conflicted file shows the common ancestor as well as ours and theirs. That extra block is the base. Read it before you delete markers.

Merge Conflicts

Merge Conflicts: In Git and in Teams

After this page you can finish a conflicted merge on Linux and leave a clean `main`.

What a conflict is

Both branches changed the same region of a file (or one side changed a file the other deleted). Git writes **conflict markers** into the working tree and leaves the merge unfinished.

```
<<<<<<< HEAD
return "ok"
=======
return {"status": "ok"}
>>>>>>> feat/ping-json
```

With `zdiff3` you also get an ancestor block (`|||||||`).

The loop

```
git switch main
git merge feat/ping-json
# Git stops and prints "fix conflicts and then commit"

git status
# both modified: src/ping.py
```

Edit the file in `nvim`. Remove every `<<<<<<<`, `=======`, `>>>>>>>` (and `|||||||` if present). Leave the code you want.

```
git add src/ping.py
git status          # "all conflicts fixed"
git commit          # default message is "Merge branch 'feat/ping-json'"
```

Git opens `core.editor` for the merge message if you omit `-m`. A conventional subject is optional here; merge commits are often left as Git's default so `git log --first-parent` stays readable.

Abort vs continue

```
git merge --abort     # give up; back to pre-merge
```

After `git add`, there is no `--continue` for merge; `git commit` finishes it. Rebase *does* use `--continue` (chapter 9).

Binary files and deletes

`git status` will say `deleted by them` or `added by us`. Pick a side:

```
git restore --ours --worktree --staged path
git restore --theirs --worktree --staged path
git rm path
```

Then `git add` / `git commit` as usual.

Teams

A conflict is a **data** problem (two diffs) plus a **people** problem (whose behavior should win). On a forge, the same markers appear in the pull-request UI. Resolve on your laptop, commit, push. Do not resolve only in the UI if you cannot run the tests; Linux is where `desk-api` actually runs.

`rerere` (chapter 2) records how you resolved a hunk and reapplies it if the same conflict comes back. Leave it enabled.

After the merge

```
git log --oneline --graph --decorate --all
git branch -d feat/ping-json
```

`-d` succeeds once the work is reachable from `main`.

Fast-Forward for Clean History

Creating a Fast-Forward Merge for a Clean History

After this page you can keep `main` a straight line when the feature branch is ahead of it.

When a fast-forward is possible

`main` has not moved since you branched. Every commit on the feature is a descendant of `main`.

```
before --ff-only                    after --ff-only

A ← B ← C          main             A ← B ← C ← D ← E   main
          ← D ← E  feat                         feat
```

```
cd ~/desk/desk-api
git switch main
git merge --ff-only feat/ping-json
```

`--ff-only` **fails** if Git would need a merge commit. That is the setting you already put in `pull.ff only` for pulls. Use it on purpose for local merges when you want a linear `main`.

If it fails

Someone committed to `main` while you worked. Options:

1. Merge anyway (`git merge feat/ping-json`) → merge commit, history shows the join.
2. Rebase the feature onto `main` (chapter 9), then `--ff-only`.
3. `git merge --no-ff feat/ping-json` → always a merge commit (Git Flow release style).

See it

```
git log --oneline --graph --decorate
```

A fast-forward looks like a single chain. A merge commit is a join with two parents.

`first-parent`

```
git log --first-parent --oneline
```

Walks only `^1`. After `--no-ff` merges, this is "what landed on `main`." After a fast-forward, it is the same as a normal log. Chapter 15 picks a branching strategy; this page is the mechanism that strategy will call.

Hands-On: A Real Merge Conflict

Hands-On: Simulate a Real Merge Conflict

Two branches edit `src/ping.py` on Linux. You merge, resolve markers, and delete the feature branch.

Setup

Start from a clean `~/desk/desk-api` (chapter 3 hands-on).

```
cd ~/desk/desk-api
git switch main
git status
```

Branch A — JSON ping

```
git switch -c feat/ping-json
cat > src/ping.py <<'EOF'
def ping():
    return {"status": "ok"}
EOF
git add src/ping.py
git commit -m "feat(ping): return a dict"
git switch main
```

Branch B — string change on main

```
cat > src/ping.py <<'EOF'
def ping():
    return "OK"
EOF
git add src/ping.py
git commit -m "feat(ping): shout the status"
```

`main` and `feat/ping-json` have diverged.

```
A ← B ← C ← shout     main           ping.py returns "OK"

            dict      feat/ping-json  ping.py returns {"status": "ok"}
```

Merge and conflict

```
git merge feat/ping-json
git status
cat src/ping.py
```

You should see markers. Open the file in `nvim` (or your editor) and make it:

```
def ping():
    return {"status": "OK"}
```

No marker lines left.

```
git add src/ping.py
git commit -m "merge feat/ping-json"
git log --oneline --graph --decorate -8
git branch -d feat/ping-json
```

Done when

- `git status` is clean on `main`
- `src/ping.py` returns a dict with `"OK"`
- `feat/ping-json` is deleted
- `git log --graph` shows a merge commit (two parents)

If you want to retry: `git reset --hard` to the commit before the merge (only on this lab clone), or copy the repo and run the page again. Chapter 9 rebases this same divergence into a straight line instead.

mergetool

mergetool

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- `git mergetool`, `nvimdiff`, `meld` on Linux.

> **Note**
>
> Core Git depth (Linux host). See `content/_planning/SYLLABUS.md`.

ours, theirs, and Renames

ours, theirs, and Renames

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- `--ours` / `--theirs`, rename detection, `merge.renamelimit`.

> **Note**
>
> Core Git depth (Linux host). See `content/_planning/SYLLABUS.md`.

Orphan Branches

Orphan Branches

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- `git switch --orphan` for `gh-pages`-style roots.

> **Note**
>
> Core Git depth (Linux host). See `content/_planning/SYLLABUS.md`.

Comparing Branches

Comparing Branches

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- `git log A..B`, `git diff A...B`, `git merge-base`.

> **Note**
>
> Core Git depth (Linux host). See `content/_planning/SYLLABUS.md`.

rerere

rerere

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Reuse recorded resolution; when it helps a long-lived feature.

> **Note**
>
> Core Git depth (Linux host). See `content/_planning/SYLLABUS.md`.

Git 2.56 for Daily Work

Git 2.56 for Daily Work: `add --resolved`, `branch --delete-merged`, and Friends

Thesis

Git 2.56.0 (tagged 28 September 2026) adds two commands to the end of the merge cycle that most people have been scripting by hand:

- **`git add --resolved`** stages exactly the conflicted paths you have finished, and nothing else. It refuses the whole batch if any of them still contain conflict markers.
- **`git branch --delete-merged <upstream>`** deletes local branches whose work has already landed on the upstream they track. Use `--dry-run` to preview it.

There are also three smaller tools: `git bisect run --reset-when-found` cleans up after itself, `git history drop` removes one commit from a branch (and the branches stacked on it) without an interactive rebase, and `git branch --forked` lists branches by what they track. On top of that, `git merge-base` and `git branch --contains` got faster without any change on your side.

Each new flag is a sharp, narrow tool. `--resolved` only knows what a conflict *marker* is, so it does not know whether you resolved a conflict well. `--delete-merged` only removes branches that track something other than their own push destination, and most people's branches don't. This page shows both behaviours on real repositories, so you know which of your branches and conflicts the new flags will and won't handle.

Everything below was run with `git version 2.56.0`. Earlier versions reject these options as unknown.

Mental model

```
end of a merge                                    end of a topic's life

  git status --short                               local          upstream
   M README.md   (yours, unrelated)                fix/typo      origin/main  tip reachable
  UU app.py      (resolved)                        feat/wip      origin/main  no
  UU config.toml (resolved)                        feat/x        origin/feat/x  (push target

  git add -u          stages all three            git branch --delete-merged origin
  git add --resolved  stages only the UU paths      deletes fix/typo only
                      and refuses if any still
                      has <<<<<<< / ======= / >>>>>>>
```

Command	What it does	What it does *not* do
`git add --resolved [<pathspec>]`	Stages unmerged paths that have no conflict markers left. One leftover marker anywhere and nothing is staged	It does not look at binary or modify/delete conflicts. Those have no markers, so they are staged as whatever is in the working tree
`git branch --delete-merged <pattern> [<branch-glob>…]`	Deletes local branches whose **upstream** matches `<pattern>` (a ref, a remote name meaning its `HEAD`, or a glob) and whose tip is reachable from that upstream	It skips branches that track their own push destination, branches whose upstream is gone, and squash-merged branches, all without a word
`--dry-run`	With `--delete-merged`, prints `Would delete branch …` and touches nothing	
`branch.<name>.deleteMergedFilter`	Exempts matching branches, and says so (`Skipping '…'`)	Does not affect `git branch -d`
`git branch --forked <pattern>`	Lists branches by upstream. It's the same match `--delete-merged` uses, without the reachability check	
`git bisect … --reset-when-found [<original-branch>]`	Ends the bisection by itself checking out the original branch once the first bad commit is found	Can't be combined with `--no-checkout`
`git history drop <commit>`	Removes one commit and replays its descendants, including other branches stacked on top	It won't drop root or merge commits, and it won't stop for conflicts. It aborts instead

Worked examples (Cases)

Every case uses throwaway repositories under `/tmp/g256`. Source this file first so your hashes match the transcripts. It also shuts out your global config, so your aliases and `push.default` don't change the results:

```
# env.sh - source this so your hashes match the transcripts (needs git 2.56+)
export GIT_AUTHOR_NAME="Dev" GIT_AUTHOR_EMAIL="dev@example.com"
export GIT_COMMITTER_NAME="Dev" GIT_COMMITTER_EMAIL="dev@example.com"
export GIT_AUTHOR_DATE="2026-10-01T09:00:00Z" GIT_COMMITTER_DATE="2026-10-01T09:00:00Z"
export GIT_CONFIG_GLOBAL=/dev/null GIT_CONFIG_NOSYSTEM=1
export GIT_CONFIG_COUNT=2
export GIT_CONFIG_KEY_0=init.defaultBranch GIT_CONFIG_VALUE_0=main
```

```
export GIT_CONFIG_KEY_1=advice.detachedHead GIT_CONFIG_VALUE_1=false
```

Case 1: Stage what you resolved, not what you happened to touch

`git add -u` after a conflict is a habit, and it's how unrelated edits end up inside merge commits. Set up a merge with two conflicts while an uncommitted note sits in `README.md`:

```
#!/usr/bin/env bash
# setup-conflict.sh - a merge with two conflicted files and one unrelated local edit
set -euo pipefail
rm -rf /tmp/g256/conflict && mkdir -p /tmp/g256/conflict && cd /tmp/g256/conflict
git init -q
printf 'port = 8080\nlog = "info"\n' > config.toml
printf 'def greet():\n    return "hello"\n' > app.py
printf '# demo\n' > README.md
git add . && git commit -qm "base"
git switch -qc feature
printf 'port = 9090\nlog = "info"\n' > config.toml
printf 'def greet():\n    return "hello, world"\n' > app.py
git commit -qam "feature: new port and greeting"
git switch -q main
printf 'port = 8081\nlog = "info"\n' > config.toml
printf 'def greet():\n    return "hi"\n' > app.py
git commit -qam "main: different port and greeting"
echo "local note, not part of the merge" >> README.md   # unrelated, uncommitted
git merge feature || true
```

```
$ source env.sh && ./setup-conflict.sh
Auto-merging app.py
CONFLICT (content): Merge conflict in app.py
Auto-merging config.toml
CONFLICT (content): Merge conflict in config.toml
Automatic merge failed; fix conflicts and then commit the result.
$ cd /tmp/g256/conflict
$ git status --short
 M README.md
UU app.py
UU config.toml
$ printf 'def greet():\n    return "hello, world"\n' > app.py
$ git add --resolved; echo "exit=$?"
fatal: the following paths still have conflict markers:
    config.toml

exit=128
$ git status --short
```

```
 M README.md
UU app.py
UU config.toml
$ printf 'port = 9090\nlog = "info"\n' > config.toml
$ git add --resolved
$ git status --short
 M README.md
M  app.py
M  config.toml
$ git add --resolved -u
fatal: options '-u/--update' and '--resolved' cannot be used together
$ git commit -q --no-edit
$ git status --short
 M README.md
```

Three things to read off this:

1. With `app.py` fixed and `config.toml` still marked, `--resolved` staged **nothing**, not even `app.py`. It is all or nothing on purpose, so the index never shows a half-finished merge as resolved.
2. Once both were clean it staged both, and `README.md` stayed `M`. The merge commit holds the merge and nothing else, and your note is still waiting in the working tree.
3. `--resolved` can't be combined with `-u` or `-A`, because those mean "stage everything tracked". You can narrow it with a pathspec: `git add --resolved src/` stages only the resolved paths under `src/`.

Case 2: Set topics up the way `--delete-merged` expects

`--delete-merged` matches on a branch's **upstream**, so it works best when each topic tracks the branch it will land on (`origin/main`) and is pushed to a branch of its own name. Git already does the first half for you. A branch started from a remote-tracking branch gets that branch as its upstream (`branch.autoSetupMerge=true` is the default). The push needs one setting:

```
$ git switch -c fix/readme origin/main
Switched to a new branch 'fix/readme'
branch 'fix/readme' set up to track 'origin/main'.
$ echo r > r.txt && git add r.txt && git commit -qm 'fix/readme'
$ git push
fatal: The upstream branch of your current branch does not match
the name of your current branch.  To push to the upstream branch
on the remote, use

    git push origin HEAD:main

To push to the branch of the same name on the remote, use
```

```
    git push origin HEAD

To choose either option permanently, see push.default in 'git help config'.

To avoid automatically configuring an upstream branch when its name
won't match the local branch, see option 'simple' of branch.autoSetupMerge
in 'git help config'.

$ git config push.default current
$ git push
To /tmp/g256/br/origin.git
 * [new branch]      fix/readme -> fix/readme
$ git branch -vv --list fix/readme
* fix/readme 08b47a1 [origin/main: ahead 1] fix/readme
```

`push.default=simple`, the default, refuses to push a branch to a name that differs from its upstream. `push.default=current` pushes `fix/readme` to `origin/fix/readme` and leaves the upstream on `origin/main`. Now `git status` says how far you are from `main`, which is the comparison you care about for a topic. Once the work is merged, `--delete-merged origin` will recognise it.

Case 3: Preview, then sweep

This script builds a clone with eight topic branches (plus `main`), covering every state you'll meet: merged, unfinished, squash-merged, tracking its own remote branch, upstream deleted, checked out in another worktree, opted out, and tracking a release branch that never got the fix:

```
#!/usr/bin/env bash
# setup-branches.sh - a clone with eight local branches in different states
set -euo pipefail
rm -rf /tmp/g256/br && mkdir -p /tmp/g256/br && cd /tmp/g256/br
git init -q --bare origin.git
git clone -q origin.git work 2>/dev/null && cd work
echo base > app.txt && git add app.txt && git commit -qm "base" && git push -q origin main

topic() {  # topic <branch> <upstream>: branch from origin/main tracking <upstream>, one com
  git switch -qc "$1" --track "$2" 2>/dev/null || git switch -qc "$1" origin/main
  echo "$1" > "${1//\//-}.txt" && git add . && git commit -qm "$1"
}

# 1. merged: tracks origin/main, merged upstream with a merge commit
topic fix/typo origin/main;        git push -q origin fix/typo
# 2. not merged yet
topic feat/wip origin/main
# 3. squash-merged upstream (different commit, same content)
topic feat/squashed origin/main;   git push -q origin feat/squashed
# 4. tracks its own remote branch, merged, then the remote branch was deleted
```

```
git switch -qc feat/gone origin/main && echo gone > gone.txt && git add . && git commit -qm
# 5. tracks its own remote branch, merged, remote branch still there
git switch -qc feat/same-name origin/main && echo same > same.txt && git add . && git commit
# 6. merged, but checked out in a linked worktree
topic fix/in-worktree origin/main; git push -q origin fix/in-worktree
# 7. merged, but opted out
topic feat/round-two origin/main;  git push -q origin feat/round-two
git config branch.feat/round-two.deleteMerged false
# 8. merged into main, tracks a release branch that does not have it
git push -q origin main:release/1.0
git fetch -q
topic fix/backport origin/release/1.0; git push -q origin fix/backport
git switch -q main

# "upstream" merges everything except feat/wip; feat/squashed is squashed
rm -rf ../hub && git clone -q ../origin.git ../hub 2>/dev/null && cd ../hub
for b in fix/typo feat/gone feat/same-name fix/in-worktree feat/round-two fix/backport; do
  git merge -q --no-ff --no-edit "origin/$b" >/dev/null
done
git merge -q --squash origin/feat/squashed >/dev/null 2>&1 && git commit -qm "feat/squashed
git push -q origin main
git push -q origin --delete feat/gone
cd ../work
git fetch -q --prune
git worktree add -q ../wt fix/in-worktree
```

```
$ source env.sh && ./setup-branches.sh && cd /tmp/g256/br/work
$ git branch -vv
  feat/gone       db5921d [origin/feat/gone: gone] feat/gone
  feat/round-two  e157f1f [origin/main: behind 12] feat/round-two
  feat/same-name  0401489 [origin/feat/same-name] feat/same-name
  feat/squashed   d74a4cc [origin/main: ahead 1, behind 13] feat/squashed
  feat/wip        2fc13cf [origin/main: ahead 1, behind 13] feat/wip
  fix/backport    d76cc42 [origin/release/1.0: ahead 1] fix/backport
+ fix/in-worktree 15ead81 (/tmp/g256/br/wt) [origin/main: behind 12] fix/in-worktree
  fix/typo        fdcf8be [origin/main: behind 12] fix/typo
* main            9d063a9 [origin/main: behind 13] base
$ git branch --forked origin/main
  feat/round-two
  feat/squashed
  feat/wip
+ fix/in-worktree
  fix/typo
* main
$ git branch --dry-run --delete-merged origin
Skipping 'feat/round-two' (branch.feat/round-two.deleteMerged is false)
```

```
Would delete branch fix/typo (was fdcf8be).
$ git branch --dry-run --delete-merged origin 'fix/*'
Would delete branch fix/typo (was fdcf8be).
$ git branch --delete-merged origin
Skipping 'feat/round-two' (branch.feat/round-two.deleteMerged is false)
Deleted branch fix/typo (was fdcf8be).
$ git branch --list
  feat/gone
  feat/round-two
  feat/same-name
  feat/squashed
  feat/wip
  fix/backport
+ fix/in-worktree
* main
```

Read the result branch by branch:

Branch	Outcome	Why
`fix/typo`	**deleted**	Tracks `origin/main`, and its tip is reachable from it
`feat/round-two`	kept, *reported*	`branch.feat/round-two.deleteMe`
`fix/in-worktree`	kept, silent	Checked out in `/tmp/g256/br/wt`
`feat/wip`	kept, silent	Not merged yet
`feat/squashed`	kept, silent	A squash merge creates a new commit, so the branch tip is never reachable from `origin/main`
`feat/same-name`, `main`	kept, silent	`git push` would update their own upstream. Right after a pull, such a branch is always "merged", so Git refuses to guess
`feat/gone`	kept, silent	Its upstream `origin/feat/gone` was pruned after the merge
`fix/backport`	kept, silent	Merged into `main`, but its upstream `origin/release/1.0` doesn't have it

Passing `origin` means "the branch `origin/HEAD` points at", so `origin/main` here. `'origin/*'` widens the match to every upstream on that remote, and you can repeat `--delete-merged` to name several. A trailing `'fix/*'` limits which *local* branches are even considered. `--forked` uses the same upstream match as a listing filter, so `git branch --forked origin/main` is a cheap way to see the candidates before anything is checked for reachability.

Deleted branches print their old tip (`was fdcf8be`). Until `gc` prunes the commits, `git branch fix/typo fdcf8be` puts it back.

Case 4: A bisect that cleans up after itself

`git bisect run` used to leave you on a detached `HEAD` in the middle of the bisection state until you remembered `git bisect reset`. Now it can finish the job:

```
#!/usr/bin/env bash
# setup-bisect.sh - 16 commits; commit 11 breaks check.sh
set -euo pipefail
rm -rf /tmp/g256/bisect && mkdir -p /tmp/g256/bisect && cd /tmp/g256/bisect
git init -q
printf '#!/bin/sh\ngrep -q "^limit=10$" limits.conf\n' > check.sh && chmod +x check.sh
echo "limit=10" > limits.conf
git add . && git commit -qm "c1: add check"
for i in $(seq 2 16); do
  [ "$i" -eq 11 ] && echo "limit=100" > limits.conf
  echo "change $i" >> CHANGELOG && git add . && git commit -qm "c$i"
done
git switch -qc topic && git switch -q main
```

```
$ source env.sh && ./setup-bisect.sh && cd /tmp/g256/bisect
$ git switch -q topic
$ git bisect start main main~15
Bisecting: 7 revisions left to test after this (roughly 3 steps)
[977dff1cd080a1d2dc267fd03fad3d5bd34aedae] c8
$ git bisect run --reset-when-found ./check.sh
running './check.sh'
Bisecting: 3 revisions left to test after this (roughly 2 steps)
[17edb82aa1019346897ab8763f8baf74af113808] c12
running './check.sh'
Bisecting: 1 revision left to test after this (roughly 1 step)
[e7ccdbe0a5c6e71b2fea6e0f448dc4aba80c7f52] c10
running './check.sh'
Bisecting: 0 revisions left to test after this (roughly 0 steps)
[09ee7e9ff3cff397a539e7e73d2b9ac5d3a6ff5f] c11
running './check.sh'
09ee7e9ff3cff397a539e7e73d2b9ac5d3a6ff5f is the first 'bad' commit
commit 09ee7e9ff3cff397a539e7e73d2b9ac5d3a6ff5f
```

```
Author: Dev <dev@example.com>
Date:   Thu Oct 1 09:00:00 2026 +0000

    c11

 CHANGELOG   | 1 +
 limits.conf | 2 +-
 2 files changed, 2 insertions(+), 1 deletion(-)
bisect found first 'bad' commit
$ git status --short --branch
## topic
$ git bisect log
error: We are not bisecting.
$ git bisect start --reset-when-found=found main main~15
Bisecting: 7 revisions left to test after this (roughly 3 steps)
[977dff1cd080a1d2dc267fd03fad3d5bd34aedae] c8
$ git bisect run ./check.sh >/dev/null
$ git status --short --branch
## HEAD (no branch)
$ git log -1 --oneline
09ee7e9 c11
```

`--reset-when-found` (that is, `=original`) put us back on `topic`, with no bisection state left: `We are not bisecting.` `=found` leaves the culprit checked out (detached at `09ee7e9 c11`) and still clears the state. That's what you want when the next step is `git show` or a build of the bad commit. You can give the option to `bisect start`, as in the second run, or to `bisect run`. It's refused with `--no-checkout` (`options '--reset-when-found' and '--no-checkout' cannot be used together`), because there is no checkout to return from.

Case 5: Drop one commit without an interactive rebase

`git history` is still marked experimental. 2.56 adds `drop` next to `reword`, `split`, and `fixup`:

```
#!/usr/bin/env bash
# setup-history.sh - a short topic with one commit that should never have happened
set -euo pipefail
rm -rf /tmp/g256/hist && mkdir -p /tmp/g256/hist && cd /tmp/g256/hist
git init -q
echo "app" > app.txt && git add . && git commit -qm "base"
git switch -qc feat/report
echo "query"  > query.sql  && git add . && git commit -qm "add report query"
echo "debug=1" > .env.local && git add . && git commit -qm "WIP: local debug settings"
echo "render" > render.py  && git add . && git commit -qm "render report"
git branch feat/report-ui                       # a branch stacked on top
echo "ui" > ui.js && git switch -q feat/report-ui && git add . && git commit -qm "report UI"
git switch -q feat/report
```

```
$ source env.sh && ./setup-history.sh && cd /tmp/g256/hist
$ git log --oneline --decorate --graph feat/report-ui
* 4ff5f05 (feat/report-ui) report UI
* 6021dd1 (HEAD -> feat/report) render report
* e3d4de1 WIP: local debug settings
* 99efa11 add report query
* e135d0b (main) base
$ git history drop HEAD~1 --dry-run
update refs/heads/feat/report 5c85b08fe93e8dcbebb35bcd5e0fd6b7b8b725df 6021dd1b2c922ee1fa03f
update refs/heads/feat/report-ui 83fe96025062c342ad0ad189b968ba7766bd58f1 4ff5f05abfefe16eb8
$ git history drop HEAD~1
$ git log --oneline --decorate --graph feat/report-ui
* 83fe960 (feat/report-ui) report UI
* 5c85b08 (HEAD -> feat/report) render report
* 99efa11 add report query
* e135d0b (main) base
$ ls
app.txt
query.sql
render.py
```

`--dry-run` prints the ref updates it would make, in `update-ref --stdin` form (`update <ref> <new> <old>`). The real run made the same updates: `feat/report` and the stacked `feat/report-ui` both moved, so the stack stayed in one piece. To move only the current branch, pass `--update-refs=head`. Unlike `rebase -i`, it never stops halfway. If a descendant doesn't replay cleanly, nothing changes at all:

```
$ echo 'app v2' > app.txt && git commit -qam 'bump to v2'
$ echo 'app v3' > app.txt && git commit -qam 'bump to v3'
$ git history drop HEAD~1; echo "exit=$?"
error: failed replaying descendants
exit=255
$ git log --oneline -3
9be6c5a bump to v3
97e8f92 bump to v2
5c85b08 render report
```

The `9be6c5a` on top is the same commit as before the failed command. No sequencer state is left behind and there is no `--abort` to remember. When the drop needs a real conflict resolution, use `git rebase -i`.

What you get for free

There are two speedups with no new syntax, both from the release notes:

- **`git merge-base`** now stops walking once one side's exclusive commits run out, which makes queries with a one-sided history much faster. Merges, rebases, and `git log A...B` all compute

merge bases. The same release also fixes a case where `git merge-base` (without `--all`) could return a wrong answer on repositories with v1 commit-graphs and clock skew.
- `git branch --contains` and `git for-each-ref --contains` now use the memoised traversal that `git tag --contains` already had.

The release notes describe these as significant speedups without benchmark numbers. Measure on your own repository before you quote one.

The trap

`--resolved` checks for markers. It does not check that you resolved anything. Three common conflicts break that assumption in three ways:

```
#!/usr/bin/env bash
# setup-traps.sh - three conflicts that have no conflict markers to find
set -euo pipefail
rm -rf /tmp/g256/traps && mkdir -p /tmp/g256/traps && cd /tmp/g256/traps
git init -q
printf 'Release notes\n=======\n\n- v1\n' > NOTES.md
printf 'logo\0v1' > logo.png
echo "legacy" > legacy.cfg
git add . && git commit -qm "base"
git switch -qc side
printf 'Release notes\n=======\n\n- v2\n' > NOTES.md
printf 'logo\0v2' > logo.png
echo "legacy, patched" > legacy.cfg
git commit -qam "side: notes, new logo, patch legacy.cfg"
git switch -q main
printf 'Release notes\n=======\n\n- v3\n' > NOTES.md
printf 'logo\0v3' > logo.png
git rm -q legacy.cfg
git commit -qam "main: notes, other logo, drop legacy.cfg"
git merge side || true
```

```
$ source env.sh && ./setup-traps.sh >/dev/null && cd /tmp/g256/traps
$ git status --short
UU NOTES.md
DU legacy.cfg
UU logo.png
$ printf 'Release notes\n=======\n\n- v2\n- v3\n' > NOTES.md
$ git add --resolved
fatal: the following paths still have conflict markers:
    NOTES.md

$ git add --resolved logo.png legacy.cfg
```

```
$ git status --short
UU NOTES.md
A  legacy.cfg
$ git diff --cached --stat HEAD
 NOTES.md   | Unmerged
 legacy.cfg | 1 +
 1 file changed, 1 insertion(+)
```

- **False positive.** `NOTES.md` is cleanly resolved, but its Markdown heading is underlined with exactly seven `=`. That is the same shape as a conflict separator, so `--resolved` refuses. Either stage that file explicitly after reading it (`git add NOTES.md`), or set `NOTES.md conflict-marker-size=32` in `.gitattributes` *before* merging. Git then writes 32-character markers, and seven `=` no longer looks like one.
- **Silent "resolution" of a binary.** `logo.png` was `UU`, and now it isn't listed at all. `--resolved` staged the working-tree copy, which for a binary conflict is *ours*. The staged diff against `HEAD` is empty, so their logo is gone without a trace.
- **Silent resurrection.** `legacy.cfg` was deleted on `main` and modified on `side` (`DU`). It has no markers, so `--resolved` staged the surviving copy (`A legacy.cfg`) and brought back a file your branch had deleted on purpose.

The other trap is the flip side of Case 3: **`--delete-merged` is quiet about everything it doesn't delete.** If your team's flow is "push to `origin/<same-name>`, squash-merge on the forge, delete the remote branch", none of your branches qualify. They all track their own push destination, are squash-merged, or have a pruned upstream. The command prints nothing and deletes nothing, which looks the same as having nothing to clean up.

The boring rule

> After a conflicted merge, run `git status --short` and deal with every `DU`, `UD`, `AA`, and binary `UU` deliberately (`git checkout --ours/--theirs <path>`, or `git rm`). Then use `git add --resolved` for the text files and read any refusal it gives you. Never `git add -u` or `git add .` in the middle of a merge.
>
> Start topics from `origin/main`, set `push.default=current`, and clean up with `git fetch --prune && git branch --dry-run --delete-merged origin` before the real run. Mark long-lived topics with `branch.<name>.deleteMerged=false`. Delete squash-merged and same-name branches yourself with `git branch -D` after checking `git log origin/main..<branch>`.

Try this

1. Rerun Case 1, but after resolving both files use `git add -u` instead of `--resolved`. Commit with `git commit --no-edit`, then run `git show --stat HEAD` and find `README.md`, your unrelated note, inside the merge commit.

2. Copy `setup-traps.sh` and add `echo 'NOTES.md conflict-marker-size=32' > .gitattributes` just before the `base` commit. Rerun it, resolve `NOTES.md` the same way, and confirm that `git add --resolved NOTES.md` now succeeds while `logo.png` and `legacy.cfg` stay unmerged.
3. In the Case 3 clone, run `git branch --forked 'origin/*' -vv` and predict, before running it, what `git branch --dry-run --delete-merged 'origin/*'` will print.
4. In the Case 3 clone, run `git cherry origin/main feat/squashed` and `git cherry origin/main feat/wip`. A `-` line means an equivalent patch is already upstream, which is the evidence you want before `git branch -D` on a squash-merged branch.
5. Write a `git sweep` alias that runs `fetch --prune`, then `branch --dry-run --delete-merged origin`, asks `y/N`, and only then runs the real thing.

Sources

- Git 2.56.0 release notes: https://github.com/git/git/blob/v2.56.0/Documentation/RelNotes/2.56.0.adoc
- `git-add(1)`, `--resolved`: https://github.com/git/git/blob/v2.56.0/Documentation/git-add.adoc
- `git-branch(1)`, `--delete-merged`, `--dry-run`, `--forked`: https://github.com/git/git/blob/v2.56.0/Documentation/git-branch.adoc
- `git-config(1)`, `branch.<name>.deleteMerged`: https://github.com/git/git/blob/v2.56.0/Documentation/config/branch.adoc
- `git-bisect(1)`, `--reset-when-found`: https://github.com/git/git/blob/v2.56.0/Documentation/git-bisect.adoc
- `git-history(1)`, `drop`: https://github.com/git/git/blob/v2.56.0/Documentation/git-history.adoc
- `gitattributes(5)`, `conflict-marker-size`: https://git-scm.com/docs/gitattributes#_conflict_marker_size

Rebasing

Theory

Theory: Rebase Copies Commits; It Does Not Slide Them

A rebase walks a range of commits, takes each patch against its parent, and applies it onto a new base. Each successful apply writes a **new** commit (new hash, usually same author, you as committer).

Invariants

1. Original commits remain in the object store until GC; the branch **name** moves to the copied tip.
2. Conflicts happen **once per copied commit**, not once at a join.
3. Interactive rebase is the same machine plus a todo list (`pick`, `squash`, `fixup`, `reword`, `edit`, `drop`).
4. `pull --rebase` = fetch + rebase local-only commits onto `@{u}`.
5. `--onto` is "take this range and replay it on that commit," the general form of rebase.

```
before                          after git rebase main

A ← B ← C ← G     main          A ← B ← C ← G           main
     ↑
     D ← E        feat                       D' ← E'  feat
```

Why teams fight about it

Merge preserves hashes and adds a node. Rebase rewrites hashes and can keep `main` linear. Unpublished feature branches are the safe audience. Published `main` is not.

What can break

- Rebasing commits other people have based work on (force-push incident).
- `--ours`/`--theirs` confusion vs merge.
- Empty commits after rebase (`--skip` vs keep).

How to read this chapter

Why, interactive, conflicts, pull –rebase, vs merge, lab; then `--onto`, autosquash, `--exec`, `--update-refs`.

Why Rebase

Why Rebase? Clean History vs Merge Commits

After this page you can say what rebase copies, and why the hashes change.

Replay commits on a new parent

You branched from `main` at `C`. `main` moved to `G`. Rebase copies your commits `D` and `E` as `D'` and `E'` with parent `G`:

```
A ← B ← C ← G                 main
          ← D ← E             feat/ping-json

# git switch feat/ping-json && git rebase main

A ← B ← C ← G                 main
              ← D' ← E'       feat/ping-json
```

`D'` has a new hash. The patch is similar; the parent is `G`. Git then fast-forwards or you `--ff-only` merge into `main`.

```
cd ~/desk/desk-api
git switch feat/ping-json
git rebase main
```

What you gain

- `main` can stay a single chain (fast-forward).
- Reviewers read your commits on top of current `main`, not an old base.
- `git log --first-parent` on `main` after `--ff-only` is your feature commits, not a merge node.

What you pay

- New hashes. Anyone who fetched `D` and `E` now has stale names.
- Conflicts are replayed **per copied commit**, not once in a merge commit.

Rebase **unpublished** work (your feature branch). Merge **published** history you share (`main` that others pulled). Chapter 11 is force-push if you already pushed the old `D` and `E`.

`rebase` vs `merge` in one line

Merge adds a join. Rebase copies a chain onto a new base. Both produce a tree that contains both sides of the work. The graph looks different. The next pages are interactive rebase, conflicts, `pull --rebase`, and when to pick which.

Interactive Rebase

Interactive Rebase: Squash, Reorder, Fixup, Polish

After this page you can rewrite a short chain of local commits on Linux without a GUI.

Open the todo list

```
cd ~/desk/desk-api
git switch feat/ping-json
git rebase -i HEAD~3
```

Git writes a file and opens `core.editor` (`nvim` in this book). Each line is a commit, oldest first:

```
pick a1b2c3d feat(ping): return a dict
pick d4e5f6a docs: mention JSON
pick 98ab765 fix typo
```

You edit the **verbs**, save, quit. Git replays from the top.

Verbs you will use

Verb	Effect
`pick`	Keep this commit
`reword`	Keep, edit the message
`edit`	Stop here so you can amend
`squash`	Fold into the previous commit; editor for a combined message
`fixup`	Fold into the previous commit; **keep the previous message**
`drop`	Omit this commit

Reorder lines to reorder commits. Delete a line to drop (same as `drop`).

Linux without the editor (labs)

For a scripted squash of everything after the first of the last three:

```
GIT_SEQUENCE_EDITOR="sed -i '2,\$s/^pick /fixup /'" git rebase -i HEAD~3
```

`sed -i` on GNU sed (Linux) edits in place. This is for labs and hooks. Day to day, use `nvim`.

Stop and amend (`edit`)

Git stops at that commit. You `git add` / `git commit --amend`, then:

```
git rebase --continue
```

Abort

```
git rebase --abort
```

Back to the pre-rebase branch tip. Safe while the rebase is in progress.

Range

`git rebase -i main` rebases each commit reachable from HEAD but not from `main`. That is the usual "update my feature onto main" todo list.

Do this on a feature branch. Interactive rebase of shared `main` rewrites hashes everyone already has.

Rebase Conflicts

Rebase Conflicts: How They Differ from Merge Conflicts

After this page you can finish a conflicted rebase one commit at a time.

One commit at a time

A merge conflict happens **once**, at the join. A rebase conflict happens **each time** a copied commit does not apply onto the new parent.

```
merge: one fight at M

A ← B ← C ← M
     ↑      ↑
     D ← E

rebase: a fight at D', then maybe again at E'

A ← B ← C ← G ← D'  ←  E'
                 stop     stop
```

Markers look the same (`<<<<<<<`). `git status` during a rebase says `rebase in progress` and names the commit being applied.

```
git rebase main
# CONFLICT (content): Merge conflict in src/ping.py
git status
```

The loop

1. Edit files, remove markers.
2. `git add` the resolved paths.
3. `git rebase --continue` (not `git commit`, unless Git asks you to).
4. Repeat until Git prints `Successfully rebased`.

```
nvim src/ping.py
git add src/ping.py
```

```
git rebase --continue
```

If the commit becomes empty (your patch is already in the new base):

```
git rebase --skip
```

Give up:

```
git rebase --abort
```

Ours / theirs are swapped from merge

During **merge**, `--ours` is the branch you had checked out (`main`). During **rebase**, you checked out the feature and Git replays onto `main`: `--ours` is the **upstream** (the new base), `--theirs` is the commit being replayed.

That swap surprises everyone once. Prefer reading the markers over `--ours` until it is boring.

`rerere`

With `rerere.enabled` (chapter 2), Git records the resolution. A later rebase that hits the same hunk applies it again. On a long-lived feature branch that rebases onto `main` daily, this is the difference between pain and a two-second continue.

After success

```
git log --oneline --graph --decorate main feat/ping-json
```

A straight line from `main` through your copied commits. Then `git switch main && git merge --ff-only feat/ping-json`.

Pull with Rebase

Pull with Rebase When Your Branch Is Out of Date

After this page you can update a local branch from a remote without a merge commit from `git pull`.

`pull` is fetch plus integrate

```
git pull
```

means `git fetch` then, by default, a merge of `@{u}` (the upstream) into `HEAD`. If both you and the remote moved, you get a merge commit that only exists because you pulled.

```
after fetch, both moved

origin/main   A ← B ← C ← G
local main    A ← B ← C ← D ← E

git pull              (merge)     A ← B ← C ← G ← M
                                        D ← E

git pull --rebase                 A ← B ← C ← G ← D' ← E'
```

Chapter 2 set:

```
git config --global pull.ff only
```

Then `git pull` **fails** instead of merging. You choose:

```
git pull --rebase
# or
git fetch
git rebase @{u}
```

`--rebase` copies your local-only commits onto the updated upstream, then fast-forwards your branch name.

When this is the right shape

You committed two fixes on `main` locally. Someone else pushed to `origin/main`. `git pull --rebase` replays your two fixes on top of theirs. History stays linear. Conflicts are per commit (previous page).

When to merge instead

You want a merge commit that records "I integrated upstream at this point." Feature branches that you share, or `main` policies that require merge commits, use `git pull --no-rebase` or an explicit `git merge`.

Set it for this branch only

```
git config pull.rebase true          # this repo
git config --global pull.rebase true # every repo (overrides pull.ff only)
```

This book leaves `pull.ff only` as the global default so a surprise pull fails closed. Type `--rebase` when you mean it, until the habit is there.

After a rebase pull

```
git status
git log --oneline --graph --decorate -12
```

`Your branch is ahead of 'origin/main' by N commits` means you still need `git push`. If you had already pushed the old hashes, you need a force-with-lease (chapter 11).

Rebase vs Merge

The Debate: Rebase vs Merge

After this page you can pick a default for `desk-api` and know what you are giving up.

Same trees, different graphs

Both operations can produce a working tree that contains both sides of the work. Merge records a join. Rebase copies a chain.

```
start                          after merge

A ← B ← C ← G   main           A ← B ← C ← G ← M   main
     ↑                              ↑          ↑
     D ← E      feat                D ← E      feat

start                          after rebase feat onto main

A ← B ← C ← G   main           A ← B ← C ← G          main
     ↑
     D ← E      feat                           D' ← E'  feat
```

	Merge	Rebase
Hashes of your old commits	Unchanged	New
Extra commit	Often a merge commit	None if you fast-forward after
Conflicts	Once, at the join	Once per copied commit
Safe on published `main`	Yes	Usually no
Safe on your unpushed feature	Yes	Yes, and often cleaner

A default that scales

- **Feature branch you own:** rebase onto `main`, then fast-forward `main` (or open a pull request that the forge squash-merges).
- **`main` / shared long-lived branch:** merge or fast-forward only. Do not rebase commits other people have.
- **Hotfix already on `main`:** revert (chapter 10), do not rebase `main`.

Forges add a third option: **squash merge** (many commits become one on `main`). That is a server-side rewrite. Chapter 15 and chapter 31. Locally, rebase is how you squash *before* you push.

`rerere`, tests, Linux

Run the test command you actually have after every `--continue`. On Linux that is whatever `desk-api` uses (`python -m pytest`, a shell script, `make test`). Rebase does not run tests for you.

You can mix

Rebase the feature. Merge into `main` with `--no-ff` if the team wants a join node for `git log --first-parent`. That is Git Flow's "feature into develop" shape. Trunk-based skips the join and fast-forwards.

There is no global winner. There is a default per branch class. Write it down in the repo (chapter 15). The hands-on page next squashes a messy feature on Linux.

Hands-On: Clean Up a Messy History

Hands-On: Clean Up a Messy History

Make three noisy commits on a feature branch, then squash them into one conventional commit with interactive rebase.

```
before                          after fixup + amend

A ← wip ← more ← typo           A ← docs: mention a future health endpoint
    feat/health-header              feat/health-header
```

Setup

```
cd ~/desk/desk-api
git switch main
git switch -c feat/health-header
```

Three messy commits

```
echo "# health" >> README.md
git add README.md
git commit -m "wip"

echo "Health endpoint later." >> README.md
git add README.md
git commit -m "more"

echo "See src/ping.py for now." >> README.md
git add README.md
git commit -m "typo fix later"
```

```
git log --oneline -5
```

Squash with GNU sed (Linux)

Keep the first of the three; fixup the rest:

```
GIT_SEQUENCE_EDITOR="sed -i '2,\$s/^pick /fixup /'" git rebase -i HEAD~3
git log --oneline -5
```

One commit remains, message still `wip`. Retitle:

```
git commit --amend -m "docs: mention a future health endpoint"
git log --oneline -3
```

Optional: rebase onto main

If `main` moved, replay this one commit:

```
git rebase main
git log --oneline --graph --decorate -8
```

Cleanup

```
git switch main
git merge --ff-only feat/health-header
git branch -d feat/health-header
```

If `--ff-only` refuses, `main` moved; rebase the feature first or merge.

Done when

- `git log` on `main` shows one `docs:` commit for this work, not three `wip`s
- `git status` is clean
- The feature branch is deleted

Next chapter: copy a single commit onto another branch (cherry-pick) and undo a published commit (revert).

rebase –onto

rebase –onto

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Cut a commit chain off one base and replay on another.

> **Note**
>
> Core Git depth (Linux host). See `content/_planning/SYLLABUS.md`.

autosquash and fixup

autosquash and fixup

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- `git commit --fixup`, `git rebase -i --autosquash`.

> **Note**
>
> Core Git depth (Linux host). See `content/_planning/SYLLABUS.md`.

rebase –exec

rebase –exec

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Run tests after every replayed commit.

> **Note**
>
> Core Git depth (Linux host). See `content/_planning/SYLLABUS.md`.

update-refs and keep-base

update-refs and keep-base

Thesis

A plain rebase moves **one** branch name: the one you are on. Real work often comes as a **stack**: `feat/api` ← `feat/ui` ← `feat/docs`, each reviewed as its own pull request. If you rebase the top of the stack, every branch name under it is left pointing at the old commits. Two flags fix this:

- **`--update-refs`** force-moves every local branch that points *into* the rebased range, so the whole stack moves together.
- **`--keep-base`** rewrites commits **in place**, on the same merge base. You can tidy a stack (fixups, rewording) without also pulling in whatever landed on `main` overnight.

Use them together and you can edit any commit in the stack, from the top branch, in one command.

`--update-refs` arrived in Git 2.38 and `--keep-base` in Git 2.24. The transcripts on this page come from Git 2.47. The behaviour is the same in current releases (2.56 as of October 2026).

Mental model

```
before: a stack of three branches (api, ui, docs = feat/*); main has moved on to B   M

  B   A1   A2   U1   U2   D1

  base       api         ui    docs

plain `git rebase main`: only HEAD's branch moves

  B   M   A1'   A2'   U1'   U2'   D1'   ← feat/docs
     A1   A2   U1   U2                    ← feat/api, feat/ui left behind

`git rebase --update-refs main`: every branch inside the range moves

  B   M   A1'   A2'   U1'   U2'   D1'

                  api           ui     docs

`git rebase -i --keep-base --update-refs main`: rewrite in place on B
```

```
   B   A1'   A2'   U1'   U2'   D1'        (M stays off to the side)
```

Piece	What it does
`--update-refs`	Adds an `update-ref refs/heads/<name>` line to the todo list after the last commit of each branch inside the range. The refs move only when the rebase **finishes**, so `--abort` leaves them alone
`rebase.updateRefs=true`	Makes `--update-refs` the default. Turn it off for one run with `--no-update-refs`
`--keep-base <upstream>`	New base = `merge-base(<upstream>, HEAD)`. Same as `--onto <upstream>...HEAD <upstream>`, and also implies `--reapply-cherry-picks` and `--no-fork-point`
Branches checked out in another worktree	**Never** updated. The todo shows a `# Ref … checked out at …` comment instead
Refs outside the rebased range	Untouched. Rebase from the **top** of the stack, or descendants get left behind

`--keep-base` cannot be combined with `--onto` or `--root`. It is a way of *choosing* the onto point, so those flags would conflict with it.

Worked examples (Cases)

Every case starts from the same throwaway repository. Source the environment file so your commit hashes match the transcripts.

```
# stack-env.sh - source this so your hashes match the transcripts on this page
export GIT_AUTHOR_NAME="Dev" GIT_AUTHOR_EMAIL="dev@example.com"
export GIT_COMMITTER_NAME="Dev" GIT_COMMITTER_EMAIL="dev@example.com"
export GIT_AUTHOR_DATE="2026-10-02T10:00:00+05:30"
export GIT_COMMITTER_DATE="2026-10-02T10:00:00+05:30"
```

```
#!/usr/bin/env bash
# setup-stack.sh - build a three-branch stack on top of main, then move main
set -euo pipefail
export GIT_AUTHOR_NAME="Dev" GIT_AUTHOR_EMAIL="dev@example.com"
export GIT_COMMITTER_NAME="Dev" GIT_COMMITTER_EMAIL="dev@example.com"
export GIT_AUTHOR_DATE="2026-10-01T09:00:00+05:30"
export GIT_COMMITTER_DATE="$GIT_AUTHOR_DATE"

git init -q -b main "${1:-stack}" && cd "${1:-stack}"
c() { echo "$2" >> "$1"; git add "$1"; git commit -q -m "$3"; }

c README.md "# shop"                    "init: README"
```

```
c lib.sh    "price() { echo 100; }"     "main: add lib.sh"

git switch -q -c feat/api                # bottom of the stack
echo "timeout=3" > api.conf && git add api.conf
c api.sh    "get_price() { price; }"    "api: add get_price"
c api.sh    "list_items() { :; }"       "api: add list_items"

git switch -q -c feat/ui                 # middle: builds on feat/api
c ui.sh     "render() { get_price; }"   "ui: render price"
c ui.sh     "render_list() { list_items; }" "ui: render list"

git switch -q -c feat/docs               # top: builds on feat/ui
c docs.md   "Prices come from get_price." "docs: describe pricing"

git switch -q main                       # meanwhile, main moves on
c lib.sh    "tax() { echo 18; }"        "main: add tax()"
git switch -q feat/docs
```

```
mkdir -p /tmp/vcs && cd /tmp/vcs        # save both files here first
chmod +x setup-stack.sh
source ./stack-env.sh
./setup-stack.sh && cd stack
git log --graph --oneline --decorate --all
```

```
* 87455ba (HEAD -> feat/docs) docs: describe pricing
* 576a711 (feat/ui) ui: render list
* 42dfa4c ui: render price
* 87584b9 (feat/api) api: add list_items
* f2f0cff api: add get_price
| * c6a0e25 (main) main: add tax()
|/
* 6b188b4 main: add lib.sh
* 2a401c6 init: README
```

To reset between cases, run `cd /tmp/vcs && rm -rf stack && ./setup-stack.sh && cd stack`.

Case 1: The stale stack (what goes wrong without the flag)

```
git rebase main
git log --graph --oneline --decorate --all
git log --oneline main..feat/ui
```

```
Successfully rebased and updated refs/heads/feat/docs.
* 1861481 (HEAD -> feat/docs) docs: describe pricing
* f0527d0 ui: render list
* 55affa4 ui: render price
* 49a9b11 api: add list_items
* 58ab280 api: add get_price
* c6a0e25 (main) main: add tax()
| * 576a711 (feat/ui) ui: render list
| * 42dfa4c ui: render price
| * 87584b9 (feat/api) api: add list_items
| * f2f0cff api: add get_price
|/
* 6b188b4 main: add lib.sh
* 2a401c6 init: README
576a711 ui: render list
42dfa4c ui: render price
87584b9 api: add list_items
f2f0cff api: add get_price
```

`feat/docs` moved and the other two did not. The repository now holds two copies of the api and ui commits. If you push `feat/ui` now, its pull request still targets the old base. If you push `feat/docs`, its pull request shows four extra commits that "belong" to other PRs. The old way to repair this was one `git branch -f` per branch, with hashes copied by hand.

Case 2: `--update-refs` moves the whole stack

```
git rebase --update-refs main
git log --graph --oneline --decorate --all
```

```
Successfully rebased and updated refs/heads/feat/docs.
Updated the following refs with --update-refs:
    refs/heads/feat/api
    refs/heads/feat/ui
* 1861481 (HEAD -> feat/docs) docs: describe pricing
* f0527d0 (feat/ui) ui: render list
* 55affa4 ui: render price
* 49a9b11 (feat/api) api: add list_items
* 58ab280 api: add get_price
* c6a0e25 (main) main: add tax()
* 6b188b4 main: add lib.sh
* 2a401c6 init: README
```

Prove that the rewrite only replayed the commits and changed none of them. `range-diff` pairs each old commit with its new version, and `=` means "identical patch":

```
git range-diff main feat/docs@{1} feat/docs
git reflog -2 feat/ui
```

```
1:  f2f0cff = 1:  58ab280 api: add get_price
2:  87584b9 = 2:  49a9b11 api: add list_items
3:  42dfa4c = 3:  55affa4 ui: render price
4:  576a711 = 4:  f0527d0 ui: render list
5:  87455ba = 5:  1861481 docs: describe pricing
f0527d0 feat/ui@{0}: rewritten during rebase
576a711 feat/ui@{1}: commit: ui: render list
```

Each moved branch gets a `rewritten during rebase` reflog entry, so `git branch -f feat/ui feat/ui@{1}` undoes one branch if you need to.

Case 3: Fix the bottom commit from the top, without moving the base

Review on `feat/api` asks for `timeout=30`. You are on `feat/docs`, and `main` has moved, but you do **not** want to drag `main: add tax()` into three open pull requests just to fix one line.

```
sed -i 's/timeout=3/timeout=30/' api.conf
git commit -q -am "fixup! api: add get_price"
git log --oneline -1
GIT_SEQUENCE_EDITOR=cat git rebase -i --autosquash --keep-base --update-refs main
git log --graph --oneline --decorate --all
```

`GIT_SEQUENCE_EDITOR=cat` prints the todo list and accepts it unchanged. In real work you would let your editor open it. Comment lines are omitted below:

```
ddf1e1f fixup! api: add get_price
pick f2f0cff api: add get_price
fixup ddf1e1f fixup! api: add get_price
pick 87584b9 api: add list_items
update-ref refs/heads/feat/api
pick 42dfa4c ui: render price
pick 576a711 ui: render list
update-ref refs/heads/feat/ui
pick 87455ba docs: describe pricing
Successfully rebased and updated refs/heads/feat/docs.
Updated the following refs with --update-refs:
    refs/heads/feat/api
    refs/heads/feat/ui
* 6824a87 (HEAD -> feat/docs) docs: describe pricing
* 76f2b7a (feat/ui) ui: render list
* c08e7a1 ui: render price
* b46d23a (feat/api) api: add list_items
* 7e3a228 api: add get_price
```

```
| * c6a0e25 (main) main: add tax()
|/
* 6b188b4 main: add lib.sh
* 2a401c6 init: README
```

Three things to notice:

1. `--autosquash` moved the fixup next to its target. `--update-refs` put each `update-ref` line **after** the branch's last commit, so `feat/api` now includes the fix.
2. `--keep-base` kept the fork point at `6b188b4`. `main` is still off to the side, and each pull request's diff against its base changed only where you meant it to.
3. The `update-ref` lines are ordinary todo commands. You can move them, delete one (that branch then stays where it was), or add one (`update-ref refs/heads/feat/ui-split`) to cut a new branch boundary in the middle of the stack.

Verify what changed. `--creation-factor` raises the threshold for pairing commits, which helps when a commit is tiny and the fix is a large share of it:

```
git range-diff --creation-factor=95 "$(git merge-base main feat/docs)" feat/docs@{1} feat/do
```

```
1:  f2f0cff ! 1:  7e3a228 api: add get_price
    @@ Commit message

      ## api.conf (new) ##
     @@
    -+timeout=3
    ++timeout=30

      ## api.sh (new) ##
     @@
2:  87584b9 = 2:  b46d23a api: add list_items
3:  42dfa4c = 3:  c08e7a1 ui: render price
4:  576a711 = 4:  76f2b7a ui: render list
5:  87455ba = 5:  6824a87 docs: describe pricing
6:  ddf1e1f < -:  ------- fixup! api: add get_price
```

One `!` (changed) commit, four `=`, and the fixup gone: exactly the intended edit.

Case 4: Make it the default

```
git config rebase.updateRefs true          # this repo; use --global for all repos
git rebase main
git config --show-origin rebase.updateRefs
```

```
Successfully rebased and updated refs/heads/feat/docs.
Updated the following refs with --update-refs:
    refs/heads/feat/api
    refs/heads/feat/ui
file:.git/config    true
```

`git rebase --no-update-refs main` opts out for one run. That is useful when you deliberately want to fork a copy of a stack and leave the original branches alone.

Case 5: Publish the moved stack in one atomic push

A rewritten stack needs a force push. Push all the branches together, so a reviewer never sees `feat/ui` rebased while `feat/api` is still old:

```
git init -q --bare ../origin.git && git remote add origin ../origin.git
git push -q origin main feat/api feat/ui feat/docs      # the "already published" state
git rebase -q --update-refs main
git push --force-with-lease --atomic origin feat/api feat/ui feat/docs
```

```
To ../origin.git
 + 87584b9...49a9b11 feat/api -> feat/api (forced update)
 + 87455ba...1861481 feat/docs -> feat/docs (forced update)
 + 576a711...f0527d0 feat/ui -> feat/ui (forced update)
```

`--force-with-lease` refuses to overwrite a branch someone else pushed to since your last fetch. `--atomic` makes the server accept all three ref updates or none of them.

Case 6: The bottom branch was squash-merged

`feat/api` lands on `main` as one squash commit. Its original commits will never appear on `main`, so `git rebase main` would try to replay them and conflict, or produce empty commits. Cut them off explicitly with `--onto`:

```
git switch -q main && git merge -q --squash feat/api && git commit -q -m "api: get_price and
git switch -q feat/docs
git rebase --update-refs --onto main feat/api feat/docs
git log --graph --oneline --decorate --all
```

```
Successfully rebased and updated refs/heads/feat/docs.
Updated the following refs with --update-refs:
    refs/heads/feat/ui
* 49a9441 (HEAD -> feat/docs) docs: describe pricing
* 9faa213 (feat/ui) ui: render list
* cb443ba ui: render price
```

```
* a49d524 (main) api: get_price and list_items (#41)
* c6a0e25 main: add tax()
| * 87584b9 (feat/api) api: add list_items
| * f2f0cff api: add get_price
|/
* 6b188b4 main: add lib.sh
* 2a401c6 init: README
```

The range `feat/api..feat/docs` excludes the squashed commits. `feat/ui` moves because it points inside the range. `feat/api` does not, because it is the boundary. Delete it (`git branch -D feat/api`) and retarget the `feat/ui` pull request to `main`.

The trap

Rebasing from the middle of the stack, or from a worktree, silently strands branches.

`--update-refs` only touches refs that point **into the commits being rewritten**. Rebase `feat/ui` and its descendant `feat/docs` is not in that range:

```
git switch -q feat/ui
git rebase --update-refs main
git log --graph --oneline --decorate --all
```

```
Successfully rebased and updated refs/heads/feat/ui.
Updated the following refs with --update-refs:
    refs/heads/feat/api
* f0527d0 (HEAD -> feat/ui) ui: render list
* 55affa4 ui: render price
* 49a9b11 (feat/api) api: add list_items
* 58ab280 api: add get_price
* c6a0e25 (main) main: add tax()
| * 87455ba (feat/docs) docs: describe pricing
| * 576a711 ui: render list
| * 42dfa4c ui: render price
| * 87584b9 api: add list_items
| * f2f0cff api: add get_price
|/
* 6b188b4 main: add lib.sh
* 2a401c6 init: README
```

The second failure is quieter. A branch that is checked out in **another worktree** is skipped, and a non-interactive rebase gives no warning at all:

```
git worktree add -q ../wt-ui feat/ui
GIT_SEQUENCE_EDITOR=cat git rebase -i --update-refs main | grep -E 'update-ref|checked out'
```

```
update-ref refs/heads/feat/api
# Ref refs/heads/feat/ui checked out at '/tmp/vcs/wt-ui'
```

The run then reports `Updated the following refs with --update-refs: refs/heads/feat/api` only. `feat/ui` stays on the old commits, and the next `git push` from that worktree re-publishes the stale stack.

Recovery is cheap, because nothing is lost. Run `git switch feat/docs && git rebase --update-refs main` from the true top. For a worktree, remove it or detach its HEAD first (`git -C ../wt-ui switch --detach`), then rebase again.

The boring rule

> Turn on `rebase.updateRefs`. Rebase stacks **only from the top branch**, with no stack branch checked out in another worktree. Use `--keep-base` when you are tidying and plain `<upstream>` when you are catching up. Prove every rewrite with `git range-diff`, and publish with `git push --force-with-lease --atomic` naming every branch in the stack.

Try this

1. Build the stack, then run `git rebase -i --keep-base --update-refs main` and reword `ui: render price`. Use `git range-diff` to confirm that only that commit shows `!`, and only in its message.
2. In the todo list, add `update-ref refs/heads/feat/ui-price` right after `pick … ui: render price`. Check that the new branch exists after the rebase and points at the rewritten commit.
3. Start a rebase that conflicts (edit `ui.sh` on `main` first), then `git rebase --abort`. Confirm with `git log --oneline -1 feat/api` that no stack branch moved: the refs only update when the rebase completes.
4. Reproduce the worktree trap, then fix it with `git -C ../wt-ui switch --detach` and a fresh rebase from `feat/docs`.
5. Set `git config --global rebase.updateRefs true` and `rebase.autoSquash true`. For a week, write review fixes as `git commit --fixup=<sha>` and land them with one `git rebase -i --keep-base <base>`.

Sources

- git-rebase documentation: `--update-refs`, `--keep-base`, `update-ref` todo command, incompatible options (Git 2.56 docs)
- git-config: `rebase.updateRefs`
- git-range-diff documentation: `--creation-factor`, output markers `=`, `!`, `<`, `>`
- git-push documentation: `--force-with-lease`, `--atomic`

- Git 2.38 release notes (introduced `--update-refs`); Git 2.24 release notes (introduced `--keep-base`)

Cherry-Pick and Revert

Theory

Theory: Cherry-Pick and Revert are Patches with New Hashes

Both operations take a **diff** (commit vs its parent, or the inverse) and apply it to `HEAD`, writing a new commit. They do not move the original node.

Invariants

1. Cherry-pick(C) apply `diff(parent(C), C)` onto `HEAD`.
2. Revert(C) apply `diff(C, parent(C))` onto `HEAD` (the inverse patch).
3. The original commit remains reachable if any name still points at it.
4. Revert of a **merge** needs `-m` to choose which parent is the mainline; Git then considers those changes "already merged" for future merges.
5. `-x` records `cherry picked from` in the message; it does not link objects.

```
cherry-pick D onto main          revert C on main

feat  A ← B ← C ← D ← E          A ← B ← C ← R
main  A ← B ← C                  R undoes C; C stays
after A ← B ← C ← D'
```

Why not reset on production

Reset moves a name backward. Clones that already have the old tip cannot fast-forward. Revert lets them fast-forward to a commit that *cancels* the bad one. That is the distributed-model consequence, not etiquette.

What can break

- Cherry-picking a merge without `-m`.
- Reverting a merge then trying to merge the same feature again (Git thinks it is in).
- Secrets: revert does not purge blobs from history.

How to read this chapter

Cherry-pick, hotfixes, revert, emergencies; then ranges/`-x`, empty picks.

Cherry-Pick

Cherry-Pick: Moving Commits Surgically

After this page you can copy one commit onto another branch without merging the whole feature.

Copy the patch, new hash

```
cd ~/desk/desk-api
git switch main
git cherry-pick <hash>
```

Git takes the **diff** of that commit against its parent, applies it onto `HEAD`, and writes a new commit. The new object has a new hash. Author is preserved; you are the committer.

```
 feat    A ← B ← C ← D ← E          pick D only
 main    A ← B ← C

 after   A ← B ← C ← D'             main   (D'   patch of D, new hash)
 feat    A ← B ← C ← D ← E          unchanged
```

```
git log -1 --format=fuller
```

Several commits

```
git cherry-pick abc123 def456
git cherry-pick main~3..feat/ping-json
```

A range `A..B` is commits reachable from `B` excluding `A`. Oldest is applied first.

Conflicts

Same markers as merge. Then:

```
git add src/ping.py
git cherry-pick --continue
git cherry-pick --abort
git cherry-pick --skip
```

Same loop as rebase, one commit at a time.

Empty pick

If `main` already contains the change, Git says the pick is empty. `--skip` or `--allow-empty`.

What cherry-pick is for

- A **hotfix** that lives on a release branch and also needs to be on `main`
- A commit that landed on the wrong branch
- Backporting a fix to an older `release/*` line

What it is a poor fit for: dragging a whole feature. Merge or rebase the branch instead so you get the whole chain and a honest graph.

Next page: hotfixes and backports as a desk scenario.

Hotfixes and Backports

Deploying Hotfixes Without Merging Features

After this page you can land a one-commit fix on `main` and on a release branch without dragging unreleased work.

The situation

`feat/big-refactor` is half done. Production (`main` or `release/1.x`) needs a one-line fix that you already committed on the feature branch.

Merging the feature would ship unfinished work. Cherry-pick the fix commit only.

```
feat/big-refactor   A ← B ← fix ← D ← E     (D,E not ready)
main                A ← B

cherry-pick fix     A ← B ← fix'            main
                    A ← B ← fix ← D ← E     feat  (untouched)
```

```
cd ~/desk/desk-api
git log feat/big-refactor --oneline -5
# copy the hash of the fix

git switch main
git cherry-pick <hash>
```

Release branch

```
git switch -c release/1.2 main   # or an existing release line
git cherry-pick <hash>
```

Now both lines have the fix, as two commits with different hashes and the same author.

Backport the other direction

The fix landed on `main` first. An older release still needs it:

```
git switch release/1.1
git cherry-pick <hash-on-main>
```

If the older tree cannot apply the patch (the surrounding code moved), resolve like a rebase conflict or rewrite the fix by hand and commit normally. Cherry-pick is a tool, not a guarantee.

Label it in the message

```
git cherry-pick -x <hash>
```

`-x` appends `(cherry picked from commit …)` to the message. Useful on release branches so `git log` shows the origin hash.

After the pick

Run whatever `desk-api` uses to check itself on Linux (`python -c 'from src.ping import ping; print(ping())'` is enough for the toy module). Then merge or push according to chapter 11. Do not merge `feat/big-refactor` until that feature is done.

Revert and Unmerge

`git revert` and Unmerging Changes

After this page you can undo a **published** commit by adding a new commit, including a merge.

Revert writes history forward

```
cd ~/desk/desk-api
git switch main
git revert HEAD --no-edit
git log --oneline -3
```

The bad commit is still in the graph. A new commit applies the inverse patch. Shared clones can fast-forward. This is the safe undo on `main`.

```
reset --hard HEAD~1     (local only)     A ← B            main
                                         C still in reflog

revert HEAD             (shared)         A ← B ← C ← R    main
                                         R undoes C; C remains
```

`--no-edit` keeps Git's default message (`Revert "…"`). Edit with `git revert HEAD` if you need a `fix:` subject.

Several commits

```
git revert --no-edit HEAD~2..HEAD
```

Reverts newest first (the range is exclusive on the left). Conflicts: `git add` and `git revert --continue`, or `git revert --abort`.

Revert a merge commit

A merge has two parents. Git needs to know which parent is "the mainline" to diff against:

```
git revert -m 1 <merge-hash>
```

`-m 1` means "keep first parent's tree; undo what the merged branch brought in." `-m 2` is the other parent (almost never what you want on `main`).

After reverting a merge, Git considers that feature's commits **already merged**. Cherry-picking them back later can no-op. To re-land the feature, revert the revert, or rebase the feature into new hashes.

Revert vs reset vs restore

Goal	Command	History
Throw away uncommitted edits	`git restore`	Unchanged
Replace last unpublished commit	`git commit --amend` / `reset`	Rewritten
Undo a published commit	`git revert`	New commit

`git reset --hard` on shared `main` is a force-push incident. Revert instead.

Un-merge without `-m`

If the merge was never pushed and is `HEAD`:

```
git reset --hard HEAD~1
```

That moves `main` to first parent and drops the merge commit from the branch name. Only on a lab clone or an unpublished branch. Chapter 7 is reset in full. The merge commit stays in the reflog (chapter 17) until it expires.

Production Emergency Scenarios

Real Production Emergency Scenarios

After this page you have a short playbook for "production is wrong and Git is how we talk about it."

Assume Linux, `~/desk/desk-api`, and a `main` that is already on a remote. You are in a terminal, not a GUI.

```
published on origin/main?

     no       reset / amend / rebase     (rewrite)

     yes      revert or cherry-pick fix  (forward)

                     secret in history    rotate now; filter-repo later
```

1. Bad commit is `HEAD` on `main`, already pulled by others

```
git switch main
git pull --ff-only
git revert HEAD --no-edit
# run tests
git push
```

2. Bad commit is in the middle of `main`

```
git log --oneline -20
git revert --no-edit <hash>
# tests
git push
```

If several commits are one logical change, revert the range newest-first, or revert the merge with `-m 1`.

3. Fix exists on a feature branch, production cannot wait

```
git log feat/whatever --oneline
git switch main
git cherry-pick -x <fix-hash>
# tests
git push
```

Leave the feature branch as-is. The pick is a new hash on `main`.

4. You committed a secret

`git revert` **does not** remove the secret from history; it only removes it from `HEAD`. Anyone who fetched still has the blob. Rotate the secret at the provider. History cleanup is chapter 20 (`git filter-repo`). For the next five minutes: revert or a follow-up commit that deletes the file, rotate, tell the team.

5. Merge to `main` was wrong, merge commit is `HEAD`, not pushed

```
git reset --hard HEAD~1
```

If it **was** pushed, revert with `-m 1` instead.

6. You are not sure what `main` is

```
git status
git log --oneline --graph --decorate -15
git reflog -10
```

Status, graph, reflog. Then pick revert (shared) or reset (local only).

Write the team a one-line message: hash reverted, why, who is rotating secrets if any. Git is the record; the pager is the people.

Ranges and -x

Ranges and -x

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- `cherry-pick A..B`, `-x` recorded origin.

> **Note**
>
> Core Git depth (Linux host). See `content/_planning/SYLLABUS.md`.

Empty Picks and –skip

Empty Picks and –skip

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- When the patch is already in the tree.

> **Note**
>
> Core Git depth (Linux host). See `content/_planning/SYLLABUS.md`.

Remotes

Theory

Theory: A Remote is Another Repository Plus a Nickname

`origin` is not special to Git. It is a default name for a URL. Fetch copies objects and updates **remote-tracking refs**. Push asks the other repository to move a name it owns.

Invariants

1. `git fetch` does not move your `main`. It updates `refs/remotes/origin/main`.
2. `git pull` = fetch + merge or rebase of `@{u}` into `HEAD`.
3. Non-fast-forward push is rejected because it would drop commits the remote still names.
4. `--force-with-lease` succeeds only if the remote ref still matches what you last fetched.
5. Several remotes means several photographs of the same DAG, not several Git identities.

```
laptop                          origin (bare or forge)
refs/heads/main                 refs/heads/main
refs/remotes/origin/main      (fetch updates this)
         push               (moves theirs if fast-forward)
```

Why PRs are not Git objects

A pull request is a forge row: "please integrate this ref into that ref," plus comments and checks. The Git underneath is still fetch + merge/rebase/squash on the server.

What can break

- `git pull` on a dirty tree.
- Force-push of `main`.
- Two remotes with the same objects but different `main` (divergent policy).

How to read this chapter

Fetch/pull/push, extra remotes, force-with-lease, PRs, two-clone lab; then upstream, refspecs, bundle, prune, fork vs clone.

Fetch, Pull, and Push

Remote Mechanics: Fetch, Pull, Push

After this page you can talk to another Git repository on Linux without confusing fetch with pull.

A remote is a named URL

```
cd ~/desk/desk-api
git remote add origin git@host:ada/desk-api.git
git remote -v
```

`origin` is a convention. The URL is SSH (chapter 2) or `https://`. For labs on one machine, a **bare** repo on disk is a valid remote:

```
 ~/desk/desk-api                          origin
 working tree + .git                      (bare) objects + refs
        git fetch    origin/main       refs/heads/main
        git push     main          refs/heads/main

 refs/heads/main          refs/remotes/origin/main
```

```
git init --bare ~/desk/desk-api.git
git remote add origin ~/desk/desk-api.git
```

`file://` and a path both work on Linux. Bare means no working tree — the shape a forge stores.

Fetch

```
git fetch origin
git fetch --all
git log --oneline --graph --decorate --all
```

Fetch downloads objects and updates **remote-tracking** names (`origin/main`). It does not move your `main`. Safe to run any time.

Push

```
git push -u origin main
```

`-u` sets **upstream** so later `git push` and `git pull` know the default. After that:

```
git push
git status
# Your branch is up to date with 'origin/main'
```

Push refuses if the remote has commits you do not. Fetch, integrate, push again.

Pull

```
git pull
```

Fetch plus integrate `@{u}` into `HEAD`. With `pull.ff only` (chapter 2), it fast-forwards or fails. `git pull --rebase` copies your local commits onto the updated upstream (chapter 9).

Prefer `git fetch` then look at `git log --oneline main..origin/main` before you integrate.

Tracking

```
git branch -vv
git rev-parse --abbrev-ref --symbolic-full-name '@{u}'
```

`@{u}` is the upstream of the current branch. `git push -u` is how it gets set.

Delete a remote branch

```
git push origin --delete feat/ping-json
```

Local delete is still `git branch -d`. The two are independent.

Multiple Remotes

Understanding Multiple Remotes

After this page you can push the same `desk-api` history to two places.

Extra remotes are normal

```
cd ~/desk/desk-api
git remote add origin git@forgejo.home:ada/desk-api.git
git remote add github git@github.com:ada/desk-api.git
git remote -v
```

Fetch both:

```
git fetch --all
git branch -r
```

`origin/main` and `github/main` are separate photographs.

```
                    git push origin
                              Forgejo
 laptop desk-api
                              GitHub
                    git push github

 refs/remotes/origin/main
 refs/remotes/github/main
```

Push to both

```
git push origin main
git push github main
```

A convenience:

```
git remote add all git@forgejo.home:ada/desk-api.git
git remote set-url --add --push all git@forgejo.home:ada/desk-api.git
git remote set-url --add --push all git@github.com:ada/desk-api.git
git push all main
```

One `git push all` updates both push URLs. Fetch still uses the first URL of that remote unless you add more fetch URLs. Many people just type two pushes.

Independence preview

Chapter 22 is lock-in. The Git-level move is this page: the objects live in every clone. Two remotes means a forge outage does not strand `desk-api`. Issues and CI still live on each forge; Git history does not.

Rename and remove

```
git remote rename origin forgejo
git remote remove github
```

Removing a remote deletes remote-tracking refs for it. Local branches stay.

`origin` on a USB disk

```
git init --bare /media/ada/stick/desk-api.git
git remote add stick /media/ada/stick/desk-api.git
git push -u stick main
```

Linux mounts the stick; Git sees a path. Same protocol family as `~/desk/desk-api.git`.

Force Push

Force Push Dangers

After this page you can update a remote after a rebase without clobbering someone else's commits.

Why push refuses

You rebased `feat/ping-json`. Local hashes are new. The remote still has the old chain. A plain `git push` is a non-fast-forward and Git rejects it.

```
origin/feat    A ← B ← C ← D ← E
local feat     A ← B ← C ← G ← D' ← E'     (rebased onto new main)

plain push     rejected (not a fast-forward)
--force-with-lease
  if origin still at E  →  origin becomes E'
  if teammate added F   →  lease fails, fetch first
```

`--force-with-lease`

```
git push --force-with-lease origin feat/ping-json
```

Git checks that the remote ref still points at what you last fetched. If a teammate pushed in the meantime, the lease fails and you fetch again. That is the default force this book uses.

`--force`

```
git push --force origin feat/ping-json
```

Overwrites the remote ref even if it moved. Use on a branch only you own, and only when you meant to throw away the remote's extra commits. Never a habit on `main`.

`main`

Do not force-push `main` on a shared remote. Revert instead (chapter 10). Forges often deny force on the default branch (chapter 31).

Recover if you force-pushed by mistake

The old commits may still be on your laptop (`git reflog`) or another clone. Push them back:

```
git reflog
git push --force-with-lease origin <old-hash>:main   # only if you own this recovery
```

That is an emergency. Coordinate on chat first.

Lease plus fetch

Always:

```
git fetch origin
git log --oneline --graph --decorate HEAD origin/feat/ping-json
git push --force-with-lease
```

Look at the graph. Then push.

Pull Requests and Code Review

Pull Requests, Code Review, and Handling Conflicts in Teams

After this page you can describe a pull request as Git operations plus a forge UI.

Git has no pull request object

A **pull request** (GitHub, Gitea, Forgejo, Bitbucket) or **merge request** (GitLab) is:

1. A branch you pushed (`feat/ping-json`)
2. A request that another branch (`main`) integrate it
3. Discussion, checks, and a button that runs `merge`, `squash merge`, or `rebase merge` on the server

Gerrit uses Change-Ids (chapter 27). SourceHut uses mailing lists (chapter 28). The Git underneath is still fetch + merge or rebase.

```
origin/main     A ← B ← C
origin/feat     A ← B ← C ← D ← E     ← you pushed this

forge PR        "merge feat into main"
                = merge, squash, or rebase on the server
```

Local shape

```
git switch -c feat/ping-json
# commits
git push -u origin feat/ping-json
```

On the forge: open a PR from `feat/ping-json` into `main`. Reviewers fetch that branch:

```
git fetch origin feat/ping-json
git log --oneline main..origin/feat/ping-json
git switch -c review-ping origin/feat/ping-json
```

Conflicts in a PR

The server's merge is the same three-way merge as chapter 8. Resolve **on Linux** in your clone:

```
git fetch origin
git switch feat/ping-json
git rebase origin/main          # or: git merge origin/main
# fix, add, rebase --continue
git push --force-with-lease
```

The PR updates because the branch name moved.

Review comments

Comments live on the forge. They are not Git objects. Closing a PR without merging leaves the branch on the remote until you delete it. Merging may delete the branch if the forge is set to.

Hands-on next

The next page uses two directories on one Linux machine as "you" and "origin," so you can push and fetch without an account.

Hands-On: Two Clones

Hands-On: Fork, Clone, Branch, Commit, Push

Simulate a remote with a bare repo on Linux. No forge account required.

Origin (bare)

```
~/desk/desk-api          ~/desk/desk-api.git        ~/desk/desk-api-b
(clone A, you)           (bare origin)              (clone B, teammate)
      push/fetch                                     fetch/merge
```

```
mkdir -p ~/desk
git init --bare ~/desk/desk-api.git
```

Your clone

If `~/desk/desk-api` already has history:

```
cd ~/desk/desk-api
git remote remove origin 2>/dev/null || true
git remote add origin ~/desk/desk-api.git
git push -u origin main
```

If you are starting empty:

```
git clone ~/desk/desk-api.git ~/desk/desk-api
cd ~/desk/desk-api
# if clone was empty, make the chapter 3 tiny project first, then:
git push -u origin main
```

A second clone (teammate or your own review checkout)

```
git clone ~/desk/desk-api.git ~/desk/desk-api-b
cd ~/desk/desk-api-b
git log --oneline -3
```

Feature on A, fetch on B

```
cd ~/desk/desk-api
git switch -c feat/readme-note
echo "Lab remote: ~/desk/desk-api.git" >> README.md
git add README.md
git commit -m "docs: note the local origin path"
git push -u origin feat/readme-note
```

```
cd ~/desk/desk-api-b
git fetch origin
git log --oneline main..origin/feat/readme-note
git merge origin/feat/readme-note
```

On a forge, that merge on B is the reviewer's local check; the **PR merge** happens on the server into `main`. Here, integrate on A:

```
cd ~/desk/desk-api
git switch main
git merge --ff-only feat/readme-note
git push origin main
git branch -d feat/readme-note
git push origin --delete feat/readme-note
```

```
cd ~/desk/desk-api-b
git fetch origin
git switch main
git merge --ff-only origin/main
```

Done when

- `~/desk/desk-api.git` has `main` with the docs commit
- Both clones' `main` match `git rev-parse HEAD`
- Feature branch is gone locally and on origin

You just did clone, branch, commit, push, fetch, and integrate. A pull-request button is that last integrate, with a UI. Next: stash, tags, and cleanup on a single clone.

Tracking and Upstream

Tracking and Upstream

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- `-u`, `@{u}`, `branch.autoSetupMerge`.

> **Note**
>
> Core Git depth (Linux host). See `content/_planning/SYLLABUS.md`.

Refspecs

Refspecs

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Fetch and push refspecs; `+` force; tags.

> **Note**
>
> Core Git depth (Linux host). See `content/_planning/SYLLABUS.md`.

git bundle

git bundle

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Sneakernet / air-gap: `git bundle create` and `git clone file.bundle`.

> **Note**
>
> Core Git depth (Linux host). See `content/_planning/SYLLABUS.md`.

Prune and Rename

Prune and Rename

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- `fetch.prune`, `git remote rename`, stale remote-tracking branches.

> **Note**
>
> Core Git depth (Linux host). See `content/_planning/SYLLABUS.md`.

Forks vs Clones

Forks vs Clones

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Forge fork vs `git clone`; `origin` and `upstream` naming.

> **Note**
>
> Core Git depth (Linux host). See `content/_planning/SYLLABUS.md`.

Stash, Tags, and Cleanup

Theory

Theory: Stash is Commits; a Tag is a Name Meant to Stay Put

`git stash` writes commit objects (index state + working-tree state) and points `refs/stash` at them. A **lightweight tag** is a ref. An **annotated tag** is an object (message, tagger, date) that then points at a commit.

Invariants

1. Stash does not change `HEAD`'s commit; it makes the working tree match `HEAD` (plus options for untracked).
2. `stash@{n}` is reflog syntax on `refs/stash`.
3. Tags are not pushed by default (`git push origin v1.0.0`).
4. `git clean` deletes untracked files Git never hashed; there is no Git undo.
5. `git describe` finds the nearest reachable annotated tag and counts commits after it.

```
stash push                      annotated tag

HEAD C  (unchanged)             refs/tags/v0.1.0 → tag object → commit C
WD    clean
refs/stash → W  (WIP commit)
              parent C
```

Why stash vs a branch

A branch is a name you can push and PR. A stash is local, easy to drop, easy to lose in `stash clear`. If the WIP will survive the afternoon, `switch -c wip/…` and commit.

What can break

- `stash pop` with conflicts (entry may still drop).
- Moving a published tag (`-f`) while others have already fetched it.
- `git clean -f -x` deleting ignored `.env`.

How to read this chapter

Stash, tags, cleanup, lab; then `stash branch`, `describe`, `archive`.

Stash

Stash Mechanics, Including Patch Stashes

After this page you can park uncommitted work, switch branches, and bring the work back.

A stash is a commit Git keeps on the side

```
cd ~/desk/desk-api
echo "wip" >> README.md
git stash push -m "wip readme"
git status
git stash list
```

Working tree is clean. `stash@{0}` holds the diff. Switch branches, then:

```
 before stash                 after stash push

 working  dirty README        working  clean
 index    clean               index    clean
 HEAD     C                   HEAD     C
                              stash@{0}     a pair of commits
                                           (index + working tree)
```

```
git stash pop
# or
git stash apply
```

`pop` applies and drops. `apply` applies and keeps the stash entry.

Include untracked

```
git stash push -u -m "wip plus new files"
```

Without `-u`, untracked files stay in the way and can block a switch.

Patch stash (`-p`)

```
git stash push -p -m "only the ping hunk"
```

Git offers hunks, same as `git add -p`. Stash the ones that are in the way; leave the rest in the working tree.

Named and multiple

```
git stash push -m "experiment A"
git stash push -m "experiment B"
git stash list
git stash apply stash@{1}
git stash drop stash@{1}
git stash show -p stash@{0}
```

Numbers shift when you drop. The message is how you find them.

Conflicts on apply

Markers appear. Add and continue as a normal working-tree conflict. The stash entry remains if you used `apply`; `pop` may still drop — prefer `apply` when you are unsure.

Stash vs commit

A commit on a throwaway branch is easier to push and to recover. Stash is for "I must switch in the next ten seconds." If the WIP will last, `git switch -c wip/…` and commit.

Tags

Tags: Lightweight vs Annotated

After this page you can mark a commit as a release and push that name.

Two kinds

```
cd ~/desk/desk-api
git tag v0.1.0                    # lightweight: a ref file
git tag -a v0.1.0 -m "desk-api 0.1.0"   # annotated: a tag object
```

Lightweight is a name pointing at a commit (like a branch that you do not move).

Annotated is a real object: message, tagger, date, optional signature (chapter 20). Releases should be annotated.

```
 lightweight                          annotated

 refs/tags/v0.1.0    commit C       refs/tags/v0.1.0    tag object


                                                            commit C
                                        tag object has message, tagger, date
```

If `v0.1.0` already exists, Git refuses. `git tag -f` overwrites locally; moving a published tag surprises clones. Prefer `v0.1.1`.

List and show

```
git tag
git tag -l 'v0.1.*'
git show v0.1.0
```

Push

Tags are not pushed by default.

```
git push origin v0.1.0
git push origin --tags
```

`--tags` pushes all local tags. Prefer naming one tag for a release.

Checkout

```
git switch --detach v0.1.0
```

Detached HEAD (chapter 1). Build or inspect, then `git switch main`. To develop from a tag: `git switch -c hotfix/0.1.1 v0.1.0`.

Delete

```
git tag -d v0.1.0
git push origin --delete v0.1.0
```

Forges may still show a release page; that is forge metadata, not the Git tag.

Cleanup Commands

Cleanup Commands

After this page you can delete merged branches, drop stashes, and remove untracked files **on purpose**.

Merged branches

```
cd ~/desk/desk-api
git switch main
git branch --merged
git branch -d feat/readme-note
```

`--merged` lists names whose tips are reachable from `HEAD`. `-d` refuses otherwise; `-D` deletes anyway.

Remote leftovers:

```
git fetch --prune
git branch -r
```

`--prune` deletes remote-tracking names that the remote no longer has.

Stashes

```
git stash list
git stash drop stash@{0}
git stash clear          # every stash
```

`clear` is permanent for the stash reflog's practical purposes. Apply first if you need the diff.

Untracked files: `git clean`

```
git clean -n             # dry run
git clean -n -d          # include untracked directories
git clean -f -d          # delete
```

`-n` is mandatory the first time you run it on a tree you care about. `-f` is required (Git refuses to delete without it). `-x` also removes ignored files (build artifacts listed in `.gitignore`). That can delete `.env` if it is ignored — check `-n -x` first.

On Linux you can recover a deleted untracked file only from backups or the editor; Git never had a blob.

Garbage collection

```
git gc --aggressive --prune=now
```

Packs objects, drops unreachable ones older than the prune date. Rarely needed by hand; Git runs `gc` in the background. After a huge filter-repo (chapter 20) you will run it on purpose.

`git fsck` (peek)

```
git fsck --full
```

Reports dangling objects (often old amended commits) and real corruption. Dangling is normal. Missing blobs is chapter 21.

Hands-On: Stash and Switch

Hands-On: Stash, Work on Another Branch, Apply

Park a README edit, fix a typo on `main`, come back, apply the stash.

```
cd ~/desk/desk-api
git switch main
git status
```

Park work

```
echo "TODO: metrics" >> README.md
git stash push -m "metrics todo"
git status
git stash list
```

Other work

```
git switch -c fix/readme-typo
echo "typo-lab $(date -Iseconds)" >> README.md
git add README.md
git commit -m "docs: lab timestamp"
git switch main
git merge --ff-only fix/readme-typo
git branch -d fix/readme-typo
```

Restore the stash

```
git stash list
git stash apply stash@{0}
cat README.md
git add README.md
git commit -m "docs: TODO for metrics"
git stash drop stash@{0}
```

```
git stash list
git status
```

If `apply` conflicted, resolve markers, `git add`, commit, then `drop`.

Tag this point

```
git tag -a v0.1.0-lab -m "after stash lab"
git show v0.1.0-lab --quiet
git tag -d v0.1.0-lab
```

Done when

- `TODO: metrics` is committed on `main`
- `git stash list` is empty (or only older stashes you had before)
- status is clean

Next: hooks that run on this Linux clone before a commit exists.

stash branch

stash branch

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- `git stash branch` to avoid apply conflicts.

> **Note**
>
> Core Git depth (Linux host). See `content/_planning/SYLLABUS.md`.

git describe

git describe

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Nearest tag plus commits; version strings from Git.

> **Note**
>
> Core Git depth (Linux host). See `content/_planning/SYLLABUS.md`.

git archive

git archive

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Tarballs from a commit without the `.git` dir.

> **Note**
>
> Core Git depth (Linux host). See `content/_planning/SYLLABUS.md`.

Hooks

Theory

Theory: Hooks are Local Policy; the Server is Real Policy

A hook is an executable Git runs by **filename**. Exit non-zero aborts the Git operation. Client hooks are not cloned (by design: a clone must not execute a stranger's scripts). Therefore they are **courtesy**. Server `pre-receive` / `update` and CI are **enforcement**.

Invariants

1. `.git/hooks/pre-commit` runs on this clone only, unless `core.hooksPath` points at a shared dir.
2. `--no-verify` skips client `pre-commit` and `commit-msg`.
3. The web editor on a forge never runs your laptop hooks.
4. `core.hooksPath` and `init.templateDir` are how a Linux user shares hooks without committing `.git`.

```
git commit
   → pre-commit → commit-msg → object exists

git push
   → pre-push (client) → network → pre-receive (server) → CI
```

Why both

Client hooks fail in milliseconds (whitespace, `WIP` subject). CI fails in minutes (tests, secret scan) and cannot be skipped with `--no-verify`. Put the same *checks* in both; put *policy* on the server.

What can break

- Forgetting `chmod +x`.
- Hooks that print to stdout and corrupt `commit-msg` files.
- Assuming teammates have your hooks after `git clone`.

How to read this chapter

Types, writing, templates vs CI, lab; then `prepare-commit-msg`, post-checkout/merge, `--no-verify`.

Hook Types

Hook Types: Pre-Commit, Commit-Msg, Pre-Push

After this page you can name the client hooks this book uses and where they live on Linux.

A hook is a program Git runs

Executable files in `.git/hooks/`. Git looks at the filename. If `pre-commit` exists and is executable (`chmod +x`), Git runs it before it writes a commit object. Exit status `0` continues. Non-zero aborts.

```
git commit


pre-commit      exit   0    abort, no object
     0

commit-msg      exit   0    abort, no object
     0

commit object written, main slides forward

git push


pre-push        exit   0    abort, remote unchanged
     0

objects uploaded
```

Sample files are `*.sample` and are **not** run. Copy and chmod:

```
cd ~/desk/desk-api
ls .git/hooks
install -m 0755 .git/hooks/pre-commit.sample .git/hooks/pre-commit
# then edit; or write your own
```

`install -m 0755` is the Linux way to copy and set the executable bit in one step.

Client hooks you will write

Hook	When	Typical job
`pre-commit`	After you type `git commit`, before the object exists	Lint, block secrets, `git diff --check`
`commit-msg`	After you save the message	Conventional Commits pattern
`pre-push`	After you type `git push`, before objects upload	Tests, block WIP branches

There are more (`prepare-commit-msg`, `post-commit`, `post-checkout`). Start with these three.

Server hooks (later)

A forge runs `pre-receive` / `update` / `post-receive` on the server when you push. Chapter 23 names them; chapter 31 uses them for policy. Your laptop hooks do not run on the server. A teammate can skip `pre-commit` with `git commit --no-verify`. Server hooks are the ones that actually enforce.

Environment

Hooks run with a POSIX shell on Linux unless the shebang says otherwise. Always start with:

```
#!/usr/bin/env bash
set -euo pipefail
```

Git sets `GIT_DIR` and runs with the working tree as cwd for client hooks (with nuances for worktrees, chapter 14). Print to stderr so you do not pollute commit output:

```
echo "pre-commit: running" >&2
```

Writing a Hook

Writing Your First Hook

After this page you have a `pre-commit` that refuses a commit containing a tab-only whitespace error, and a `commit-msg` that checks a conventional subject.

`pre-commit`

```
cat > ~/desk/desk-api/.git/hooks/pre-commit <<'EOF'
#!/usr/bin/env bash
set -euo pipefail
git diff --cached --check
EOF
chmod +x ~/desk/desk-api/.git/hooks/pre-commit
```

`git diff --cached --check` flags leftover conflict markers and bad whitespace in the **index**. Exit status non-zero aborts the commit.

Try:

```
cd ~/desk/desk-api
printf 'bad \n' >> README.md    # trailing space
git add README.md
git commit -m "docs: trigger hook"
```

Fix the line, `git add`, commit again.

`commit-msg`

Git passes the path to a temporary file that holds the message:

```
cat > ~/desk/desk-api/.git/hooks/commit-msg <<'EOF'
#!/usr/bin/env bash
set -euo pipefail
file="$1"
subject="$(head -n1 "$file")"
if [[ "$subject" == WIP* ]]; then
  echo "commit-msg: subject cannot start with WIP" >&2
  exit 1
fi
```

```
EOF
chmod +x ~/desk/desk-api/.git/hooks/commit-msg
```

A stricter conventional check can wait until the team agrees on types. Start by blocking `WIP`.

pre-push

Git passes the remote name and URL, and on stdin: local ref, local hash, remote ref, remote hash.

```
cat > ~/desk/desk-api/.git/hooks/pre-push <<'EOF'
#!/usr/bin/env bash
set -euo pipefail
while read -r local_ref local_sha remote_ref remote_sha; do
  if [[ "$remote_ref" == refs/heads/main && "$local_sha" != "$remote_sha" ]]; then
    echo "pre-push: running tests before updating main" >&2
    # python -m pytest -q || exit 1
  fi
done
EOF
chmod +x ~/desk/desk-api/.git/hooks/pre-push
```

Uncomment the test command when `desk-api` has tests.

--no-verify

```
git commit --no-verify -m "docs: skip hooks"
```

Emergency only. Treat it as breaking the seatbelt.

Hook Templates and CI

Hook Templates: Sharing Hooks Across Repos

After this page you can point new repos at a shared hooks directory, and you know why CI still has to run on the server.

`.git/hooks` is local

Cloning does **not** copy hooks (Git ignores them in the payload on purpose: a clone would otherwise execute someone else's scripts). Each clone needs its own install step.

`core.hooksPath`

```
mkdir -p ~/desk/git-hooks
# put pre-commit, commit-msg, … there, chmod +x
git config --global core.hooksPath ~/desk/git-hooks
```

Every repo on this Linux user account now runs those hooks. A repo can override with `git config core.hooksPath .githooks` (a directory **inside** the project that you *do* commit).

Project-local, committed:

```
mkdir -p ~/desk/desk-api/.githooks
# copy scripts, chmod +x, commit them
git config core.hooksPath .githooks
```

New clones still must run the `git config` once, or a `Makefile` / `just` recipe / `bin/install-hooks` script. Document that in `README.md`.

`init.templateDir`

```
mkdir -p ~/desk/git-template/hooks
# scripts in that hooks dir
git config --global init.templateDir ~/desk/git-template
```

`git init` and `git clone` copy the template's `hooks/` into the new `.git/hooks`. Good for your laptop. Teammates need the same config.

CI is the hook you cannot skip

A `pre-commit` hook is a courtesy. `git commit --no-verify` and a forge's web editor skip it. **CI on the forge** (chapter 32, GitHub Actions in Part VI) runs on the server after push. Put the same checks there: `git diff --check`, tests, secret scanning.

```
laptop                          forge
.git/hooks/pre-commit           pre-receive / CI job
    skippable                       not skippable

git commit                      git push accepted

         git push
```

Client hooks catch mistakes in milliseconds on Linux. CI is the policy. Use both.

Frameworks

pre-commit.com is a Python manager that installs a tiny `.git/hooks/pre-commit` and runs a YAML list of linters. Optional. Raw bash hooks in this chapter teach the mechanism; a framework is fine in production if the team already uses it.

Hands-On: A Pre-Commit Hook

Hands-On: Create a Pre-Commit Hook

Install a `pre-commit` that runs `git diff --cached --check` on `desk-api`.

```
cd ~/desk/desk-api

cat > .git/hooks/pre-commit <<'EOF'
#!/usr/bin/env bash
set -euo pipefail
echo "pre-commit: git diff --cached --check" >&2
git diff --cached --check
EOF
chmod +x .git/hooks/pre-commit
test -x .git/hooks/pre-commit
```

Fail on purpose

```
printf 'trailing space \n' >> README.md
git add README.md
git commit -m "docs: should fail the hook"
```

Expect a non-zero exit and no new commit. `git log -1 --oneline` is unchanged.

Pass

```
git restore --staged README.md
git restore README.md
echo "hooks lab" >> README.md
git add README.md
git commit -m "docs: hooks lab"
git status
```

Clean status, new commit. Remove the hook when you are done with the lab, or leave it — `--check` is a good default:

```
rm -f .git/hooks/pre-commit
```

Leave it in place if you want the check on this clone.

Next: submodules (a repo inside a repo) and worktrees (a second working directory, same `.git`).

prepare-commit-msg

prepare-commit-msg

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Fill in a template before the editor opens.

> **Note**
>
> Core Git depth (Linux host). See `content/_planning/SYLLABUS.md`.

post-checkout and post-merge

post-checkout and post-merge

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Hooks after switch and merge (deps, submodules).

> **Note**
>
> Core Git depth (Linux host). See `content/_planning/SYLLABUS.md`.

–no-verify

–no-verify

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- When skipping hooks is justified; why CI still must run.

> **Note**
>
> Core Git depth (Linux host). See `content/_planning/SYLLABUS.md`.

Submodules and Worktrees

Theory

Theory: Submodules Pin a Hash; Worktrees Share an Object Store

A **submodule** is a second repository whose current commit is recorded in the parent as a gitlink (`mode 160000`). A **worktree** is a second working directory for the *same* repository: one `.git`, several checkouts, **one branch per worktree**.

Invariants

1. The superproject stores a URL (`.gitmodules`) and a **gitlink hash**, not the child's files as blobs.
2. `git clone` of the parent does not fetch submodule objects unless `--recurse-submodules` / `submodule update --init`.
3. Two worktrees cannot check out the same branch (indexes would fight).
4. Worktree `.git` is a **file** pointing at the main repo's `worktrees/` metadata.

```
submodule                          worktrees

desk-api tree                      ~/desk-api        HEAD → main
  src/desk-lib → gitlink 9fce      ~/desk-api-hf     HEAD → hotfix
desk-lib repo  A ← 9fce                  \              /
                                          shared objects/refs
```

Why not always submodule

If two projects always change together, you are paying pin-and-update tax for a monorepo's job. If they have separate release cycles, the pin is the feature.

What can break

- Committing in a submodule and forgetting to push it (parent points at a hash nobody has).
- Detached HEAD in the submodule by default after `update`.
- `rm -rf` a worktree directory without `git worktree remove`.

How to read this chapter

Mechanics, worktrees, when to use which, lab; then foreach/sync, deinit, locks.

Submodules

Submodule Mechanics: Add, Update, Clone

After this page you can attach another Git repository at a path and record **which commit** it should be.

A submodule is a nested repository plus a gitlink

`desk-api` might need a shared `desk-lib` history without merging the trees. A **submodule** stores a commit hash in the parent tree (a **gitlink**, mode `160000`) and a URL in `.gitmodules`.

```
desk-api  (superproject)
   README.md
   src/ping.py
   src/desk-lib/          gitlink 160000 → commit 9fceb02 of desk-lib
           (nested .git)

desk-lib  (its own history)
A ← B ← 9fceb02    ← the hash the parent pins
```

```
cd ~/desk/desk-api
git submodule add ~/desk/desk-lib src/desk-lib
git status
git commit -m "chore: add desk-lib submodule"
```

`.gitmodules` is a tracked file:

```
[submodule "src/desk-lib"]
    path = src/desk-lib
    url = /home/ada/desk/desk-lib
```

On Linux use an SSH URL for anything you will clone elsewhere (`git@host:ada/desk-lib.git`). A path only works on this machine.

Clone a superproject

```
git clone --recurse-submodules <url> desk-api
# or after a plain clone:
git submodule update --init --recursive
```

Without `--recurse-submodules` / `update --init`, the path exists as an empty directory.

Update the pin

```
cd src/desk-lib
git fetch
git switch main
git pull --ff-only
cd ../..
git add src/desk-lib
git commit -m "chore: bump desk-lib"
```

The parent records a **new gitlink hash**. Pulling the parent does not move the submodule until `git submodule update`.

Pitfalls

- Committing in the submodule and forgetting to push it: the parent points at a hash nobody else can fetch.
- Editing files in `src/desk-lib` while detached: `git switch -c` in the submodule first.
- Nested submodules: `--recursive` on clone and update.

If the team is small and `desk-lib` changes with `desk-api` every day, a **monorepo** (chapter 15) or a copy may hurt less. Submodules shine when the nested project has its own release cycle.

Worktrees

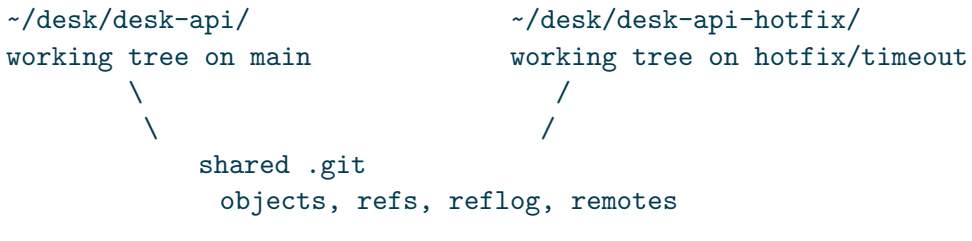
Git Worktrees: The Modern Alternative

After this page you can check out a second branch in a second directory that shares the same `.git`.

One object store, several working trees

```
cd ~/desk/desk-api
git worktree add ~/desk/desk-api-hotfix hotfix/timeout
```

Git creates `~/desk/desk-api-hotfix` on a new branch `hotfix/timeout` (or an existing branch if you name one that is not already checked out). Both directories share objects and refs. `git commit` in the hotfix tree moves `hotfix/timeout`. `main` in the original tree is untouched.




```
git worktree list
```

Why not `git clone` again

A second clone duplicates objects. A worktree is cheap and sees the same remotes, stash, and reflog. You cannot check out the **same** branch in two worktrees; Git refuses so the two indexes cannot fight.

Linux workflow

```
git fetch origin
git worktree add ~/desk/desk-api-pr origin/feat/ping-json
cd ~/desk/desk-api-pr
# review, run tests
cd ~/desk/desk-api
```

```
git worktree remove ~/desk/desk-api-pr
```

`remove` deletes the extra directory after the working tree is clean. `git worktree prune` cleans metadata if you `rm -rf` the directory by hand.

Detached review

```
git worktree add --detach ~/desk/desk-api-tag v0.1.0
```

Inspect a tag without leaving your feature branch in the main directory.

Hooks and `GIT_DIR`

Worktrees have `.git` as a **file** pointing at the main repo's `worktrees/…` metadata. Hooks still run from `core.hooksPath` or the main `.git/hooks`. Scripts that assume `.git/` is a directory should call `git rev-parse --git-dir`.

When to Use Which

When to Use Which

After this page you can pick submodule, worktree, or neither for a desk problem.

Worktree

Use a worktree when **one repository** needs two checkouts: a hotfix while a feature is dirty, a PR review, a tag build. Same remotes, same hooks, cheap on Linux disk.

Submodule

Use a submodule when **two repositories** must stay separate histories, with the parent pinning a commit of the child: a shared library, a firmware blob, a docs site with its own release train.

Copy the files

If `desk-lib` is three Python files and only `desk-api` uses them, put them in `desk-api`. Submodule overhead is real.

Monorepo

If `desk-web`, `desk-api`, and `desk-infra` always change together, one repository with three directories may be simpler (chapter 15). Worktrees still apply (two branches of the monorepo). Submodules do not.

Extra clone

`git clone` to a second path is fine for a throwaway experiment you might delete with `rm -rf`. You duplicate objects until `git gc`. Worktree is cleaner for a second **branch**.

Extra remote

Two forges is remotes (chapter 11), not submodules. Push the same history twice.

Rule of thumb

Same project, two directories → worktree. Two projects, one pin → submodule. Two projects, always in lockstep → one repo.

```
one repo, two checkouts         worktree
two repos, pin a commit         submodule
two repos, always together      one monorepo
two forges, same history        extra remote
```

Hands-On: Submodule and Worktree

Hands-On: A Repo with a Submodule and a Worktree

Two small Git repos on Linux, then a worktree of the parent.

Child library

```
mkdir -p ~/desk/desk-lib
cd ~/desk/desk-lib
git init
echo "DESK_LIB=1" > version.txt
git add version.txt
git commit -m "chore: desk-lib version pin"
```

Parent with submodule

```
cd ~/desk/desk-api
git submodule add ~/desk/desk-lib src/desk-lib
git status
git commit -m "chore: add desk-lib submodule"
ls src/desk-lib/version.txt
```

Worktree of the parent

```
git worktree add ~/desk/desk-api-wt -b feat/wt-lab
cd ~/desk/desk-api-wt
git status
echo "worktree lab" >> README.md
git add README.md
git commit -m "docs: worktree lab"
cd ~/desk/desk-api
git log --oneline feat/wt-lab -1
```

`main` in `~/desk/desk-api` does not have that commit until you merge.

Cleanup

```
cd ~/desk/desk-api
git worktree remove ~/desk/desk-api-wt
git switch main
git merge --ff-only feat/wt-lab
git branch -d feat/wt-lab
```

Leave the submodule in place for later chapters, or remove it:

```
git rm src/desk-lib
git commit -m "chore: remove desk-lib submodule"
rm -rf ~/desk/desk-lib
```

Removing the submodule does not delete `~/desk/desk-lib` unless you `rm -rf` it.

Done when

- `git worktree list` shows only the main tree
- You know whether `src/desk-lib` is still a gitlink (`git ls-files -s src/desk-lib` shows mode `160000`) or is gone

Next: Git Flow, GitHub Flow, trunk-based — how teams name these branches on purpose.

foreach and sync

foreach and sync

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- `git submodule foreach`, `git submodule sync`.

> **Note**
>
> Core Git depth (Linux host). See `content/_planning/SYLLABUS.md`.

deinit and absorbgitdirs

deinit and absorbgitdirs

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- `submodule deinit`, `absorbgitdirs`.

> **Note**
>
> Core Git depth (Linux host). See `content/_planning/SYLLABUS.md`.

Worktree Locks and prune

Worktree Locks and prune

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- `git worktree lock`, `prune`, why two checkouts cannot share a branch.

> **Note**
>
> Core Git depth (Linux host). See `content/_planning/SYLLABUS.md`.

Worktree Lifecycle for Parallel Work: add, prune, repair

Worktree Lifecycle for Parallel Work: `add`, `prune`, `repair`

Thesis

A linked worktree is a second checkout of the **same** repository: one object database and one set of refs, but separate files, a separate index and a separate `HEAD`. That makes worktrees the cheapest way to run several lines of work side by side: a hotfix next to a feature, a throwaway build next to your editor, or several coding agents that must not overwrite each other's files.

Creating worktrees is easy. What goes wrong is the **end of their life**. Delete the directory with `rm -rf` instead of `git worktree remove` and Git still thinks it exists. The branch stays "checked out" somewhere nobody can see, you cannot re-create the worktree at the same path, and you cannot delete the branch. Move the directory and Git loses track of it. Scripts that create one worktree per run pile up these dead entries until something fails.

This page walks the whole lifecycle on a real repository: create, inspect, break, prune, lock, move, repair. It ends with a throwaway-worktree script that cannot leak.

Everything below was run with `git version 2.56.0`. `worktree list` has shown `prunable` since Git 2.31, and `--relative-paths` needs 2.48 or later. Commit hashes in your run will differ.

Mental model

```
app/.git/                          ← the one repository
   objects/, refs/                ← shared by every worktree
   HEAD, index                    ← main worktree's own
   worktrees/
       app-auth/                  ← admin dir for ../app-auth
          HEAD, index             ← that worktree's own
          gitdir                 /tmp/wt/app-auth/.git   (back-link)
          locked                    ← optional, holds the reason
       app-api/  ...

app-auth/.git   (a file, not a directory)
    "gitdir: /tmp/wt/app/.git/worktrees/app-auth"   (forward link)
```

Fact	Consequence
Objects and refs are shared	A commit made in one worktree is immediately visible from all of them. No fetch or push between them
A branch can be checked out in only **one** worktree	`git switch` refuses with "already used by worktree". Two agents can never share a branch by accident
Git links worktrees in both directions with absolute paths (by default)	Moving either the repository or the worktree breaks a link until `git worktree repair`
Deleting the directory does not delete the admin dir	The entry stays, marked `prunable`, until `git worktree prune` (or `gc`) removes it
`gc` prunes stale entries only after `gc.worktreePruneExpire` (default `3.months.ago`)	Dead entries from scripts can linger for months
A `locked` worktree is never pruned	Use it for worktrees on removable or network storage

Worked examples (Cases)

```
#!/bin/sh
# setup.sh - a tiny repository to practise worktrees on
set -eu
git init -q -b main app
cd app
git config user.name demo; git config user.email demo@example.com
echo "v1" > README; git add README; git commit -q -m "init"
```

```
mkdir -p /tmp/wt && cd /tmp/wt && sh setup.sh && cd app
```

Case 1: One worktree per line of work

```
git worktree add -b feat/auth ../app-auth
git worktree add -b feat/api ../app-api
git worktree list
git switch feat/auth
```

```
Preparing worktree (new branch 'feat/auth')
HEAD is now at 06fb4a0 init
Preparing worktree (new branch 'feat/api')
HEAD is now at 06fb4a0 init
/tmp/wt/app      06fb4a0 [main]
```

```
/tmp/wt/app-api  06fb4a0 [feat/api]
/tmp/wt/app-auth 06fb4a0 [feat/auth]
fatal: 'feat/auth' is already used by worktree at '/tmp/wt/app-auth'
```

The refusal is the feature. If you give each agent or terminal its own branch *and* its own worktree, they cannot write to each other's files or move each other's `HEAD`.

Work done in one worktree is visible in the main one straight away, because the refs are shared:

```
echo token > ../app-auth/auth.txt
git -C ../app-auth add auth.txt
git -C ../app-auth commit -q -m "auth: token"
git log --oneline feat/auth
```

```
ef29406 auth: token
06fb4a0 init
```

Case 2: `rm -rf` leaves a ghost

```
rm -rf ../app-api
git worktree list
git worktree add ../app-api feat/api
git branch -d feat/api
```

```
/tmp/wt/app      06fb4a0 [main]
/tmp/wt/app-api  06fb4a0 [feat/api] prunable
/tmp/wt/app-auth ef29406 [feat/auth]
Preparing worktree (checking out 'feat/api')
fatal: '../app-api' is a missing but already registered worktree;
use 'add -f' to override, or 'prune' or 'remove' to clear
error: cannot delete branch 'feat/api' used by worktree at '/tmp/wt/app-api'
```

Three symptoms with one cause: `.git/worktrees/app-api/` still exists. `prune` removes it. Preview first:

```
git worktree prune -v --dry-run
git worktree prune -v
git worktree list
```

```
Removing worktrees/app-api: gitdir file points to non-existent location
Removing worktrees/app-api: gitdir file points to non-existent location
/tmp/wt/app      06fb4a0 [main]
/tmp/wt/app-auth ef29406 [feat/auth]
```

`add -f` would also "work", but it re-registers over the ghost without telling you what the ghost was. Prune first, so you know what you are clearing.

Case 3: Lock what lives somewhere unreliable

A worktree on a USB disk or a network mount looks missing whenever that storage is unmounted, and `prune` would delete its entry. Lock it:

```
git worktree lock --reason "on USB disk" ../app-auth
git worktree list --porcelain
```

```
worktree /tmp/wt/app
HEAD 06fb4a058502ead9fe687ef46f0b713757558027
branch refs/heads/main

worktree /tmp/wt/app-auth
HEAD ef2940632571767084 20c85cab3c9f9655d74166
branch refs/heads/feat/auth
locked on USB disk
```

Use `--porcelain` in scripts: it is stable, one attribute per line, and includes the lock reason. A locked worktree survives `prune` even when its directory is gone:

```
git worktree add -q --lock --reason ci -b feat/l ../app-l
rm -rf ../app-l
git worktree prune -v
git worktree list
```

```
/tmp/wt/app     06fb4a0 [main]
/tmp/wt/app-l   06fb4a0 [feat/l] locked
/tmp/wt/app-rel 06fb4a0 [feat/rel]
/tmp/wt/moved   ef29406 [feat/auth]
```

(`app-rel` and `moved` come from the next two cases; this listing was taken at the end of the session.) Nothing was pruned. A lock is a promise that the directory will come back. Unlock it with `git worktree unlock` when that is no longer true.

Case 4: `remove` protects your work, `mv` breaks the link, `repair` fixes it

```
git worktree unlock ../app-auth
echo wip > ../app-auth/x
git worktree remove ../app-auth
```

```
fatal: '../app-auth' contains modified or untracked files, use --force to delete it
```

`remove` refuses to throw away uncommitted work. That safety check is exactly what `rm -rf` skips.

Now move the worktree with plain `mv`:

```
mv ../app-auth ../moved
git worktree list
git -C ../moved status --short
git worktree repair ../moved
git worktree list
cat ../moved/.git
cat .git/worktrees/app-auth/gitdir
```

```
/tmp/wt/app      06fb4a0 [main]
/tmp/wt/app-auth ef29406 [feat/auth] prunable
?? x
repair: gitdir incorrect: /tmp/wt/app/.git/worktrees/app-auth/gitdir
/tmp/wt/app   06fb4a0 [main]
/tmp/wt/moved ef29406 [feat/auth]
/tmp/wt/moved/.git
gitdir: /tmp/wt/app/.git/worktrees/app-auth
```

Inside `../moved`, Git still works (the forward link is intact). From the main repository the worktree looks dead, and a `prune` at that moment would have deleted its entry. `repair` rewrote the back-link. Better still, move worktrees with `git worktree move`, which updates both links, and run `repair` from inside the main repository after moving the repository itself.

Case 5: Relative links for repositories that move

If the whole tree gets moved together (a container bind mount at a different path, a synced home directory), use relative links:

```
git worktree add --relative-paths -b feat/rel ../app-rel
cat ../app-rel/.git
cat .git/worktrees/app-rel/gitdir
git config --get extensions.relativeWorktrees
```

```
Preparing worktree (new branch 'feat/rel')
HEAD is now at 06fb4a0 init
gitdir: ../app/.git/worktrees/app-rel
../../../../app-rel/.git
true
```

Git turns on the repository extension `extensions.relativeWorktrees`, so Git versions older than 2.48 will refuse to open the repository. Set `worktree.useRelativePaths=true` to make it the default for new worktrees.

Case 6: A throwaway build that cannot leak

```
#!/bin/sh
# build-in-worktree.sh - build a ref in a throwaway worktree and always clean up
set -eu
ref=${1:-HEAD}
dir=$(mktemp -d)/build
git worktree add --detach --quiet "$dir" "$ref"
trap 'git worktree remove --force "$dir"; rmdir "$(dirname "$dir")"' EXIT
cd "$dir"
echo "building $(git rev-parse --short HEAD) in $dir"
cat README
```

```
for i in 1 2 3; do sh ../build-in-worktree.sh main; done
git worktree list
```

```
building 06fb4a0 in /tmp/tmp.7WzwdgzHCW/build
v1
building 06fb4a0 in /tmp/tmp.JUblBcUqaw/build
v1
building 06fb4a0 in /tmp/tmp.I9w49C4jYp/build
v1
/tmp/wt/app     06fb4a0 [main]
/tmp/wt/app-rel 06fb4a0 [feat/rel]
/tmp/wt/moved   ef29406 [feat/auth]
```

Three runs, zero leftovers. `--detach` avoids tying up a branch, and the `EXIT` trap removes the worktree through Git even when the build fails. Compare a script that cleans up with `rm -rf`:

```
d=$(mktemp -d)/b; git worktree add --detach -q "$d" main; rm -rf "$d"
git worktree list
git worktree prune -v
```

```
/tmp/wt/app           06fb4a0 [main]
/tmp/tmp.AkImVxMd26/b 06fb4a0 (detached HEAD) prunable
/tmp/wt/app-rel       06fb4a0 [feat/rel]
/tmp/wt/moved         ef29406 [feat/auth]
Removing worktrees/b: gitdir file points to non-existent location
```

Each such run leaves one more `prunable` entry, and `gc` will not clean them up for three months.

The trap

A fixed worktree path plus one crash equals a script that never runs again. Many deploy scripts and agent harnesses use the same path every time (`../build`, `../agent-1`). If one run is

killed before its clean-up, the next run hits Case 2's error at `git worktree add` (Git checks the *path* against registered entries) and stops. The usual "fix" is `add -f`, which hides the problem instead of finding out why the last run died.

With unique paths (`mktemp`) the failure is quieter but still there. Git names admin directories after the last path component, so the leftovers from Case 6's `rm -rf` variant look like this:

```
for i in 1 2 3; do d=$(mktemp -d)/build; git worktree add --detach -q "$d" main; rm -rf "$d"
ls .git/worktrees
```

```
app-auth
app-l
app-rel
build
build1
build2
```

Nothing in those names tells you which run created them or whether anything still owns them. The fix is not a cleverer naming scheme. Remove worktrees with `git worktree remove`, and run `git worktree prune` at the **start** of any script or harness that creates worktrees, so each run begins from a known state.

The boring rule

> One task, one branch, one worktree. Create it with `git worktree add`, end it with `git worktree remove` (in a `trap` for scripts), move it with `git worktree move`, and lock anything on storage that comes and goes. Start every worktree-creating script with `git worktree prune`, and read state with `git worktree list --porcelain`, never by listing directories.

Try this

1. Run Case 6 with the `rm -rf` variant ten times, then list `.git/worktrees/`. Run `git worktree prune -v` and count the lines.
2. Move the **main** repository (`mv /tmp/wt/app /tmp/wt/app2`), run `git -C /tmp/wt/moved status`, and then fix it with `git worktree repair /tmp/wt/moved` from inside `app2`.
3. Set `gc.worktreePruneExpire=now`, create and `rm -rf` a worktree, and confirm that `git gc` removes the entry.
4. Write a harness that starts three parallel jobs, each in its own `--detach` worktree, and prove with `git worktree list --porcelain` that none survive a `kill` of the harness (hint: trap `INT` and `TERM` as well as `EXIT`).

Sources

- git-worktree(1): `add`, `list --porcelain`, `lock`, `move`, `prune`, `remove`, `repair`, `--relative-paths`, `worktree.useRelativePaths`
- git-config(1): `gc.worktreePruneExpire`, `extensions.relativeWorktrees`
- git-gc(1): when `gc` prunes worktree metadata
- Git release notes: 2.31 (`prunable` in `worktree list`), 2.48 (relative worktree paths)

Branching Workflows

Theory

Theory: A Workflow is a Policy on Names, Not a Git Feature

Git Flow, GitHub Flow, and trunk-based development are **agreements** about which refs are long-lived, how long a feature name lives, and how `main` stays deployable. Git will merge whatever you ask. The workflow is which asks you allow.

Invariants

1. Long-lived branches **diverge**; every extra one is a merge or rebase tax.
2. Short-lived `feat/*` off `main` (GitHub Flow topology) is independent of GitHub the product.
3. Trunk-based requires **runtime flags** or very small commits so `main` can take unfinished code.
4. Monorepo vs polyrepo is a **repository boundary** decision; worktrees still apply; submodules usually do not in a monorepo.

```
Git Flow          main + develop + release/*     high ceremony
GitHub Flow       main + short feat/*            this book's desk default
Trunk-based       main + flags                   high CI, tiny commits
```

Why write it down

A strategy that exists only in chat is not enforceable. Chapter 31 (governance) is how a forge *requires* reviews and checks. This chapter is the graph you are asking Git to keep.

What can break

- Git Flow on a two-person `desk-api` (ceremony without versions in the wild).
- Trunk-based without tests or flags.
- A monorepo whose CI always builds everything.

How to read this chapter

Git Flow, GitHub Flow/trunk, flags, mono/poly, choosing; then forking, stacked changes, release tags.

Git Flow

Git Flow: The Classic Model

After this page you can name Git Flow's long-lived branches and when the extra machinery is worth it.

The picture

```
main          production; tagged releases
develop       next release; features merge here
feat/…        branched from develop, merged back (often --no-ff)
release/x.y   branched from develop, hotfixes, then merge to main *and* develop
hotfix/…      branched from main, merge to main *and* develop
```

`main` is always what is in production. `develop` is the integration line. Features never commit straight to `main`.

```
 main      A           H1   v1.2
             \
 develop      B ← C ← D ← E ← F
                    \
 feat/invoices        X ← Y
```

Commands (shape, not a plugin)

```
cd ~/desk/desk-api
git switch main
git switch -c develop
git switch -c feat/invoices develop
# … commits …
git switch develop
git merge --no-ff feat/invoices
git branch -d feat/invoices
```

`--no-ff` keeps a join node so `git log --first-parent develop` lists features.

What it costs

Two long-lived branches that **diverge**. Every hotfix is merged twice. Release branches exist for weeks. For `desk-api` with one or two people, this is a lot of graph.

What it buys

A support team that must ship `1.2.3` while `develop` already contains `2.0` work. Multiple versions in the wild. A release manager.

Use Git Flow when you **actually** have that. The original `git-flow` CLI is optional; the branch names are the model.

Chapter 31 is how a forge **enforces** this (protected `main`, required PRs). This page is the topology.

GitHub Flow and Trunk-Based

GitHub Flow and Trunk-Based Development

After this page you can contrast a short-lived PR branch with committing to `main` behind flags.

GitHub Flow (as a Git topology)

```
main          always deployable
feat/…        branched from main, opened as a PR, merged, deleted
```

There is no long-lived `develop`. Production is `main` (or a tag on it). CI on the PR is the gate.

```
 GitHub Flow

 main     A ← B ← C ← M          always deployable

 feat/ping      D ← E            PR, then delete
```

```
git switch main
git pull --ff-only
git switch -c feat/ping-json
# commits, push, PR, merge on the forge
git switch main
git pull --ff-only
git branch -d feat/ping-json
```

The name "GitHub Flow" is a topology. You can do it on Forgejo or GitLab. The forge UI is a PR/MR; Git is short-lived branches off `main`.

Trunk-based

Engineers commit to `main` (the **trunk**) at least daily, often several times a day. Features that are not ready stay **dark** behind a flag (next page). Release branches, if any, are cut from `main` and live days, not months.

```
trunk-based

main     A ← B ← C ← D ← E ← F     tiny commits, flags hide unfinished work
```

```
                    release/1.2   short-lived, if you even cut one
```

```
git switch main
# small commit
git pull --rebase
git push
```

Feature branches, when used, last hours. Rebase onto `main` before merge. `--ff-only` after a rebase is the usual integrate.

Compare

	Git Flow	GitHub Flow	Trunk-based
Long-lived lines	`main` + `develop`	`main`	`main`
Feature life-time	Days–weeks	Hours–days	Hours or a flag
Release	`release/*` branch	Tag / deploy `main`	Tag / short `release/*`
Fits	Multi-version products	Web apps, desk tools	High-CI teams

`desk-api` in this book defaults to **GitHub Flow topology**: short `feat/` branches, `main` deployable. Trunk-based is the same graph with even shorter branches.

Enforcement (required reviews, required checks) is chapter 31. The next page is flags, which make trunk-based possible.

Feature Flags

Feature Flags and Deployment

After this page you can ship code to `main` that is not yet on for users.

A flag is a runtime switch

The commit on `main` contains the new code. Production **does not call it** until a config value is true. That is how trunk-based teams merge daily without releasing half-built ping JSON.

```
main     A ← B ← C ← flag-commit ← D     all on main

production
  DESK_PING_JSON=0   →  old string path
  DESK_PING_JSON=1   →  JSON path

Git shipped the code at flag-commit; the env var is the release.
```

Toy `desk-api` (Python, Linux):

```
# src/ping.py
import os

def ping():
    if os.environ.get("DESK_PING_JSON") == "1":
        return {"status": "ok"}
    return "ok"
```

```
export DESK_PING_JSON=1
python -c 'from src.ping import ping; print(ping())'
```

The flag lives in the environment, a file under `/etc`, or a later config service. Git holds the code; the flag holds the **release**.

What Git still does

- Small commits on `main` (or short `feat/` branches)
- Revert the commit if the code is wrong (chapter 10)
- The flag can stay off while you fix

A flag is not a substitute for `git revert` when the code crashes before the flag is read.

Cleanup

Every flag is debt. When `DESK_PING_JSON` is on everywhere, delete the old branch in the `if` and the env var. A follow-up commit: `chore: remove ping JSON flag`.

GitOps preview

Later parts deploy from Git. A flag in the **application** is still an env var on the cluster. A flag in **Git** (a value in `desk-infra`) is GitOps: changing the file and merging to `main` is the release. Same idea, different file.

Monorepos vs Polyrepos

Monorepos vs Polyrepos

After this page you can place `desk-web`, `desk-api`, and `desk-infra` in one Git repository or three, on purpose.

Polyrepo (this book's default so far)

Three repositories, three histories, three remotes:

```
~/desk/desk-web
~/desk/desk-api
~/desk/desk-infra
```

Each clones, branches, and CI-runs on its own. A change that spans API and web is two pull requests. Versions of each other are documented ("web 1.2 needs api 1.4") or ignored until something breaks.

Monorepo

One repository:

```
~/desk/desk/
  web/
  api/
  infra/
```

One `git clone`, one `main`, one PR can touch all three. CI must learn to run only what changed, or it becomes slow. Sparse checkout and partial clone (chapter 19) exist because monorepos grow.

Trade-offs

	Monorepo	Polyrepo
Cross-project change	One PR	Several PRs, several versions
Access control	One repo (paths, CODEOWNERS)	Separate remotes, easier per-team ACLs
CI	Must be smart or it is slow	Simple per repo

	Monorepo	Polyrepo
Submodules	Rarely needed	Common for shared libs
Git scale	Packfiles, sparse, LFS	Each repo stays small

Linux tooling

Path-based CI (`git diff --name-only origin/main...HEAD | grep '^api/'`) is how a monorepo stays fast. `CODEOWNERS` (chapter 31) assigns `api/` to one team. None of that requires GitHub; the files are in Git.

This book

Labs keep **three polyrepos** so platform chapters can push `desk-api` to GitLab and Forgejo independently. If your real desk is a monorepo, the Git commands are the same; only the directory layout changes. Worktrees still apply. Submodules usually do not.

Choosing a Strategy

Choosing a Strategy

After this page you can write a short default for `desk-api` and know where this book will enforce it later.

```
versions in the wild?   yes    Git Flow (main + develop + release/*)

       no

several merges/day + flags?   yes    trunk-based

       no

GitHub Flow     short feat/* off main     ← desk default
```

A default for this book's desk

- **Topology:** GitHub Flow — short `feat/` / `fix/` branches off `main`, delete after integrate
- **Integrate:** rebase the feature onto `main`, then `--ff-only` or a forge squash
- **`main`:** always deployable; revert to undo (never rebase published `main`)
- **Repos:** polyrepo (`desk-web`, `desk-api`, `desk-infra`) until a chapter says otherwise
- **Host:** Linux; hooks and CI run there first

Write it in `README.md` or `CONTRIBUTING.md` in the repo. A strategy that exists only in someone's head is not a strategy.

When to change it

- Multiple supported versions in production → Git Flow release branches
- Several merges to `main` per engineer per day and strong tests → trunk-based plus flags
- One PR always touches all three desks → consider a monorepo
- A compliance officer needs a merge commit per change → `--no-ff` on `main`

What Git will not decide

Git will merge whatever you ask. Forges add buttons (squash, rebase, merge). Chapter 31 is branch protection, code owners, required checks. This chapter is the **agreement**. The next part (power

skills) is internals, reflog, bisect, and repair — the tools you want when the agreement meets a bad Tuesday.

The Forking Workflow

The Forking Workflow

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Personal fork, `upstream`, PRs from forks.

> **Note**
>
> Core Git depth (Linux host). See `content/_planning/SYLLABUS.md`.

Stacked Changes

Stacked Changes

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Dependent branches, rebase –update-refs, Gerrit preview.

> **Note**
>
> Core Git depth (Linux host). See `content/_planning/SYLLABUS.md`.

Release Tags and Changelogs

Release Tags and Changelogs

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Annotated tags, Conventional Commits → changelog.

> **Note**
>
> Core Git depth (Linux host). See `content/_planning/SYLLABUS.md`.

Power Skills

Internals

Theory

Theory: Content Addressing is the Whole Trick

Git is a content-addressed filesystem plus names. Internals are not trivia; they are why hosting, LFS, and `fsck` look the way they do.

Invariants

1. Object name = hash of type + size + payload (SHA-1 historically, SHA-256 optional).
2. Blobs have no filename; trees bind names to hashes.
3. Packfiles are compression (deltas) of that object set, not the data model.
4. A server is constrained by quarantine, `receive.denyCurrentBranch`, and hook timing around these objects.

```
commit → tree → blob
   \→ parent commit
packfile = many objects + delta chains  (disk, not mental model)
```

What can break

- Editing files under `.git/objects` by hand.
- Assuming SHA-1 collision is a daily concern (it is a research/hosting concern).

How to read this chapter

Object store, hashes, packfiles/`git log`, server-facing internals, `cat-file` lab.

The Object Store

The Object Store

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Blobs, trees, commits, and tags in `.git/objects`.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

Content Addressing — SHA-1, SHA-256, and the Git 3.0 Switch

Content Addressing — SHA-1, SHA-256, and the Git 3.0 switch

Every Git object is named by a hash of its own bytes. That one decision is why history is tamper-evident, why two clones agree on what "commit `7c77260`" means without talking to each other, and why deduplication is free. It is also why changing the hash function is the hardest migration Git has ever planned: the hash *is* the name, so a new function renames every object in every repository.

Git has supported **SHA-256 repositories** since 2.29. The project's `BreakingChanges` document now commits to making SHA-256 the **default for new repositories in Git 3.0** (no release date yet), alongside `main` as the initial branch and reftable as the ref backend. This chapter shows what content addressing actually hashes, what a SHA-256 repository looks like, what does not cross between the two formats, and how to get ready without breaking anything today.

Mental model

```
object bytes  =  "<type> <size>\0" + payload
object name   =  H(object bytes)          H = SHA-1  (40 hex, 20 bytes)
                                          H = SHA-256 (64 hex, 32 bytes)

blob
blob     tree   commit   commit   …     every arrow is a hash
blob                                       → change one byte, every name above it changes

one repository = ONE object format (extensions.objectFormat)
two formats    = two different names for the same content → no shared history on the wire
```

	SHA-1 repository	SHA-256 repository
Object name length	40 hex	64 hex
`core.repositoryformatversion`	0 (or 1)	1
`extensions.objectFormat`	unset (or `sha1`)	`sha256`
Empty tree	`4b825dc…`	`6ef19b4…`
Push/fetch with the other format	refused	refused
Default today (2.56)	yes	`--object-format=sha256`
Default in Git 3.0 (planned)	no	yes, for **new** repositories

SHA-1 is not being removed: `BreakingChanges` states there is no plan to deprecate the `sha1` format. Existing repositories keep working. What changes is what `git init` creates when you do not say.

Worked examples

All cases were run on Debian 13 with **Git 2.56.0** built from the release tarball, plus two extra builds of the same source: one with `WITH_BREAKING_CHANGES=YesPlease` (a preview of Git 3.0 defaults) and one with Rust enabled (needed for Case 6). Commands assume `git` is 2.56; the other two builds are called by full path.

```
git --version
```

```
git version 2.56.0
```

To make the object names below reproducible on your machine, fix identity and dates first:

```
export GIT_AUTHOR_NAME=Ada GIT_AUTHOR_EMAIL=ada@example.com
export GIT_COMMITTER_NAME=Ada GIT_COMMITTER_EMAIL=ada@example.com
export GIT_AUTHOR_DATE='2026-10-08T09:00:00+05:30' GIT_COMMITTER_DATE='2026-10-08T09:00:00+0
```

Case 1: hash an object by hand

A blob is `"blob <size>\0"` followed by the file contents. Hash those bytes with the ordinary coreutils tools and compare with what Git computes in a repository of each format:

```
printf 'hello\n' > hello.txt
printf 'blob 6\0hello\n' | sha1sum
printf 'blob 6\0hello\n' | sha256sum

git init -q s1
git init -q --object-format=sha256 s256
for r in s1 s256; do (cd $r && cp ../hello.txt . && echo "$r: $(git hash-object hello.txt)")
```

```
ce013625030ba8dba906f756967f9e9ca394464a  -
2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4  -
s1: ce013625030ba8dba906f756967f9e9ca394464a
s256: 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4
```

Same bytes, same algorithm, same name, with no Git involved. That is the whole trick, and it is also the whole problem: the SHA-256 name of `hello` shares nothing with its SHA-1 name.

Case 2: what a SHA-256 repository is

```
for r in s1 s256; do
  (cd $r && echo "== $r" &&
   git rev-parse --show-object-format --show-ref-format &&
   git config --get core.repositoryformatversion; git config --get extensions.objectformat)
```

```
done
```

```
== s1
sha1
files
0
== s256
sha256
files
1
sha256
```

The format is one line of repository config plus a format-version bump to 1, which makes Git versions that predate repository extensions refuse to open the repository rather than misread it. Never edit `extensions.objectFormat` by hand on a repository with objects in it: the existing objects keep their old names, and Git will not find them.

Commits look the same, just with longer names:

```
git init -q -b main --object-format=sha256 app256 && cd app256
echo 'hello' > README && git add README && git commit -q -m 'first commit'
git log --format='%H %s'
git rev-parse --short HEAD
git cat-file -p HEAD
git log --format=%H | awk '{print length($0)}'
cd ..
```

```
7c77260fad0cad11b0ae2ca24098d6d36ee56d96ad221d0d6f485f0911ac3c33 first commit
7c77260
tree cacbd5aeac8aafd15f1acb153e9a189e0b607b7ee7d874aee3534d1babdfcb21
author Ada <ada@example.com> 1791430200 +0530
committer Ada <ada@example.com> 1791430200 +0530

first commit
64
```

Abbreviated names stay seven characters by default, so everyday output barely changes. Scripts that consume full names are a different story (Case 4).

Case 3: what does not cross between formats

```
git init -q --bare -b main remote-sha1.git
(cd app256 && git push ../remote-sha1.git main)

git init -q -b main app1 && (cd app1 && echo x > f && git add f && git commit -q -m one)
(cd app256 && git fetch ../app1 main)
```

```
git clone -q app256 clone256 && git -C clone256 rev-parse --show-object-format
```

```
fatal: the receiving end does not support this repository's hash algorithm
fatal: the remote end hung up unexpectedly
fatal: mismatched algorithms: client sha256; server sha1
sha256
```

- **Push and fetch across formats are refused**, in both directions. No silent conversion happens.
- **Clone adopts the remote's format.** A SHA-256 server produces SHA-256 clones; you do not choose.
- The consequence for teams: a repository's format is a property of the **project**, decided once at creation, and every mirror, fork, CI cache, and forge that touches it must speak that format. `BreakingChanges` makes the Git 3.0 default switch conditional on exactly that ecosystem readiness: libraries, applications, and forges. Check your hosting platform, CI runners, and Git libraries before you create SHA-256 repositories that others must use.

Case 4: tooling that hard-codes SHA-1

Two assumptions break first: "an object name is 40 hex characters" and magic constants like the empty-tree ID.

```
for r in app1 app256; do
  echo "== $r ($(git -C $r rev-parse --show-object-format))"
  git -C $r hash-object -t tree /dev/null
  git -C $r rev-parse HEAD | grep -Exq '[0-9a-f]{40}' && echo "40-hex check: pass" || echo "
  git -C $r rev-parse HEAD | grep -Exq '[0-9a-f]{40}([0-9a-f]{24})?' && echo "40-or-64 check
done
```

```
== app1 (sha1)
4b825dc642cb6eb9a060e54bf8d69288fbee4904
40-hex check: pass
40-or-64 check: pass
== app256 (sha256)
6ef19b41225c5369f1c104d45d8d85efa9b057b53b14b4b9b939dd74decc5321
40-hex check: FAIL
40-or-64 check: pass
```

Audit hooks, CI scripts, and database columns for:

Assumption	Where it hides	Replace with
`[0-9a-f]{40}`	Hook validation, log parsers, webhook handlers	Accept 40 **or** 64, or ask `git rev-parse --show-object-format`
`CHAR(40)` / fixed-width columns	Deploy trackers, review tools	Width 64
`4b825dc…` empty tree	"diff against nothing" scripts	`git hash-object -t tree /dev/null`
`0000…0000` (40 zeros) for created/deleted refs	`pre-receive` / `update` hooks	Compare against a string of zeros of the *received* length
"short SHA is unique enough" in URLs	Dashboards	Unchanged; abbreviation rules are the same

Case 5: previewing Git 3.0 defaults, and pinning yours

Git's source can be built with the planned Git 3.0 behaviour switched on:

```
make prefix=/opt/git-3preview NO_TCLTK=1 NO_RUST=1 WITH_BREAKING_CHANGES=YesPlease all
sudo make prefix=/opt/git-3preview NO_TCLTK=1 NO_RUST=1 WITH_BREAKING_CHANGES=YesPlease inst
/opt/git-3preview/bin/git version --build-options | grep default
```

```
default-ref-format: reftable
default-hash: sha256
```

```
/opt/git-3preview/bin/git init g3
cd g3 && /opt/git-3preview/bin/git rev-parse --show-object-format --show-ref-format && /opt/
```

```
hint: Using 'main' as the name for the initial branch since Git 3.0.
hint: If you expected Git to create 'master', the just-created
hint: branch can be renamed via this command:
hint:
hint:   git branch -m master
hint:
hint: Disable this message with "git config set advice.defaultBranchName false"
Initialized empty Git repository in /tmp/vcs/g3/.git/
sha1
reftable
refs/heads/main
```

`main` and reftable flipped as documented. The object format did **not**: this 2.56.0 preview build reports `default-hash: sha256` in its build options, but in our runs `git init` without other settings (also tried with an empty environment) still created a SHA-1 repository. The `--object-format` flag, `init.defaultObjectFormat`, and `GIT_DEFAULT_HASH` all produced SHA-256 as expected. Reading the source shows why tests may not notice: `t/test-lib.sh` exports `GIT_DEFAULT_HASH` from the build's `default-hash`. The lesson is the boring one: **a preview build is a preview**. Do not infer

the future default from it; state the format you want.

Stating it is cheap and works on today's Git, on the preview, and on Git 3.0:

```
git -c init.defaultObjectFormat=sha256 init -q -b main k1 && git -C k1 rev-parse --show-obje
GIT_DEFAULT_HASH=sha256 git init -q -b main k2 && git -C k2 rev-parse --show-object-format
/opt/git-3preview/bin/git -c init.defaultRefFormat=files -c init.defaultBranch=master init -
/opt/git-3preview/bin/git -C k3 rev-parse --show-object-format --show-ref-format
/opt/git-3preview/bin/git -C k3 symbolic-ref HEAD
```

```
sha256
sha256
sha1
files
refs/heads/master
```

For a fleet, put the choices in system or global config (or your provisioning tooling) so the Git 3.0 upgrade changes nothing you did not decide:

```
# /etc/gitconfig (or ~/.gitconfig) - make today's defaults explicit
[init]
    defaultObjectFormat = sha1     # or sha256, once your forge and tools are ready
    defaultRefFormat = files
    defaultBranch = main
```

Case 6: the interoperability work (`compatObjectFormat`)

Git 2.52's release notes mark "the beginning of SHA1-SHA256 interoperability work": a SHA-256 repository that also maintains a SHA-1 name for every object, so that it can one day talk to SHA-1 peers. In 2.56 the switch exists but needs a Rust-enabled build. The default `NO_RUST=1` build refuses:

```
git init -q -b main --object-format=sha256 compat && cd compat
git config extensions.compatObjectFormat sha1
echo hi > a && git add a
```

```
fatal: compatibility hash algorithm support requires Rust
```

With Git 2.56.0 built with Rust enabled (`rust: enabled` in `--build-options`, rustc 1.99), the same steps work. Then build the same content in a plain SHA-1 repository and compare:

```
G=/opt/git-2.56-rust/bin/git
$G init -q -b main --object-format=sha256 compat && cd compat
$G config extensions.compatObjectFormat sha1
echo hi > a && $G add a && $G commit -q -m c1
$G rev-parse HEAD
```

```
$G rev-parse --output-object-format=sha1 HEAD
ls .git/objects
cd .. && $G init -q -b main plain1 && cd plain1
echo hi > a && $G add a && $G commit -q -m c1 && $G rev-parse HEAD
cd ../compat
$G rev-parse --output-object-format=sha1 'HEAD^{tree}'
$G -C ../plain1 rev-parse 'HEAD^{tree}'
$G cat-file -t 15307cff4ea19f61d3bb0b98bc73102e2454352c
$G push ../r1.git main      # r1.git: a bare SHA-1 repository
```

```
13c18aac72ade018e9d88a6f80239f88397fdf997c3437bce30b870bf87e720a
15307cff4ea19f61d3bb0b98bc73102e2454352c
00
13
96
info
loose-object-idx
pack
15307cff4ea19f61d3bb0b98bc73102e2454352c
c49897f29f9819a0ab6850d7e22443508a1a29d5
c49897f29f9819a0ab6850d7e22443508a1a29d5
commit
fatal: the receiving end does not support this repository's hash algorithm
fatal: the remote end hung up unexpectedly
```

The SHA-256 repository computed **the same SHA-1 commit and tree names** as the native SHA-1 repository: it re-serialises each object the way a SHA-1 repository would and hashes that, recording the pairs in `loose-object-idx`. `cat-file` can already look objects up by their SHA-1 name. The network protocol cannot use them yet, so the push is still refused. This is the foundation for a future where a project migrates to SHA-256 while contributors on SHA-1 keep pushing. It is groundwork, not something to deploy.

Case 7: migrating an existing repository today

There is no in-place conversion. The supported route is to replay history into a new repository, which **gives every commit a new name**:

```
git init -q -b main --object-format=sha256 conv256
(cd app1 && git fast-export --all) | (cd conv256 && git fast-import --quiet)
git -C app1 log --format='%H %s'
git -C conv256 log --format='%H %s'
```

```
aaec54ba980d0b7c3e8f4afe2fd0e72e8fa15d5d one
9e2c3721012ae0d829ab2717b83446230a71ab4fc873830ae5c98b7ee39aa75b one
```

Everything that refers to old names breaks: commit hashes quoted in messages and issues, signed

commits and tags (the signature covered the old bytes), submodule pointers, CI caches, and every clone. Treat it like a history rewrite of the whole project: announce it, freeze pushes, convert, re-sign tags if you rely on signatures, and have everyone re-clone.

Cheat sheet

Task	Command
Which format is this repo?	`git rev-parse --show-object-format`
Which ref backend?	`git rev-parse --show-ref-format`
What does this Git build default to?	`git version --build-options | grep default`
New SHA-256 repo	`git init --object-format=sha256`
Pin the default	`git config --global init.defaultObjectFormat sha1|sha256`
One-off override	`GIT_DEFAULT_HASH=sha256 git init …`
Hash a file for this repo's format	`git hash-object <file>`
Empty-tree name for this repo	`git hash-object -t tree /dev/null`
Convert history (new names)	`git fast-export --all | git -C new fast-import`

Safety

- Changing `extensions.objectFormat` in an existing repository does not convert anything. It just makes the repository unreadable.
- Push and fetch refuse to cross formats, so you cannot corrupt a remote by accident. You *can* strand yourself by creating SHA-256 repositories your forge or CI cannot host.
- A converted repository is a different project as far as names go. Keep the SHA-1 original read-only for as long as anything references its commit IDs.
- `compatObjectFormat` is experimental and Rust-only in 2.56. Do not enable it on repositories other people depend on.

Hands-on checklist

1. Reproduce Case 1 and confirm `sha256sum` of `"blob 6\0hello\n"` matches `git hash-object` in a SHA-256 repository.
2. Run `git version --build-options` on your workstation, CI image, and server. Write down `default-hash` and `default-ref-format` for each.
3. Grep your hooks and CI for `{40}`, `CHAR(40)`, and `4b825dc`. Fix what you find.
4. Add `init.defaultObjectFormat`, `init.defaultRefFormat`, and `init.defaultBranch` to the config you provision, set to what you use today.
5. Create a throwaway SHA-256 repository on your forge (if it allows that) and push to it from CI. That is the real readiness test.

Pitfalls

Pitfall	Symptom
Assuming 40 hex everywhere	Hooks reject valid pushes; parsers truncate names
Hard-coded empty-tree ID	`fatal: bad object 4b825dc…` in a SHA-256 repo
Creating SHA-256 repos before the forge supports them	Push refused or repository unusable on the host
Mixing formats in mirrors or forks	`mismatched algorithms` on fetch
Trusting a preview build's defaults	Repos created with a format you did not intend (Case 5)
Converting without a plan	Broken signatures, dead commit links, confused clones

Sources

- Git — BreakingChanges (Git 3.0: SHA-256 default, reftable, `main`, Rust): https://git-scm.com/docs/BreakingChanges
- Git — hash-function-transition (design, compat object format, translation tables): https://git-scm.com/docs/hash-function-transition
- Git — `git init` (`--object-format`, `--ref-format`, `init.defaultObjectFormat`): https://git-scm.com/docs/git-init
- Git — `git rev-parse` (`--show-object-format`, `--output-object-format`): https://git-scm.com/docs/git-rev-parse
- Git — repository-version / extensions (`extensions.objectFormat`, `extensions.compatObjectFormat`): https://git-scm.com/docs/repository-version and https://git-scm.com/docs/git-config#Documentation/git-config.txt-extensionsobjectFormat
- Git v2.52.0 release notes ("The beginning of SHA1-SHA256 interoperability work"): https://github.com/git/git/blob/master/Documentation/RelNotes/2.52.0.adoc
- Git source, `hash.h` (`GIT_HASH_DEFAULT` under `WITH_BREAKING_CHANGES`) and `t/test-lib.sh`: https://github.com/git/git
- GitHub Blog — Highlights from Git 2.54: https://github.blog/open-source/git/highlights-from-git-2-54/

Packfiles and git log

Packfiles and git log

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Packfiles, compression, and how `git log` works under the hood.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

Internals That Affect Servers

Internals That Affect Servers

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Internals that affect server design (preview of chapter 23, platform architecture).

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

Hands-On: git cat-file

Hands-On: git cat-file

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Explore the object store with `git cat-file`.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

Reflog

Theory

Theory: The Reflog is a Local Journal of Names

Every time a ref in this clone moves, Git can append a line to that ref's log. It is not replicated. It is not a backup of other clones.

Invariants

1. `HEAD` and each branch can have a reflog.
2. Default expire is on the order of 30 days (`gc.reflogExpire`).
3. `HEAD@{1}` is "where HEAD was last time," not "parent commit."
4. Recovering a deleted branch is `git switch -c name <hash-from-reflog>`.

```
main moved C → D → E
reflog  E HEAD@{0}
        D HEAD@{1}
        C HEAD@{2}     (local only)
```

What can break

- Expecting a teammate's reflog to contain your amended commit.
- `gc --prune=now` after expire.

How to read this chapter

How it works, recover a branch, the 30-day window, step-by-step scenarios.

How the Reflog Works

How the Reflog Works

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- How the reference log works.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

Recover Deleted Branches

Recover Deleted Branches

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Recover deleted branches in seconds.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

The 30-Day Safety Window

The 30-Day Safety Window

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- The 30-day safety window.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

Recovery Scenarios

Recovery Scenarios

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Real recovery scenarios, step by step.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

Bisect

Theory

Theory: Bisect is Binary Search on First-Parent History

If a test can label a commit good or bad, `git bisect` finds the first bad commit in O(log N) checkouts.

Invariants

1. You mark an old good and a current bad; Git checks out the midpoint.
2. `git bisect run ./test.sh` automates the oracle.
3. Skip unbuildable commits; they are not evidence.
4. The answer is a commit hash, not a file.

```
G G G G B B B B    (good/bad along a chain)
      ^ first bad
```

What can break

- A flaky test (non-monotonic good/bad).
- Bisecting a merge-heavy history without understanding first-parent.

How to read this chapter

Manual bisect, automate with tests, 2 AM story, lab.

Binary Search for Bugs

Binary Search for Bugs

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Binary search to hunt bugs and regressions.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

Automate Bisect with Tests

Automate Bisect with Tests

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Automate with your test suite.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

2 AM Production Debugging

2 AM Production Debugging

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- 2 AM production debugging example.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

Hands-On: git bisect

Hands-On: git bisect

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Find a regression with `git bisect`.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

Performance and Scale

Theory

Theory: Scale is Reachability and Pack Shape

"Large" is commit count, tree width, blob size, and clone graph — not only disk bytes. Shallow, sparse, and partial clones cut different axes.

Invariants

1. Shallow (`--depth`) cuts **history** length.
2. Sparse checkout cuts **which paths** materialize in the working tree.
3. Partial clone cuts **which blobs** you download until needed (`--filter`).
4. Commit-graph and bitmaps speed `log` and fetch on the server and client.

```
full clone     all commits, trees, blobs
shallow        last N commits
sparse         all objects, few WD paths
partial        delayed blobs
```

What can break

- Shallow clone then needing `blame` across all history.
- LFS-sized files without LFS.

How to read this chapter

What large means, the three clone cuts, `git config` tuning.

Large Repositories

Large Repositories

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- What “large” means and common bottlenecks.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

Shallow, Sparse, and Partial Clones

Shallow, Sparse, and Partial Clones

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Shallow clones, sparse checkout, and partial clones.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

Performance Tuning

Performance Tuning

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Performance tuning with `git config`.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

git maintenance

git maintenance

Git commands that **add** data (`add`, `commit`, `fetch`) optimize for a snappy UI. They do not spend time reshaping the whole object database. `git maintenance` is the operator tool that runs background (or manual) **optimization tasks**: commit-graph, prefetch, loose-object packing, multi-pack-index repacks, and more — without turning every `git status` into a full `gc`.

Mental model

```
foreground Git     writes packs / loose objects / refs  (fast path)


git maintenance run / start

            commit-graph      (faster reachability / log)
            prefetch          (objects ready before real fetch)
            loose-objects     (loose → pack, carefully)
            incremental-repack (multi-pack-index dance)
            pack-refs, …      (ref iteration speed)
```

`git maintenance start` registers the repo and installs a **scheduler** (systemd user timers on modern Linux, else cron; launchctl / schtasks on macOS / Windows). `register` alone updates config; `start` = register + schedule.

Tasks that matter at scale

Task	What it does	Default under `incremental` strategy
`commit-graph`	Incremental commit-graph write + verify; safe beside concurrent Git	hourly
`prefetch`	`git fetch` into `refs/prefetch/` (no tag update; does not move remote-tracking branches)	hourly

Task	What it does	Default under `incremental` strategy
`loose-objects`	Delete loose objects already packed; pack a batch of remaining loose objects	daily
`incremental-repack`	`multi-pack-index expire` then `repack` small packs into a larger one	daily
`pack-refs`	Pack loose refs into a single file	weekly (when enabled by strategy)
`gc`	Full garbage collection / aggressive repack — **expensive**; not on the incremental schedule	manual / other strategies

Prefetch makes the **next** interactive `git fetch` mostly a tip update when objects were already downloaded. Exclude a remote with `remote.<name>.skipFetchAll` if that remote should not be prefetched.

`incremental-repack` uses the **multi-pack-index (MIDX)**: expire unreferenced packs, then repack a batch of small packs and retarget MIDX entries. Default batch size `0` means "try to fold into one pack."

Worked examples

Case 1: One-shot tasks on a lab clone

```
cd /path/to/large-repo

# Inspect whether auto thresholds say work is due
git maintenance is-needed --auto; echo "exit=$?"

# Run specific safe tasks explicitly
git maintenance run --task=commit-graph
git maintenance run --task=loose-objects
git maintenance run --task=incremental-repack

# Optional: see commit-graph files appear
ls -la .git/objects/info/commit-graph 2>/dev/null || ls -la .git/objects/info/commit-graphs/
```

Use `--task` when you want a known order. Without `--task`, Git runs tasks with `maintenance.<task>.enabled=tr` (by default that is mainly `gc` unless you set a strategy).

Case 2: Enable incremental strategy and register

```
cd /path/to/large-repo

git maintenance register
# Sets maintenance.strategy=incremental if unset, adds repo to global maintenance.repo,
# enables recommended schedules, sets maintenance.auto=false in this repo

git config --get maintenance.strategy
git config --get-all --global maintenance.repo | tail -n 5
git config --get maintenance.auto
```

Expected shape: `incremental`, a path entry under `maintenance.repo`, and `maintenance.auto=false` so foreground commands do not also kick auto-maintenance while the scheduler owns the job.

Case 3: `git maintenance start` on Linux

```
cd /path/to/large-repo
git maintenance start --scheduler=auto
# On Linux with user systemd: systemd-timer; otherwise crontab

systemctl --user list-timers 'git-maintenance*' 2>/dev/null || true
# Or: crontab -l | sed -n '/GIT MAINTENANCE/,/END GIT MAINTENANCE/p'
```

Timers run `git for-each-repo --config=maintenance.repo maintenance run --schedule=hourly|daily|w`
Stop with `git maintenance stop` (leaves the repo registered). `unregister` removes the repo from the global list (`--force` if unsure).

Case 4: Prefetch vs a normal fetch

```
# After maintenance has run prefetch on a registered remote:
git show-ref | grep prefetch | head
git fetch origin
# Ideal case: tips move with little or no object transfer
```

Prefetch refs live under `refs/prefetch/` so your `origin/main` does not jump until **you** fetch.

Case 5: Manual vs scheduled, and locks

```
# Manual full gc via maintenance (preferred over bare git gc when using the framework)
git maintenance run --task=gc

# Auto mode: only if thresholds say so (loose object count, pack count, …)
```

```
git maintenance run --auto
```

Each `git maintenance run` takes an **object database lock**. Overlapping hourly runs on huge multi-repo schedules can skip a cycle — shrink task cost or stagger repos if windows exceed an hour. Do not mix bare `git gc` with concurrent `git maintenance run`; prefer `git maintenance run --task=gc`.

Strategy cheat sheet

`maintenance.strategy`	Intent
`none`	Scheduled maintenance runs nothing by default
`gc`	Run the `gc` task
`geometric`	Geometric pack repack + auxiliary cleanup; strong default for **manual** maintenance on large repos (Git 2.50+ docs)
`incremental`	Small non-deleting tasks on a schedule; manual path still uses `gc`

Tune with `maintenance.<task>.enabled` and `maintenance.<task>.schedule` (`hourly` / `daily` / `weekly`). Threshold knobs include `maintenance.commit-graph.auto`, `maintenance.loose-objects.auto`, `maintenance.incremental-repack.auto`.

Safety

- Prefetch contacts remotes on a timer — ensure credentials/agents work non-interactively or exclude remotes.
- `gc` deletes stale data; incremental tasks prefer not to. Know which strategy you enabled before leaving a laptop on a huge monorepo overnight.
- Customizing systemd units: use drop-ins under `~/.config/systemd/user/git-maintenance@.service.d/`; `start` overwrites the main unit files.
- `maintenance.auto=false` after `register` is intentional; re-enable only if you understand double scheduling.

Hands-on checklist

1. On a disposable clone, run `commit-graph` and `incremental-repack` with `--task`.

2. `git maintenance register` and confirm `maintenance.strategy` / `maintenance.repo`.
3. `git maintenance start`; list timers or crontab region.
4. Trigger prefetch; inspect `refs/prefetch/`; run a normal `git fetch` and note transfer size.
5. `git maintenance stop` and `unregister` when the lab is done.

Pitfalls

Symptom	Likely cause	Fix
Hourly tasks sometimes skip	Overlap on object DB lock	Fewer repos per user, cheaper tasks, or longer spacing
Remote-tracking branches “randomly” move	Not prefetch — something else fetching	Prefetch should only touch `refs/prefetch/`
Disk grows after incremental mode	No full `gc`; MIDX keeps more packs briefly	Periodic manual `gc` / geometric strategy if policy allows
Scheduler missing on Linux	No user systemd / lingering	`loginctl enable-linger $USER` or use `--scheduler=crontab`
`unregister` errors	Repo not in list	`git maintenance unregister --force`

Sources

- git-maintenance documentation
- git-multi-pack-index (expire / repack used by `incremental-repack`)

Git Security

Theory

Theory: Signatures Bind a Key to a Hash; History Still Contains Secrets

A signed commit proves a key holder produced that object. It does not encrypt the tree. A secret once hashed is in every clone that fetched it.

Invariants

1. SSH or GPG signing covers the commit/tag object.
2. Forges display verified if the key is uploaded.
3. `git filter-repo` rewrites hashes; everyone must reclone.
4. Pre-commit secret scanning is courtesy; rotation is mandatory.

```
commit hash   signed by key K
forge matches K to account
secret blob   still in pack until rewrite + rotate
```

What can break

- Signing with a key the forge does not know.
- Revert of a secret (blob remains).

How to read this chapter

Sign, secrets, audit trails, scanning, lab to remove a secret.

Signed Commits and Tags

Signed Commits and Tags

After this page you can sign commits and annotated tags with **SSH** (preferred on a Linux desk in 2026) or **OpenPGP**, prove a signature locally, and get a forge **Verified** badge by uploading the matching public key. A signature binds a key to a commit or tag object. It does **not** encrypt the tree, and it does **not** stop a secret that already landed in history.

Desk repos for the labs: `~/desk/desk-api` (chapter 3). Linux is the primary host.

What a signature covers

```
commit / annotated tag object

  payload hashed   signed by key K

  forge matches K to your account → Verified
```

Kind	Signs	Typical tool
Commit	The commit object (tree + parents + author/committer + message)	`git commit -S`
Annotated tag	The tag object (points at a commit + tagger + message)	`git tag -s`
Lightweight tag	Nothing cryptographic — name only	Avoid for releases

Unsigned history is still valid Git. Signing is an authenticity layer you and your forge agree on.

SSH signing (boring default)

OpenSSH 8.2+ can sign Git objects. Git 2.34+ speaks `gpg.format=ssh`. You reuse the same key you already use for `git@` remotes.

One-time setup

```
# Prefer an Ed25519 key dedicated to signing if your org splits roles
ls -l ~/.ssh/id_ed25519.pub

git config --global gpg.format ssh
git config --global user.signingKey ~/.ssh/id_ed25519.pub
git config --global commit.gpgsign true
git config --global tag.gpgSign true
```

`user.signingKey` for SSH is the **public** key path (or the literal `key::ssh-ed25519 AAAA…` form). Git asks `ssh-keygen -Y sign` under the hood; your agent holds the private key.

Upload the **same** public key to the forge as a **signing** key (GitHub: Settings → SSH and GPG keys → New SSH key → Key type **Signing Key**). Authentication keys and signing keys can be the same material, but the forge must list it for signing or the badge stays unsigned.

Sign a commit and prove it locally

```
cd ~/desk/desk-api
git switch -c lab/signed-commits
echo "signed-lab $(date -Iseconds)" >> NOTES.md
git add NOTES.md
git commit -m "lab: demonstrate SSH-signed commit"
# -S is implied when commit.gpgsign=true; pass -S explicitly if you left that unset

git log -1 --show-signature
git verify-commit HEAD
```

Expected shape (wording varies by Git/OpenSSH version):

```
Good "git" signature for … with ED25519 key SHA256:…
```

If verification says the key is unknown, configure an allowed-signers file (next section) or rely on the forge UI after push.

Allowed signers (local trust)

SSH verification needs a map from principals to public keys — the allowed-signers file:

```
mkdir -p ~/.config/git
cat >> ~/.config/git/allowed_signers <<'EOF'
you@example.com namespaces="git" ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAA… comment
EOF
git config --global gpg.ssh.allowedSignersFile ~/.config/git/allowed_signers
```

```
git verify-commit HEAD
```

Principals should match the email (or forge identity) you care about. `namespaces="git"` limits the key to Git signatures. Without this file, `git verify-commit` may report a good cryptographic signature with an untrusted principal — same idea as OpenPGP "Good signature" with unknown trust.

Sign an annotated release tag

```
git tag -s v0.0.0-lab -m "lab signed tag"
git verify-tag v0.0.0-lab
git show --show-signature v0.0.0-lab
```

Lightweight tags (`git tag v0.0.0-lab` without `-s` / `-a`) carry no signature. Release process: annotated + signed.

OpenPGP signing (when policy requires it)

Some orgs still mandate OpenPGP. Install GnuPG, generate or import a key, then:

```
gpg --list-secret-keys --keyid-format=long
# Copy the signing key fingerprint (40 hex), not a short ID

git config --global gpg.format openpgp   # or unset; openpgp is the historical default
git config --global user.signingKey REPLACE_WITH_FINGERPRINT
git config --global commit.gpgsign true

git commit -S -m "lab: OpenPGP-signed commit"
git log -1 --show-signature
```

Export the public key and upload it to the forge as a GPG key. Prefer fingerprint match over short key IDs. Keep `~/.gnupg` mode `700`; never commit secret key material.

You can keep SSH for auth remotes and OpenPGP only for signing by setting `gpg.format openpgp` and `user.signingKey` to the fingerprint while SSH continues to handle `git@` URLs.

Selective signing and overrides

Goal	Command / config
Always sign commits	`commit.gpgsign=true`
Always sign annotated tags	`tag.gpgSign=true`
One unsigned commit	`git commit --no-gpg-sign`

Goal	Command / config
One signed commit when defaults are off	`git commit -S`
Repo-only policy	omit `--global`; set the same keys in this repo's `.git/config`

CI bots that merge with a forge token often produce **unsigned** merge commits unless the bot has its own signing key. Branch protection "require signed commits" applies to the commits you push; understand whether merges rewrite history or add unsigned merge nodes.

Push and forge Verified

```
git push -u origin lab/signed-commits
git push origin v0.0.0-lab
```

On the forge, open the commit: a **Verified** badge means the signature matched a signing key attached to the author account. No badge usually means: wrong key uploaded, auth-only SSH key not marked for signing, or author email does not match the forge account.

```
# Inspect what you actually pushed
git cat-file -p HEAD | head -20
# Look for gpgsig / signed tag blocks in the object
```

Safety

- Signing proves **who held key K**, not that the diff is safe to run. Still review patches.
- A compromised laptop with an unlocked agent can sign forever — use short agent lifetimes, hardware keys, or confirm prompts where your stack supports them.
- Rewriting signed history (`rebase`, `filter-repo`) drops or invalidates old signatures; re-sign deliberately after rewrites.
- Do not paste private keys or `ssh-keygen -Y sign` intermediate files into tickets.
- Tag signing and commit signing are separate habits. Releases need both a signed tag and a review of the commit it points at.

Hands-on checklist

1. Configure SSH signing (`gpg.format`, `user.signingKey`, `commit.gpgsign`).
2. Create `lab/signed-commits`, make one signed commit, run `git verify-commit HEAD`.
3. Add an allowed-signers entry for your principal; re-verify.
4. Create `v0.0.0-lab` with `git tag -s`; run `git verify-tag`.
5. Push branch + tag; confirm Verified on the forge for both.
6. Optional: make one `--no-gpg-sign` commit and contrast the forge UI.

Pitfalls

Symptom	Likely cause	Fix
cannot sign / ssh signing error	Agent empty or wrong key path	`ssh-add -l`; fix `user.signingKey`
Good signature, untrusted	No allowed-signers / trust	Configure `gpg.ssh.allowedSignersFile` or forge upload
Forge shows Unverified	Key not uploaded as **signing** key	Add signing key; match author email
`tag.gpgSign` ignored	Lightweight tag	Use `git tag -s` / annotated tags
Works locally, CI unsigned	Bot has no signing key	Accept for merges or equip the bot

Sources

- git-config — gpg.format, user.signingKey, commit.gpgsign, gpg.ssh.allowedSignersFile
- git-commit -S / –gpg-sign
- git-tag -s
- OpenSSH `ssh-keygen -Y sign` / `-Y verify` (man pages on your Ubuntu host)

Secrets in History

Secrets in History

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Secrets in history: removal and prevention.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

Audit Trails

Audit Trails

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Audit trails at the Git layer.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

Security Scanning

Security Scanning

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Security scanning: pre-commit and CI integration.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

Hands-On: Remove a Secret

Hands-On: Remove a Secret

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Remove a committed secret.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

Troubleshooting

Theory

Theory: fsck Walks the Graph; Repair is Restore Reachability

Corruption is missing or malformed objects. Dangling objects are normal (amended commits). Repair means getting a reachable graph again — often from another clone.

Invariants

1. `git fsck` reports missing, unreachable, and malformed objects.
2. Another clone of the same DAG is the best object source.
3. Reflog may still name "lost" commits locally.
4. Do not `rm -rf .git` until you have a copy of objects you need.

```
tip → commit → tree → blob    all hashes must exist
missing blob  = unreadable file at that path in that commit
```

What can break

- Running random `rm` in `.git/objects`.
- Assuming `fsck` dangling = disaster.

How to read this chapter

fsck, error patterns, procedures, corruption lab.

fsck and Repair

fsck and Repair

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Corrupted repositories: `git fsck` and repair.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

Common Error Patterns

Common Error Patterns

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Common error patterns: causes and fixes.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

Recovery Procedures

Recovery Procedures

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Recovery procedures.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

Hands-On: Corrupted Repo

Hands-On: Corrupted Repo

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Simulate a corrupted repo and recover.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

Hosting Platforms

GitHub Monoculture

Theory

Theory: Default is Gravity; Lock-in is Extra Objects Outside Git

GitHub became default by network effects, not by owning the Git protocol. Lock-in lives in identity, Actions YAML, Packages, issue graphs, and marketplace apps — data Git does not clone.

Invariants

1. A Git remote is replaceable; a social graph is not.
2. CI, packages, and issues are forge databases.
3. Multi-homing (two remotes) preserves Git objects only.
4. Regulation and data residency constrain *where* a forge may run.

```
Git objects     portable
PRs, Actions, Packages, stars    forge-specific
```

What can break

- Calling `git clone` a full backup of a GitHub org.

How to read this chapter

How GitHub won, lock-in, geopolitics, what "beyond" means, comparison rubric.

How GitHub Became Default

How GitHub Became Default

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- How GitHub became the default.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

Risks of Platform Lock-in

Risks of Platform Lock-in

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Risks of platform lock-in.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

Regulatory and Geopolitical

Regulatory and Geopolitical

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- The regulatory and geopolitical dimension.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

What Beyond GitHub Means

What Beyond GitHub Means

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- What “beyond GitHub” actually means.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

A Framework for This Part

A Framework for This Part

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- A framework for this part.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

Platform Architecture

Theory

Theory: A Forge is Git Plus Metadata, Identity, and Jobs

`git init --bare` is hosting. A platform adds a database, authn/z, HTTP/SSH frontends, object storage, and workers. Operational complexity is the hidden product tax.

Invariants

1. Smart HTTP and SSH both speak the Git protocol; they differ in auth.
2. Repo bytes may live on local disk, NFS, or S3-style object stores.
3. Webhooks are the universal outbound glue.
4. Monolith vs microservices is a scaling and failure-domain choice.

```
clients --SSH/HTTP--> frontend --→ git storage
                          \--→ metadata DB
                          \--→ jobs / CI / hooks
```

What can break

- Treating GitLab Omnibus RAM as "just Git."

How to read this chapter

Git vs hosting, components, protocols, storage, identity box, extensibility, styles, cost.

Git Alone vs Git Hosting

Git Alone vs Git Hosting

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Git alone vs Git hosting.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

Core Platform Components

Core Platform Components

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Core platform components.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

The Protocol Stack

The Protocol Stack

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- SSH, HTTP, and Git smart protocols.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

Storage Architectures

Storage Architectures

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Filesystem, distributed storage, and object stores.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

Authn, Authz, and Identity

Authn, Authz, and Identity

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Authentication, authorization, and identity as a component.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

Extensibility

Extensibility

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Webhooks, APIs, plugins.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

Monolith vs Microservices

Monolith vs Microservices

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Architectural styles: monolith vs microservices.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

Operational Complexity

Operational Complexity

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Operational complexity: the hidden cost.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

GitLab

Theory

Theory: GitLab Sells a Full SDLC on a (Mostly) Monolith

GitLab's theory is one application for Git, CI, registry, and security. Omnibus is that app plus dependencies as a package. Kubernetes is the same app shredded into services.

Invariants

1. FOSS vs Premium is feature gating on the same codebase.
2. Runners are the horizontal scale knob for CI, not the Rails app.
3. A small team can run CE; HA is a different product shape.

```
Omnibus: one node, many processes
Geo / K8s: DB, Gitaly, Sidekiq, registry, runners
```

What can break

- GitLab CE on a 2 GB VPS with CI enabled.

How to read this chapter

History, architecture, features, ops/license, lab, when it fits.

History and Philosophy

History and Philosophy

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- History and philosophy.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

GitLab Architecture

GitLab Architecture

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Omnibus, monolith, Kubernetes.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

GitLab Feature Set

GitLab Feature Set

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- CI/CD, registry, security, DevOps.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

Operations and Licensing

Operations and Licensing

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Operational requirements, scaling, FOSS vs Premium.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

Hands-On: GitLab

Hands-On: GitLab

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- GitLab CE compose or gitlab.com.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

When GitLab Is the Right Choice

When GitLab Is the Right Choice

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- When GitLab is the right choice.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

Gitea and Forgejo

Theory

Theory: A Single Binary is an Architectural Statement

Gitea/Forgejo keep Git hosting in one Go process, SQLite-capable. That is a complexity budget. The Forgejo fork is a governance and license theory, not a protocol fork.

Invariants

1. SQLite + local disk is enough until it is not (locking, size).
2. Actions-compatible CI is optional, not the product core. The runner is a second process.
3. Codeberg is Forgejo-as-a-service with community ops (chapter 29).
4. The Forgejo fork did not change the Git protocol. A remote is still SSH or HTTP.

```
one binary    git repos on disk
               SQLite/Postgres
               optional runners
```

What can break

- Expecting GitLab-class HA from a hobby compose file.
- Treating a current Gitea database as a drop-in for current Forgejo. The hard fork ended that promise.

How to read this chapter

Origins, fork, comparison, scale, two labs, when it fits.

Origins and Philosophy

Origins and Philosophy

After this page you can say what problem a single-binary forge is built to solve, and which project solved it first.

Two forks, one shape

Gogs was a lightweight Git server in Go. Development sat behind one maintainer. In 2016 contributors forked it and called the result **Gitea**: the same idea, with a group that could review and release.

The idea is small on purpose.

```
one Go process
     Git repositories as bare repos on disk
     issues, pull requests, users  →  one database
     HTTP on port 3000, and SSH for the Git remote
```

Default database is **SQLite**, a file next to the repositories. Postgres and MySQL are configuration, not a requirement. There is no separate Rails app, no Gitaly, no search cluster. Chapter 23 named those boxes. This product leaves most of them empty until you opt in.

What you get on the first afternoon

A Gitea or Forgejo instance gives a small team:

- repositories, with SSH and HTTP remotes (chapter 11)
- issues and pull requests
- a web UI to browse files and diffs
- organizations, teams, and repository permissions
- webhooks, when you want an outside CI to notice a push (chapter 32)

Packages, Actions, and a container registry exist. They are extra processes and extra disks. The core stays the binary plus the data directory.

What this shape refuses to be

GitLab's theory is one application for the whole lifecycle (chapter 24): Git, CI, registry, and security scanning in one product. Gitea's theory is the opposite budget. A desk, a lab, or a small company can run it on a machine that would struggle to boot GitLab Omnibus.

That budget is the point. The moment the database, the Git files, and CI runners all need to scale independently, you have left the afternoon install and entered chapter 36. The binary does not stop you. It also does not hide the work.

The next page is why Forgejo exists as a second project with this same shape.

The Forgejo Fork

The Forgejo Fork

After this page you can explain why Forgejo split from Gitea, and what that split changes for an operator.

Who holds the name

Gitea's code was community-run. In 2022 a for-profit company, Gitea Ltd, took the trademark and the project's direction. An open letter from contributors went unanswered. In October 2022 Codeberg and other operators published **Forgejo** as a fork.

Codeberg e.V., a non-profit in Berlin, holds the Forgejo domains. Forgejo has not registered a trademark. The public forge at `codeberg.org` runs Forgejo. Chapter 29 is that hosted instance. This chapter is the software you run yourself.

```
Gogs (single maintainer)
     Gitea, 2016          Gitea Ltd, MIT, company edition
           Forgejo, Oct 2022
              soft fork through 2023
              hard fork, early 2024     Codeberg e.V., GPL-3.0-or-later since v9
```

Soft fork, then hard fork

Through 2023 Forgejo was a **soft fork**: it tracked Gitea and stayed a drop-in. Changing the download URL was enough to upgrade a Gitea server onto Forgejo.

On 15 February 2024 the project became a **hard fork**. Stable Forgejo stopped promising to contain all of Gitea. The codebases diverge on purpose, so each side can change without rebasing onto the other. A Gitea database from the 1.22 era is the last generation where people treated the switch as a binary swap. Later pairs need the Forgejo upgrade guide for those two versions. Do not point a new Forgejo binary at a production Gitea data directory and hope.

What actually changed

The Git protocol did not change. Clients, remotes, and hooks from Parts I–III work against either UI.

What changed is governance and the license:

	Gitea	Forgejo
Steward	Gitea Ltd	Codeberg e.V.
License	MIT	MIT through v8; **GPL-3.0-or-later from v9**
Contributor terms	copyright assignment to the company	no copyright assignment
Paid edition	Enterprise	none
Developed on	GitHub	Forgejo

GPL-3.0-or-later means a modified Forgejo you ship to others comes with source under the same terms. MIT does not require that. Gitea can legally ship a closed Enterprise build on the same code. Forgejo's license is how the project refuses that path.

Day to day, issues, pull requests, and the settings screens still feel like the same product. Actions exist on both (Forgejo Actions and Gitea Actions), with a runner you install separately. Federation between Forgejo instances is a Forgejo project and is not something to design a company on until you have tried it on the version you run.

The next page puts Forgejo, Gitea, and GitLab in one table. The lab pins **Forgejo 15** LTS, not whatever tag your registry calls latest.

Gitea vs Forgejo vs GitLab

Gitea vs Forgejo vs GitLab

After this page you can choose among the three from the constraint you actually have, not from a feature checklist.

Same afternoon, different contracts

Gitea and Forgejo are one architectural family. GitLab is a different product that also hosts Git. Versions below are the ones this book checked in September 2026.

	Forgejo 15 LTS	Gitea 1.27	GitLab CE
Steward	Codeberg e.V.	Gitea Ltd	GitLab Inc.
License of the server you run	GPL-3.0-or-later	MIT	MIT (CE). Paid tiers are a separate license
Process model	one Go binary	one Go binary	Omnibus: many processes (chapter 24)
Default database	SQLite file	SQLite file	Postgres
CI	Forgejo Actions, external runner	Gitea Actions, external runner	CI in the product
Paid tier	none	Enterprise	Premium and above
Public hosted twin	Codeberg	Gitea.com	GitLab.com
Support window	15.0 LTS through 15 July 2027. 16.0 is the short cycle (through 29 October 2026)	upstream releases	your own upgrade calendar

Forgejo 15.0.9 and 16.0.5 both shipped on 17 September 2026. The lab uses the **15** image tag so a reader is on the LTS line. Track 16 only if you intend to move every quarter.

What is not a difference

Push, fetch, branches, and review comments are Git plus a database row. A developer who learned chapters 8–11 does not relearn them here. The UI names a pull request; Gerrit (chapter 27) names a change. Under both, the bytes are still a branch on a remote.

Webhooks out of either lightweight forge are the same idea as GitLab's: HTTP to your CI when a ref moves. Chapter 32 is that integration. This chapter does not stand up a runner.

How to read the license row

Pick Forgejo when the operator must be able to show that the forge itself stays free software, with a non-profit holding the name and no Enterprise fork of the same tree.

Pick Gitea when the MIT terms are a requirement (you need to ship a closed product derived from the forge) or you are already paying for Gitea Enterprise and the company build is the point.

Pick GitLab when the requirement is CI, a container registry, and security scanning inside one installer, and you can afford the machine that installer expects. A 2 GB lab VM is a Forgejo conversation. It is a bad GitLab conversation.

The next page is how the lightweight shape is deployed, and where SQLite stops being the right database.

Deployment and Scaling

Deployment and Scaling

After this page you can sketch a Forgejo or Gitea layout and name the first limit you will hit.

One directory is the whole server

The container's data directory holds the bare repositories, the database, and the config. On the standard image that directory is `/data`. SQLite, when you leave the database unset, is a file in that tree (`gitea.db`, including on Forgejo: the filename stayed). Config is `app.ini` under the same tree.

```
./data/                       (bind mount → /data)
  git/repositories/           bare repos
  gitea/gitea.db              SQLite, until you move it
  gitea/conf/app.ini          generated on first start
```

Environment variables of the form `FORGEJO__section__KEY` write `app.ini` for you. Gitea uses the same pattern with the prefix `GITEA__`. A value already stored in `app.ini` is not cleared by unsetting the variable. Edit the file, or start from an empty data directory.

Two images ship:

Image	Data mount	SSH inside the container
`codeberg.org/forgejo/forgejo:15`	`/data`	port 22
`codeberg.org/forgejo/forgejo:15-rootless`	`/var/lib/gitea`	port 2222

The lab uses the first. Rootless is the same product with a different mount and a different internal SSH port. Mirror `data.forgejo.org` when `codeberg.org` is not reachable from the build machine.

Where it stops being small

SQLite has one writer. A desk, a class, or a company of tens of people is the design center. A busy public forge is not. The move is mechanical:

1. Keep Git bytes on local disk. NFS for the repository directory is a corruption risk unless you already know that filesystem's locking story. Chapter 36 is the storage chapter.

2. Move the database to Postgres when SQLite lock time shows up in the logs. Set `FORGEJO__database__DB_TYPE=postgres` and a host. Do this before the file is large, and take a backup first (chapter 37).
3. Put LFS and package blobs on object storage when the data disk fills with artifacts, not when the Git packs do.
4. Scale CI by adding runners, not by cloning the Forgejo process. Actions are optional. The server is still one writer of its own database.

There is no supported active-active pair of Forgejo nodes over one SQLite file. High availability here means backups, a second disk, and a practiced restore. GitLab's multi-node shape is a different product.

Upgrades

Pin an image tag (`:15` or `:15.0`), not `latest`. Moving from major N to N+1 is a manual upgrade with a backup of `/data` in front of it. Patch releases inside 15.0 replace the container and keep the directory.

Gitea's image is `docker.io/gitea/gitea:1.27`. Same directory layout, same port story, `GITEA__` environment names. Do not run both compose projects against one data directory.

The hands-on page brings Forgejo up, pushes `desk-api`, then shows the Gitea file as the same lab with the image line changed.

Hands-On: Gitea and Forgejo

Hands-On: Gitea and Forgejo

Bring up Forgejo 15 on Linux, push `desk-api`, tear it down, then read the Gitea file as the same lab with one image line changed.

Examples were checked against the Forgejo 15 LTS docs and Gitea 1.27. You need Docker, or Podman with `podman compose`. The commands below say `docker compose`. On Podman, substitute `podman compose`. Do not point this lab at a Forgejo you already use.

Forgejo

Host port 3000 is HTTP. Host port 2222 is Git SSH, so the lab does not take the machine's own sshd on port 22. If either host port is already taken, stop. Another forge is probably there.

```
mkdir -p ~/desk/forgejo-lab
cd ~/desk/forgejo-lab
mkdir -p data
cat > compose.yml <<'EOF'
services:
  server:
    image: codeberg.org/forgejo/forgejo:15
    environment:
      USER_UID: "1000"
      USER_GID: "1000"
      FORGEJO__server__DOMAIN: localhost
      FORGEJO__server__ROOT_URL: http://localhost:3000/
      FORGEJO__server__SSH_PORT: "2222"
      FORGEJO__server__HTTP_PORT: "3000"
    restart: unless-stopped
    volumes:
      - ./data:/data
    ports:
      - "3000:3000"
      - "2222:22"
EOF
docker compose up -d
docker compose ps
```

`USER_UID` should be your numeric id (`id -u`) if 1000 is not you, or the data directory will be owned by the wrong account. Wait until the HTTP port answers:

```
curl -fsS -o /dev/null -w '%{http_code}\n' http://127.0.0.1:3000/
```

A 200 or a 303 is enough. Open `http://localhost:3000/` in a browser. The installation page asks for the administrator. Create that account. Leave the database on SQLite. Submit, then sign in.

Create a repository named `desk-api`. Empty is fine. The UI shows two clone URLs. The SSH one should contain `:2222`. If it says port 22, `SSH_PORT` did not land in `app.ini` and the compose environment block needs another look.

Push desk-api

`~/desk/desk-api` is the project from chapter 3. Add this forge as a second remote (chapter 11). Use the HTTP URL from the UI if you have not loaded an SSH key into the forge yet.

```
cd ~/desk/desk-api
git remote add forgejo http://127.0.0.1:3000/ADMIN/desk-api.git
git push -u forgejo main
```

Replace `ADMIN` with the username you just created. The password is the one from the installation page. After the push, the file list in the browser matches `git log --oneline` locally.

SSH, once the UI has your public key:

```
git remote set-url forgejo ssh://git@127.0.0.1:2222/ADMIN/desk-api.git
```

Tear it down

```
cd ~/desk/forgejo-lab
docker compose down
```

`./data` is still on disk. `docker compose down` does not delete that directory. Delete `~/desk/forgejo-lab` when you are finished with the lab. Leave `~/desk/desk-api` in place, and remove the remote if you do not want it:

```
cd ~/desk/desk-api
git remote remove forgejo
```

The Gitea file

Do not start this while the Forgejo container is still bound to 3000 and 2222. Same lab, other project:

```
mkdir -p ~/desk/gitea-lab/data
cd ~/desk/gitea-lab
cat > compose.yml <<'EOF'
services:
  server:
    image: docker.io/gitea/gitea:1.27
    environment:
      USER_UID: "1000"
      USER_GID: "1000"
      GITEA__server__DOMAIN: localhost
      GITEA__server__ROOT_URL: http://localhost:3000/
      GITEA__server__SSH_PORT: "2222"
      GITEA__server__HTTP_PORT: "3000"
    restart: unless-stopped
    volumes:
      - ./data:/data
    ports:
      - "3000:3000"
      - "2222:22"
EOF
```

`docker compose up -d`, the same installation page, a repository, a push, `docker compose down`. The screen is Gitea's. The Git remote is still just a URL.

Done when

- Forgejo's UI showed `desk-api` after `git push`
- the SSH clone URL contained port 2222
- both compose projects are down, and `~/desk/desk-api` still has its history
- you can point at the one line that differs between the two files (the image, and `FORGEJO__` versus `GITEA__`)

When Gitea or Forgejo Fits

When Gitea or Forgejo Fits

After this page you can decide whether this shape is the forge for the job in front of you.

Choose it when

The requirement is Git hosting for a desk, a course, or a company that can name every user. You want issues and pull requests, you can live with CI as a webhook or a separate runner, and you would rather administer one data directory than a platform of services.

Forgejo is the default in this book for that job:

- the steward is a non-profit, and the license from v9 on is GPL-3.0-or-later
- 15.0 is a long-term line (security support through 15 July 2027)
- Codeberg is the same software, operated by someone else, when you do not want a server (chapter 29)

Gitea is the right lightweight forge when MIT is the constraint, or when a support contract with Gitea Ltd is the constraint. The operator experience is the sibling of Forgejo, not a different architecture.

Run it yourself when the repositories must stay on a machine you control. Use Codeberg when the code can live on a public community forge and you only need an account.

Choose something else when

- You need merge trains, a built-in registry, dependency scanning, and GitLab CI on day one. That is chapter 24, and it is a larger machine.
- Review must be a stack of patchsets with a Change-Id. That is Gerrit (chapter 27), not a pull-request UI with a different coat of paint.
- The company standard is already GitHub or GitLab.com and the lock-in discussion in chapter 22 has been accepted. A second forge "for practice" is this lab. A second forge "for production beside the real one" is two metadata stores and one confused team.
- You expect two nodes to share one SQLite file and call it high availability. Stop at the previous page and read chapter 37 before you draw that diagram.

A decision you can write down

```
small team, self-hosted, free-software forge  →  Forgejo 15 LTS
small team, MIT or a Gitea contract           →  Gitea 1.27
hosted community, no server to patch          →  Codeberg
CI and registry inside the installer          →  GitLab
```

Whichever line you pick, the developer workflow does not restart. `desk-api` is still a repository, the remote is still a URL, and force-push rules from chapter 11 still apply on `main`. Part V is what changes when you are the operator of that URL for other people: identity, branch protection, and backups. The binary will not do those for you.

Bitbucket

Theory

Theory: Bitbucket is Git Hosting Inside an Issue Database

Atlassian's theory is that code hosting is a satellite of Jira. Cloud vs Data Center is who operates the satellite.

Invariants

1. Jira keys in branch names and commits are the integration surface.
2. Data Center is the self-host shape; Cloud is the SaaS shape.
3. Lock-in is the Atlassian user directory and issue graph.

```
Jira  <  links  >  Bitbucket repos
         same org, same identity
```

What can break

- Evaluating Bitbucket without counting Jira.

How to read this chapter

Cloud vs DC, architecture, Jira, enterprise features, comparison, lab, when it fits.

Cloud vs Data Center

Cloud vs Data Center

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Bitbucket Cloud vs Bitbucket Data Center.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

Bitbucket Architecture

Bitbucket Architecture

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Architecture and deployment models.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

Jira and Atlassian

Jira and Atlassian

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Jira and the Atlassian ecosystem.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

Enterprise Features

Enterprise Features

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Branch permissions, Code Insights, audit logs.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

Bitbucket Compared

Bitbucket Compared

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Comparison with GitLab and GitHub.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

Hands-On: Bitbucket Cloud

Hands-On: Bitbucket Cloud

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Bitbucket Cloud lab.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

When Bitbucket Fits

When Bitbucket Fits

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- When Bitbucket fits.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

Gerrit

Theory

Theory: The Change is the Object, Not the Branch

Gerrit stores a **change** with a Change-Id, patch sets, and an approval predicate. Branches are an output of submit, not the collaboration primitive.

Invariants

1. Each push of the same Change-Id is a new patch set (new Git commits).
2. Submit is a server-side integrate (merge/rebase/cherry-pick) after labels.
3. Android/Chromium scale is why this model exists.

```
refs/for/main  →  change 123 patch set 3
labels  Code-Review+2  Verified+1  →  submit  →  main
```

What can break

- Treating Gerrit like GitHub PRs (one branch per feature forever).

How to read this chapter

Model, Change-Ids, CI, ops, practice, lab, when it fits.

What Gerrit Is

What Gerrit Is

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- The review-centric model.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

Change-Ids, Drafts, Approvals

Change-Ids, Drafts, Approvals

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Change-Ids, drafts, and approval chains.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

Gerrit and CI

Gerrit and CI

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Integration with Jenkins, CI, and other tools.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

Gerrit Operations

Gerrit Operations

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Performance and complexity.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

Gerrit in Practice

Gerrit in Practice

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Android, Kubernetes, and other major projects.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

Hands-On: Gerrit

Hands-On: Gerrit

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Gerrit compose — one change, one review.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

When Gerrit Is the Right Choice

When Gerrit Is the Right Choice

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- When Gerrit is the right choice.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

SourceHut

Theory

Theory: Tools in a UNIX Pipeline, Not a Platform Product

SourceHut's theory is that email, git, CI, and tickets can be separate programs with glue. The mailing list is a first-class review transport.

Invariants

1. `git send-email` / patches are the review objects.
2. Each sr.ht service is independently hostable.
3. Hook scripts replace a marketplace.

```
git.sr.ht  --patch-->  lists.sr.ht  --job-->  builds.sr.ht
```

What can break

- Expecting in-browser PR gadgets as the default.

How to read this chapter

Philosophy, suite, architecture, comparison, lab, when it fits.

SourceHut Philosophy

SourceHut Philosophy

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Tools, not platforms.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

The sr.ht Suite

The sr.ht Suite

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Git, issues, mailing lists, CI.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

SourceHut Architecture

SourceHut Architecture

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Python, mail-driven workflows, hook scripts.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

SourceHut Compared

SourceHut Compared

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Comparison with feature-rich platforms.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

Hands-On: SourceHut

Hands-On: SourceHut

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- sourcehut.org or a documented subset self-host.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

When SourceHut Makes Sense

When SourceHut Makes Sense

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- When SourceHut makes sense.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

Niche Platforms

Theory

Theory: Niche is a Different Default, Not a Toy

Codeberg, Gogs, GitBucket, Phorge, GHES, and git-without-a-UI occupy corners: community ops, ancestry, JVM, review heritage, on-prem GitHub, or no web forge at all.

Invariants

1. GHES is still GitHub lock-in, on your power bill.
2. Gogs is the ancestor story for Gitea.
3. A forge-less host (gitolite, Soft Serve) is still Git hosting.

```
UI-heavy  GitLab … Gitea … cgit/gitolite  UI-minimal
```

What can break

- Picking a niche forge for features it does not claim.

How to read this chapter

Codeberg, GitBucket, Gogs, Phorge, GHES, summary table.

Codeberg

Codeberg

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Codeberg and community-run instances.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

GitBucket

GitBucket

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- GitBucket and GitHub-compatible alternatives.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

Gogs

Gogs

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Gogs: the ancestor of Gitea.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

Phorge and Arcanist

Phorge and Arcanist

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Phabricator / Phorge and Arcanist.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

GitHub Enterprise Server

GitHub Enterprise Server

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Self-hosted GitHub Enterprise Server.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

Summary of Niche Platforms

Summary of Niche Platforms

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Summary of niche platforms.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

Operating a Platform

Identity

Theory

Theory: Authn is Who; Authz is What a Ref May Do

A forge maps credentials to a principal, then to permissions on repos, orgs, and refs. SSH keys, OAuth, SAML, and LDAP are different *proofs* of the same principal problem.

Invariants

1. Password/SSH/OAuth/SAML all produce a session or git credential.
2. RBAC is groups × resources × verbs (clone, push, admin, force).
3. Fine-grained tokens are more blast-radius-aware than a single password.
4. MFA protects the *account*, not a stolen deploy key unless you require it.

```
proof (key/token/SAML) → principal → role → refs + APIs
```

What can break

- Deploy keys with write access in CI forever.

How to read this chapter

Methods, LDAP, RBAC, keys at scale, MFA.

Password, SSH, OAuth, SAML

Password, SSH, OAuth, SAML

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Password, SSH keys, OAuth, SAML.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

LDAP and Directory Services

LDAP and Directory Services

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- LDAP, Active Directory, and enterprise identity.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

RBAC and Fine-Grained Permissions

RBAC and Fine-Grained Permissions

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- RBAC and fine-grained permissions.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

SSH Keys at Scale

SSH Keys at Scale

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- SSH key management at scale.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

MFA and Hardening

MFA and Hardening

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Multi-factor authentication and hardening.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

Governance

Theory

Theory: Governance is a Server-Side Predicate on Refs

Branch protection, CODEOWNERS, and required checks are hooks and config that reject a push or a merge button. The Git objects may be fine; the *name movement* is denied.

Invariants

1. Developer workflow (ch 15) is advice; this chapter is enforcement.
2. CODEOWNERS is path → reviewer, evaluated on the proposed tree.
3. Monorepo governance is path-scoped; polyrepo is repo-scoped.

```
push/merge request
  → protected branch?
  → required signatures/checks/owners?
  → accept or reject
```

What can break

- Required reviews with no CODEOWNERS on a 200-person monorepo.

How to read this chapter

Enforcement, protection, mono/poly operator view, automation.

Enforcement Implications

Enforcement Implications

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Branching models revisited: enforcement implications.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

Protecting Branches

Protecting Branches

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Protecting branches; code owners and review requirements.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

Monorepo vs Polyrepo (Operator)

Monorepo vs Polyrepo (Operator)

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Monorepo vs polyrepo trade-offs (operator view).

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

Governance Automation

Governance Automation

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Conventional commits, structured history, and governance automation.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

CI/CD

Theory

Theory: CI is a Function from Git Events to Jobs

A webhook or native parser turns a ref movement into a graph of jobs. Runners are the computers. GitOps later inverts this: the cluster pulls desired state from Git.

Invariants

1. Webhooks are the universal outbound event.
2. Native CI (Actions, GitLab CI, Gitea Actions) stores YAML in Git.
3. External CI (Jenkins, Drone) stores jobs elsewhere, triggered by Git.
4. Runners expand horizontally; the control plane does not always.

```
push/PR → event → pipeline YAML → queue → runner → status back to forge
```

What can break

- Untrusted PRs on privileged runners (injection).

How to read this chapter

Webhooks, native, external, runners, GitOps preview.

Webhooks

Webhooks

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Webhooks: the universal glue.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

Native CI

Native CI

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- GitLab CI, GitHub Actions, Gitea Actions.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

External CI

External CI

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Jenkins, Drone, Cirrus, Buildkite.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

Pipelines and Runners

Pipelines and Runners

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Pipeline architecture, runners, executors, and build infrastructure.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

GitOps Preview

GitOps Preview

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Git as the source of truth (full treatment in Part VII).

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

Packages

Theory

Theory: A Registry is an Object Store with Language Semantics

Container images and language packages are not Git blobs. They need retention, GC, and authn that match the forge's principals.

Invariants

1. A digest (sha256) is the real name of an image; a tag is a moving ref.
2. Retention is a cost and a supply-chain control.
3. CI should push by digest; humans talk in tags.

```
git tag v1.2.0          oci image registry.example/app@sha256:…
both are names over content-addressed bytes
```

What can break

- Immutable tags that were mutated (`latest`).

How to read this chapter

Registries, language managers, retention, self-host architecture.

Container Registries

Container Registries

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- GitLab Registry, Harbor, and alternatives.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

Language Package Managers

Language Package Managers

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Language-specific package managers.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

Retention and CI Integration

Retention and CI Integration

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Artifact storage, retention policies, and CI integration.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

Self-Hosted Registry Architecture

Self-Hosted Registry Architecture

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Self-hosted registry architecture.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

Security

Theory

Theory: Supply Chain is Trust in What You Did Not Write

Git signatures, SBOMs, dependency scans, and secret detection are layers on *provenance*. A platform hardens the perimeter; it cannot see an unsigned `curl | bash` in a Dockerfile unless you look.

Invariants

1. Verify on the server what ch 20 taught on the client.
2. SBOM is a graph of what you shipped, not a scanner by itself.
3. A supply-chain incident is usually a trusted path (CI, dep, maintainer).

```
source (signed) → CI (OIDC) → registry (digest) → runtime
           SBOM + scan attach evidence to this path
```

What can break

- Signing commits but not pinning Actions by digest.

How to read this chapter

Server-side git security, scanning, SBOM, secrets, incidents, hardening.

Signed Commits on the Server

Signed Commits on the Server

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Signed commits, tags, verification on the platform.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

Dependency Scanning

Dependency Scanning

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Dependency scanning and vulnerability management.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

SBOM

SBOM

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Software Bill of Materials (SBOM).

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

Secrets Detection

Secrets Detection

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Secrets detection and management.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

Supply Chain Attacks

Supply Chain Attacks

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Lessons from real incidents.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

Platform-Level Hardening

Platform-Level Hardening

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Platform-level security hardening.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

Federation

Theory

Theory: Git is Already Distributed; Forges are Not

Every clone is a replica of objects. Federation of *issues and PRs* needs extra protocols (ForgeFed, ActivityPub) or boring mirroring.

Invariants

1. Mirroring is practical federation of Git objects.
2. Cross-instance PRs are still experimental.
3. Radicle and similar try to drop the forge as a single host.

```
clone A  ↔  clone B     (Git, solved)
forge A   ?   forge B     (issues/PRs, unsolved at scale)
```

What can break

- Assuming Codeberg and GitHub issues sync because remotes do.

How to read this chapter

Git as primitive, federated hosting, readiness.

Git as a Decentralization Primitive

Git as a Decentralization Primitive

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Git's native distribution as a decentralization primitive.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

Federated Hosting

Federated Hosting

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Federated hosting and cross-instance collaboration.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

Are We Ready?

Are We Ready?

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Are we ready for decentralized code hosting?

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

Storage and Scale

Theory

Theory: Git Storage is Packs Plus a Database of Metadata

Repo growth hurts fetch time, `gc`, and inode counts. The forge DB (users, issues) scales on a different axis than packfiles.

Invariants

1. Local disk vs NFS vs Ceph vs S3 changes latency of `git fetch`.
2. Gitaly-style git servers shard repos, not tables.
3. CDNs help clone of *public* objects; auth still hits origin.

```
packfiles / alternates / object storage
metadata DB  (issues, users)   separate scale plan
```

What can break

- NFS for lots of tiny Git objects without cache.

How to read this chapter

Growth, storage options, fetch, DB, CDN/geo, benchmarks.

Repository Growth

Repository Growth

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- When size becomes a problem.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

Storage Options

Storage Options

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Local, NFS, Ceph, S3-compatible.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

Clone and Fetch Performance

Clone and Fetch Performance

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Cloning and fetch performance.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

Database Scaling

Database Scaling

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Database scaling for Git platforms.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

CDN and Geo-Distributed Hosting

CDN and Geo-Distributed Hosting

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- CDN strategies and geo-distributed hosting.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

Benchmarks

Benchmarks

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Benchmarks and performance profiles.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

HA and DR

Theory

Theory: Backup is Another Clone Plus Metadata Dump

Git objects backup with `git clone --mirror`. Issues, PRs, packages, and CI logs need the forge's dump. HA is keeping both classes online.

Invariants

1. A mirror clone is not a point-in-time of the issue tracker.
2. Geo-redundant Git needs object replication *and* DB replication.
3. DR tests are restores, not only snapshots.

```
objects  --mirror-->  cold clone
postgres dump  --→  object store
both required for a forge restore
```

What can break

- Backing up `/data/git` and forgetting the DB.

How to read this chapter

Backup, PITR, geo, DR plan, repo health alerts.

Backup Strategies

Backup Strategies

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Bare clones, dumps, and replication.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

Point-in-Time Recovery

Point-in-Time Recovery

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Point-in-time recovery and versioning.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

Geo-Redundant Deployments

Geo-Redundant Deployments

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Geo-redundant and multi-region deployments.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

Disaster Recovery Planning

Disaster Recovery Planning

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Disaster recovery planning.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

Monitoring Repository Health

Monitoring Repository Health

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Monitoring and alerting for repository health.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

Observability

Theory

Theory: If You Cannot See Ref Movement, You Cannot Audit

Operational metrics (latency, error rate, disk) keep the forge up. Audit logs keep it honest. They are different streams.

Invariants

1. Golden signals for a git frontend: fetch/push latency, 5xx, queue depth.
2. Audit: who moved which ref, who changed access.
3. SOC 2 / ISO / GDPR ask for *retention and access* of those logs.

```
metrics  (SRE)     logs  (debug)     audit  (compliance)
all three, different retention
```

What can break

- DEBUG application logs as your only audit trail.

How to read this chapter

Metrics, logs, audit, tracing, frameworks, detection.

Operational Metrics

Operational Metrics

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- What to measure and why.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

Log Aggregation

Log Aggregation

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Log aggregation and analysis.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

Audit Logging

Audit Logging

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Audit logging for compliance.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

Tracing User Actions

Tracing User Actions

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Tracing user actions across the platform.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

Compliance Frameworks

Compliance Frameworks

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- SOC 2, ISO 27001, GDPR.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

Security Monitoring

Security Monitoring

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Security monitoring and incident response.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

Enterprise and Air-Gap

Theory

Theory: Air-Gap is a Closed Universe of Bytes

Offline forges must import Git, packages, container images, and CI images as **files**. TCO includes humans who carry those files.

Invariants

1. `git bundle` and registry exports are the transport.
2. Data residency is a legal constraint on *where disks live*.
3. SLAs are people and spare capacity, not a YAML flag.

```
connected world --bundle/image-->  sneakernet  -->  air-gapped forge
```

What can break

- CI that `docker pull`s from the internet on an air-gapped runner.

How to read this chapter

Air-gap, hybrid, residency, support/SLA, TCO.

Air-Gapped Deployments

Air-Gapped Deployments

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Air-gapped and offline deployments.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

Multi-Cloud and Hybrid

Multi-Cloud and Hybrid

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Multi-cloud and hybrid architectures.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

Data Residency

Data Residency

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Compliance and data residency.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

Support and SLAs

Support and SLAs

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Enterprise support and SLAs.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

Capacity and TCO

Capacity and TCO

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Capacity planning and TCO analysis.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

Migration and Future

Theory

Theory: Migration is Mapping Names, Then Objects, Then Metadata

Git history is the easy part (`push --mirror`). Users, PRs, CI, and packages are lossy maps. A decision framework is which losses you accept.

Invariants

1. Object hashes can stay; URLs and issue numbers usually cannot.
2. Interoperability is shared Git + webhooks, not identical UIs.
3. AI features will attach to whatever forge has the social graph.

```
Git objects    lossless     new forge
PRs/issues     best effort     importer
Actions YAML   rewrite     new CI
```

What can break

- Promising zero-loss PR comments without reading the importer docs.

How to read this chapter

Strategies, preserving history, interoperability, trends, decision, future.

Migration Strategies

Migration Strategies

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Tools and processes.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

Preserving History

Preserving History

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Preserving history, permissions, and metadata.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

Interoperability

Interoperability

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Interoperability between platforms.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

Emerging Trends

Emerging Trends

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- AI-assisted development, collaboration, ecosystem evolution.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

A Decision Framework

A Decision Framework

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Making your choice: a decision framework.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

The Future of Code Hosting

The Future of Code Hosting

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- The future of code hosting.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

jj (Jujutsu): Working Copy as a Commit

jj (Jujutsu): Working Copy as a Commit

Git teaches a three-layer story: working tree, index/staging area, commits. Most day-to-day anxiety — "did I stage that?", "is this amend or a new commit?", "how do I undo the last amend?" — lives in the gaps between those layers. **Jujutsu (`jj`)** collapses the gap: the working copy *is* a real commit (`@`). Every edit is snapshotted into that commit the next time you run a `jj` command. There is no staging area to reconcile. Undo is an operation-log rewind. This chapter builds that mental model on a colocated Git repo, runs the handful of commands you need every day (`jj new`, `jj describe`, `jj squash`, `jj undo`), and shows how remotes still speak Git.

Verified against **jj 0.46.0** on x86_64 Linux (2026-10-09). Primary docs: https://docs.jj-vcs.dev/latest/.

Learning goals

By the end of this chapter you can:

- Explain why `@` is a commit ID that changes as you type
- Distinguish **change ID** (stable) from **commit ID** (content-addressed, rewritten)
- Colocate jj on an existing Git checkout with `jj git init`
- Start a fresh change (`jj new`), describe it, fold WIP with `jj squash`, and rewind with `jj undo`
- Push to a Git remote without abandoning Git-speaking collaborators

Thesis

> In Git, a commit is what you produce when you are *done*. In jj, a commit is what you are *editing*. The working copy is not a waiting room — it is revision `@`, rewritten in place until you say `jj new`.

That single flip removes the index. Files you create are tracked automatically (subject to `.gitignore` and `snapshot.auto-track`). Files you delete are untracked automatically. `jj st` both reports and snapshots.

Mental model

```
Git mental layers              jj mental layers

working tree
staging index   → commit      working-copy commit @  ←  you edit files here
                               parent commit      @-
                               operation log      (every jj command)
```

Term	Meaning
`@`	Working-copy commit — rewritten on almost every `jj` command when files changed
`@-`	Parent of `@` (usually the last "finished" change)
Change ID	Stable identifier for a logical change (survives rewrites); prefer this in daily use
Commit ID	Git-compatible content hash; changes whenever the snapshot changes
Operation	One recorded `jj` action; `jj undo` rewinds the latest

Case 0 — install and version

```
# jj 0.46.0 - verify yours
jj --version
```

```
jj 0.46.0-7d382314f9420b4b8d932245b49c99492c812b9f
```

Configure identity once (required before any push to a Git remote):

```
jj config set --user user.name "Your Name"
jj config set --user user.email "you@example.com"
```

Case 1 — colocated repo: Git and jj share one working tree

```
# demo.sh - colocated init and first snapshot
rm -rf /tmp/jj-wc-demo && mkdir /tmp/jj-wc-demo && cd /tmp/jj-wc-demo
jj git init --colocate
echo 'hello' > README.md
jj status
```

```
Initialized repo in "."
Working copy changes:
A README.md
Working copy  (@) : rukslwvq 762e295b (no description set)
Parent commit (@-): zzzzzzzz 00000000 (empty) (no description set)
```

What just happened:

1. `jj git init --colocate` created both `.jj/` and `.git/` in the same directory (the default for Git-backed workspaces).

2. Adding `README.md` and running `jj status` **snapshotted** the file into `@`. There was no `git add`.

3. `@` already has a change ID (`rukslwvq…`) and a commit ID (`762e295b…`). The commit ID will change on the next rewrite; the change ID will not.

On an *existing* Git clone the same command works from inside the checkout: `jj git init` (colocate is the default). jj imports Git refs and keeps exporting on every command so `git log` / `git status` stay coherent — usually with Git in a detached-HEAD-looking state, which is normal for colocated workflows.

Case 2 — describe, then `jj new` when the change is "done enough"

```
jj describe -m 'add README'
jj log -r 'all()' --limit 5
jj new
jj log -r '@ | @-'
```

```
Working copy  (@) now at: rukslwvq c4ec738f add README
Parent commit (@-)      : zzzzzzzz 00000000 (empty) (no description set)
@  rukslwvq … c4ec738f
   add README
   zzzzzzzz root() 00000000
Working copy  (@) now at: pslozopm e8dc9b58 (empty) (no description set)
Parent commit (@-)      : rukslwvq c4ec738f add README
```

`jj describe` rewrote `@` (new commit ID, same change ID, now with a message). `jj new` created an *empty* successor and moved `@` there. The finished work sits at `@-`. That is the jj equivalent of "commit what I staged and start fresh" — except nothing was staged.

Case 3 — keep editing, then squash (or edit)

```
echo 'world' >> README.md
jj diff
jj squash -m 'extend README'
```

```
Modified regular file README.md:
   1    1: hello
        2: world
Working copy  (@) now at: lyxlzlzu … (empty) (no description set)
Parent commit (@-)      : rukslwvq 66cd1710 extend README
```

`jj squash` moves the changes from `@` into its parent (default) and leaves you on a new empty working-copy commit. It is the boring "amend the previous change with my current WIP" path. The alternative is `jj edit <change>` to resume amending an older change directly in `@` — useful when the change is still open; squash-then-diff is clearer when you want a reviewable delta first.

Case 4 — undo is not reflog archaeology

```
jj undo
jj log -r 'all()' --limit 8
```

```
Undid operation: … squash commits into …
Working copy  (@) now at: pslozopm 241153d1 (no description set)
Parent commit (@-)      : rukslwvq c4ec738f add README
@  pslozopm … 241153d1
   (no description set)
   rukslwvq … c4ec738f
   add README
   zzzzzzzz root() 00000000
```

Every `jj` command records an **operation**. `jj undo` rewinds the last one (here: the squash). The working-copy files and the commit graph return to the prior operation. For deeper recovery: `jj op log`, then `jj op undo` / restore by operation id. This is why "I amended the wrong thing" stops being an emergency.

Interop with Git remotes

Colocated repos still push with Git semantics:

```
# Create a bookmark (jj's branch-like movable ref) on the finished change
jj bookmark create my-feature -r @-
```

```
jj git push --bookmark my-feature
# Or let jj mint a name from a change id:
# jj git push --change @-
```

Notes for Git teammates:

- They clone and work with plain Git; the `.jj/` directory is yours.
- Prefer bookmarks for anything that must be named on the remote.
- After `jj git push`, if `@` became immutable, jj creates a fresh working-copy commit on top — run `jj new` yourself if you need a clean empty `@` before more edits.
- Mixing `git` mutation commands in a colocated workspace works but is easier if you mostly stay in `jj` and treat `git` as the remote transport / blame / bisect tool.

The trap

- **Looking for `git add`.** There isn't one. Ignore files with `.gitignore`; untrack with `jj file untrack` after ignoring.
- **Treating commit IDs as stable handles.** Use change IDs (`jj log` shows them first). Commit IDs churn every snapshot.
- **Pushing `@` when it is empty WIP.** Push `@-` or an explicit bookmark on the finished change (`jj git push --change @-`).
- **Expecting Git's staging partial-hunk workflow by default.** jj snapshots the whole working copy; use `jj split`, `jj squash -i`, or file-level tools when you need to carve hunks.
- **Forgetting identity config.** Until `user.name` / `user.email` are set, commits are empty-identity and remotes will reject them.

The boring rule

Edit freely. Let `@` absorb the snapshot. When the change is ready enough, `jj describe` + `jj new` (or `jj squash` into the parent). Wrong move? `jj undo`. Speak Git at the boundary with bookmarks and `jj git push`.

Try this

1. `jj git init --colocate` inside a throwaway Git repo you already know.

2. Make three file edits; run `jj st` after each; watch the commit ID of `@` change while the change ID stays put.

3. `jj new`, edit again, `jj squash`, `jj undo`, confirm the diff returns.

4. `jj bookmark create demo -r @- && jj git push --bookmark demo` to a personal fork.

Sources

- Working copy: https://docs.jj-vcs.dev/latest/working-copy/
- Tutorial (working-copy-as-commit): https://docs.jj-vcs.dev/latest/tutorial/
- Git compatibility / colocation: https://docs.jj-vcs.dev/latest/git-compatibility/
- GitHub workflow notes: https://docs.jj-vcs.dev/latest/github/

GitHub Actions

Fundamentals

Theory

Theory: A Workflow is a Graph of Jobs Triggered by Git Events

Actions YAML in Git is both source and config. An event (push, PR, schedule) instantiates jobs, each a sequence of steps on a runner.

Invariants

1. Jobs in a workflow may run in parallel unless `needs` says otherwise.
2. The GITHUB_TOKEN is a short-lived credential scoped to the repo/job.
3. Contexts (`github`, `secrets`, `matrix`) are the runtime environment.

```
event → workflow → jobs → steps → runner VM/container
```

What can break

- Unpinned third-party actions on `main`.

How to read this chapter

Why Actions, anatomy, runtime, authoring/debug, build/validate.

Automate Tasks with Actions

Automate Tasks with Actions

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Automate tasks in GitHub with Actions.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

Workflow Anatomy

Workflow Anatomy

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Events, jobs, steps.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

Runtime and Contexts

Runtime and Contexts

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- The workflow runtime and environment contexts.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

Authoring and Debugging

Authoring and Debugging

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Authoring, debugging, and validating workflows.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

Building and Validating Code

Building and Validating Code

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Building and validating your code.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

Custom Actions and Releases

Theory

Theory: Composite, JS, and Container Actions are Functions

A custom action is a reusable function with inputs/outputs. Releases are Git tags plus bits in GitHub Packages or attached files.

Invariants

1. Pin by commit SHA, not by moving tag, for third-party actions.
2. OIDC lets jobs prove identity to clouds without long-lived keys.
3. Matrix is cartesian expansion of a job.

```
uses: org/action@<sha>     pinned
uses: org/action@v3         moving tag  (weaker)
```

What can break

- `pull_request_target` with untrusted checkout.

How to read this chapter

Building actions, reuse/matrix/cache, releasing, security notes.

Building Custom Actions

Building Custom Actions

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Building custom GitHub Actions.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

Reuse, Matrix, Cache, Secrets

Reuse, Matrix, Cache, Secrets

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Reuse, matrix, cache, artifacts, secrets, and environments.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

Releasing Software

Releasing Software

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Releasing software with Actions.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

Actions Security Notes

Actions Security Notes

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Pinning, `GITHUB_TOKEN`, OIDC (full supply-chain is chapter 34).

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

GitOps

What Is GitOps

Theory

Theory: Desired State Lives in Git; the Cluster Converges

GitOps's four pillars: Git as source of truth, declarative desired state, pull-based agents, continuous reconciliation. Push-based CI (`kubectl apply` from a pipeline) is CIOps, not GitOps.

Invariants

1. If it is not in Git, it is drift.
2. The agent, not the human, writes to the cluster API.
3. Preview environments are extra desired states (extra Git refs or folders).

```
Git desired  --pull-->  reconciler  --apply-->  cluster actual
                 compare + converge
```

What can break

- Manual `kubectl` on a GitOps cluster (the next reconcile fights you).

How to read this chapter

Pain, pillars, benefits, push vs pull, env repos, trunk vs PRs, vs other *Ops, when.

DevOps Pain Points

DevOps Pain Points

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Manual kubectl, script proliferation, security gaps.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

The Four Pillars of GitOps

The Four Pillars of GitOps

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- The four pillars of GitOps.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

Deploy Faster, Recover Faster

Deploy Faster, Recover Faster

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Deploy faster, recover faster, secure deployments, self-documenting systems.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

Push vs Pull

Push vs Pull

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Push-based vs pull-based deployments.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

Environment Repositories

Environment Repositories

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Environment repositories; multiple apps and environments; preview environments.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

Trunk vs Pull Requests

Trunk vs Pull Requests

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Trunk-based development vs pull requests in a GitOps world.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

GitOps vs Other Ops

GitOps vs Other Ops

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- GitOps vs DevOps, CIOps, NoOps.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

When to Use GitOps

When to Use GitOps

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- When to use GitOps vs traditional CI/CD; common misconceptions.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

Flux Quickstart

Theory

Theory: Flux is a Set of Controllers that Watch Git

Flux Operator installs controllers that reconcile GitRepository, Kustomization, HelmRelease objects. The cluster is the actor.

Invariants

1. App repo vs env repo is a naming convention for desired state.
2. A GitHub Action may *update a tag in Git*; Flux *applies* it.
3. This book's deep GitOps path continues with Argo CD (different UX, same theory).

```
Actions: build image, commit tag
Flux:    see tag, helm upgrade / apply
```

What can break

- Storing cluster secrets as plain YAML in a public repo.

How to read this chapter

Repos, CD with Actions, Flux Operator, why Argo is next.

Application and Environment Repos

Application and Environment Repos

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Application and environment repositories.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

CD Pipeline with Actions

CD Pipeline with Actions

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Continuous delivery pipeline with GitHub Actions.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

Flux Operator on Kubernetes

Flux Operator on Kubernetes

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Install and pull-based deployment.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

The Future of GitOps

The Future of GitOps

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- The future of GitOps (and why Part VIII is Argo CD).

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

Argo CD

Concepts and Architecture

Theory

Theory: Reconciliation is Compare Desired to Actual, Forever

Argo CD's Application is a pointer at Git (desired) plus a cluster (actual). The controller loop diffs and optionally syncs. Immutable infrastructure: replace, do not SSH-mutate.

Invariants

1. Five-step loop: detect, compare, present, sync (if policy), record.
2. Repo server renders manifests (Helm/Kustomize/plain) in a sandbox.
3. Sync status and health are different questions.

```
Git  →  repo-server  →  desired manifests
cluster API  →  actual
controller: diff + sync policy
```

What can break

- Editing live resources that auto-sync will revert.

How to read this chapter

History, reconciliation, loop, components, apps/projects, path to cluster, desired vs actual.

Where Argo CD Came From

Where Argo CD Came From

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Where Argo CD came from, and why that gap existed.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

Reconciliation and Immutable Infrastructure

Reconciliation and Immutable Infrastructure

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- What Argo CD actually does.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

The Reconciliation Loop

The Reconciliation Loop

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- The five-step reconciliation loop; declare outcomes, not steps.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

Argo CD Components

Argo CD Components

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- API server, application controller, repo server, Redis, Dex, ApplicationSet controller, Web UI, CLI.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

Applications and Projects

Applications and Projects

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Applications and projects; how Git becomes manifests.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

From Git to the Cluster

From Git to the Cluster

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- How a change travels from Git to the cluster.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

Desired vs Actual State

Desired vs Actual State

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Desired vs actual state, sync status, and sync history.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

Install and Lab

Theory

Theory: The Control Plane is Just Another Workload

Argo CD runs in the cluster (or a management cluster). Install flavor (full, core, Helm, Kustomize) is which components you accept.

Invariants

1. A management cluster can drive many workload clusters.
2. The CLI and UI are API clients; RBAC applies to both.
3. kind/K3s on Linux is enough to learn the loop.

```
laptop kubectl  →  k3s  →  argo-cd namespace  →  in-cluster (or external) apps
```

What can break

- Exposing the Argo UI to the internet with admin:admin.

How to read this chapter

Lab shape, k3s, install methods, UI/CLI/clusters.

Lab Shape

Lab Shape

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Two servers, a cluster, and an app to deploy.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

K3s and kubectl

K3s and kubectl

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Installing K3s (or kind); kubeconfig; kubectl.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

Install Methods

Install Methods

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Full, minimal (core), custom, Helm, Kustomize.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

UI, CLI, and Clusters

UI, CLI, and Clusters

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Web UI, CLI, token login, adding clusters.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

Applications and Sync

Theory

Theory: Sync Policy is When the Loop May Write

Manual sync is a human-gated apply. Auto-sync is the loop applying whenever Git moves. Self-heal reapplies when the cluster drifts. Prune deletes objects gone from Git.

Invariants

1. Every field on spec.syncPolicy is a permission the controller has.
2. Server-side apply vs client-side merge is a conflict-management choice.
3. History limits bound how much old sync state you keep.

```
Git move → (auto-sync?) → apply → (prune?) → health
```

What can break

- Auto-prune on a shared namespace you do not fully own.

How to read this chapter

First app, policy, prune, retries/options, namespace/labels, recap.

Your First Application

Your First Application

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- CLI vs declarative manifests.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

Sync Policy

Sync Policy

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Auto-sync, self-healing, jitter, cluster drift.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

Pruning

Pruning

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Pruning, exceptions, propagation policy, and garbage collection.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

Retries and Sync Options

Retries and Sync Options

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Retries, backoff, ignore differences, selective sync, replace vs merge, server-side apply.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

Namespace, Labels, History

Namespace, Labels, History

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Namespace automation, labels, history limits.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

Recap Exercise

Recap Exercise

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Recap exercise.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

Clusters and Diff

Theory

Theory: Diff Algorithms Decide What "Out of Sync" Means

Argo compares live Kubernetes objects to rendered Git. Ignore differences, jq paths, and server-side apply all change that predicate.

Invariants

1. Webhook vs polling is how fast Git is noticed, not how sync works.
2. Health is controllers' opinion (Deployment available), not Git.
3. External clusters need credentials in Argo's DB/secret store.

```
desired YAML  vs  live JSON   →  sync status
live conditions            →  health status
```

What can break

- Ignoring `/spec/replicas` while HPA owns replicas (or vice versa).

How to read this chapter

Registration, poll vs webhook, diffs, health.

Cluster Registration

Cluster Registration

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- In-cluster vs external clusters; registration and discovery.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

Polling vs Webhooks

Polling vs Webhooks

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Git polling vs webhook-based synchronization.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

Diff Algorithms

Diff Algorithms

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Desired vs actual state: diff algorithms and change detection.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

Application Health

Application Health

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Application health and monitoring.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

Hooks and Waves

Theory

Theory: A Sync is a Phased Transaction with Optional Jobs

PreSync/Sync/PostSync hooks are Kubernetes Jobs/Pods Argo runs around apply. Waves order ordinary resources. A failed hook can fail the sync.

Invariants

1. `generateName` lets hooks rerun; a fixed `name` plus `BeforeHookCreation` is a trap.
2. Waves are integers on annotations, not a separate API.
3. Hook cleanup policy is part of the theory (delete on success/failure).

```
PreSync jobs → apply waves 0..N → PostSync jobs
```

What can break

- A hook that never completes (no backoff awareness).

How to read this chapter

Phases, names/waves, failures, anti-patterns.

Sync Phases

Sync Phases

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- How a sync runs in phases; a first PostSync smoke test.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

Names and Sync Waves

Names and Sync Waves

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- `generateName` vs `name`; hook cleanup; ordering with sync waves.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

When a Hook Fails

When a Hook Fails

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- What happens when a hook fails.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

Hook Anti-Patterns

Hook Anti-Patterns

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Hook anti-patterns and the `BeforeHookCreation` trap.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

Users and RBAC

Theory

Theory: Local Users are Passwords; RBAC is Casbin Strings

Creating a login and granting a role are two operations. `p` lines are permissions; `g` lines are group membership. Projects scope which repos and clusters an app may use.

Invariants

1. SSO (Dex) maps external identity into the same RBAC.
2. Project is a hard sandbox (source repos, destinations).
3. CLI tokens are users too.

```
account + p, g lines + project = what you may sync
```

What can break

- Admin in `policy.csv` and SSO both on, conflicting.

How to read this chapter

Creating a user, p/g, verifying grants.

Creating a User

Creating a User

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Creating a user is two unrelated operations.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

The p Line and the g Line

The p Line and the g Line

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Accounts; the `p` line and the `g` line; project scoping.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

Verifying Grants

Verifying Grants

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Verifying grants.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

Helm

Theory

Theory: Helm is a Function from Chart+Values to YAML

Argo renders Helm in the repo server. Helm hooks are not Argo hooks. Values can come from Git, multiple sources, or extras.

Invariants

1. The rendered YAML is what is synced, not the chart tarball as a black box (unless you choose that).
2. Versioning the chart *and* the values is two Git objects.
3. Helm hook annotations can surprise Argo if mixed carelessly.

```
chart + values.yaml  --helm template-->  desired YAML  --sync-->  cluster
```

What can break

- Helm hooks that create Jobs Argo then prunes.

How to read this chapter

Helm vs plain, charts/repos, deploy/values, hook comparison.

Helm vs Plain Resources

Helm vs Plain Resources

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Helm vs plain Kubernetes resources.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

Charts and Repositories

Charts and Repositories

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Creating, packaging, and versioning charts; Helm repositories.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

Deploying Charts and Values

Deploying Charts and Values

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Deploying and upgrading charts; overriding values; multiple sources.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

Helm Hooks vs Argo CD Hooks

Helm Hooks vs Argo CD Hooks

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Helm hooks vs Argo CD hooks.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

ApplicationSets and Patterns

Theory

Theory: ApplicationSets are Generators of Applications

An ApplicationSet template plus a generator (list, git, matrix, PR, cluster) produces many Application objects. App-of-apps is the same idea with a parent Application.

Invariants

1. Generators must be deterministic enough to converge.
2. One Application per cluster/tenant/branch is a naming policy.
3. Build→registry→bump tag in Git is the promotion loop.

```
generator (list of clusters) × template  →  Application objects
parent app  →  child apps           (app of apps)
```

What can break

- A generator that creates and destroys apps on every reconcile (flap).

How to read this chapter

ApplicationSets, app-of-apps, multi-branch, build-to-update.

ApplicationSets

ApplicationSets

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- The problem ApplicationSets solve; list generator and beyond.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

App of Apps

App of Apps

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- The app-of-apps pattern; repository patterns for GitOps.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

Multi-Branch CI/CD

Multi-Branch CI/CD

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- One application per Git branch.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

Build to Application Update

Build to Application Update

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Build → registry → Helm package → application update.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

Notifications, Secrets, and Blue-Green

Theory

Theory: Notifications Observe the Loop; Secrets Must Not Live as Plain Desired State

Slack fires on Application events. Secrets need an external system (Sealed Secrets, SOPS, ESO) so Git stays the *index* of secret *references*. Blue-green at the Argo CD layer is usually two apps or a Rollouts object.

Invariants

1. Notification config is itself desired state (or a cluster controller).
2. Sealed/SOPS ciphertext can live in Git; plaintext must not.
3. Blue-green without Rollouts is two Destinations or two services.

```
Application event → notification controller → Slack
Git ciphertext    → controller              → cluster Secret
```

What can break

- Base64 secrets in Git called 'encryption'.

How to read this chapter

Slack, secret strategies, blue-green.

Slack Notifications

Slack Notifications

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Slack notifications for sync events.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

Secret Management

Secret Management

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Secret management strategies.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

Blue-Green with Argo CD

Blue-Green with Argo CD

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Blue-green deployments with Argo CD.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

Argo vs Alternatives

Theory

Theory: Engines Differ in UX and Ecosystem, Not in Pull-vs-Push Physics

Flux, Argo CD, Jenkins X, Spinnaker, Tekton occupy different UI and CD-vs-CI mixes. The GitOps physics (reconcile desired from Git) is shared with Flux.

Invariants

1. Argo CD is application-centric UI + multi-cluster.
2. Flux is Kubernetes-API-centric, GitOps Toolkit CRDs.
3. Spinnaker is pipeline-centric, heavier.

```
CI (build)     CD (converge cluster)
Tekton/Jenkins    Argo CD / Flux / Spinnaker
```

What can break

- Running two reconcilers on the same objects.

How to read this chapter

Comparisons, choosing an engine.

Argo CD vs the Alternatives

Argo CD vs the Alternatives

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Argo CD vs Flux, Tekton, Jenkins X, Spinnaker.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

Choosing a GitOps Engine

Choosing a GitOps Engine

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Choosing your GitOps engine.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

Argo Rollouts

Introducing Rollouts

Theory

Theory: Progressive Delivery is Reducing Blast Radius Over Time

Kubernetes RollingUpdate is maxUnavailable/maxSurge, not traffic-aware. Canary and blue-green split *traffic* or *replicas* and add pause/analysis gates.

Invariants

1. A Rollout replaces a Deployment as the object that owns ReplicaSets.
2. Canary: some users (or pods) see the new version first.
3. Blue-green: two full stacks; cut traffic when ready.

```
RollingUpdate   gradual pods, mixed traffic
Canary          gated traffic/replicas
Blue-green      two stacks, one cutover
```

What can break

- Calling a 50% RollingUpdate a canary (no analysis, no traffic split).

How to read this chapter

Beyond RollingUpdate, canary/blue-green, risk.

Beyond RollingUpdate

Beyond RollingUpdate

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Where Kubernetes RollingUpdate stops and progressive delivery begins.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

Canary and Blue-Green Fundamentals

Canary and Blue-Green Fundamentals

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Canary and blue-green fundamentals.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

Gradual Releases and Risk

Gradual Releases and Risk

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Gradual releases and deployment risk.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

Resources and Lifecycle

Theory

Theory: A Rollout Owns Stable and Canary ReplicaSets

The controller creates ReplicaSets labeled stable/canary, shifts weights or replica counts through steps, and consults AnalysisTemplates.

Invariants

1. Experiments are extra ReplicaSets for isolated tests.
2. AnalysisRun is an instance of an AnalysisTemplate.
3. Abort returns traffic/replicas to stable.

```
Rollout → RS stable  (old)
        → RS canary  (new)
        → AnalysisRun(s)
```

What can break

- Editing the underlying Deployment that Rollout is supposed to own.

How to read this chapter

Rollout vs Deployment, stable/canary, analysis/experiments.

Rollout vs Deployment

Rollout vs Deployment

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Rollout vs Deployment; the controller; ReplicaSets.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

Stable and Canary Versions

Stable and Canary Versions

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Stable and canary versions; the rollout lifecycle.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

AnalysisTemplates and Experiments

AnalysisTemplates and Experiments

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- AnalysisTemplates, experiments, and resource relationships.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

Rollouts Lab

Theory

Theory: The Lab is a Tiny Cluster that Still Reconciles from Git

kind/k3s + Argo CD + Rollouts on Linux is enough to see steps, pauses, and aborts. Git remains desired state.

Invariants

1. A Service must select the right pods for replica-based canaries.
2. Traffic-based canaries need a mesh (Istio) in later chapters.
3. Verify with kubectl and the Rollout status, not only Git.

```
Git  →  Argo CD  →  Rollout CR  →  ReplicaSets + Services
```

What can break

- Applying Rollout YAML by hand and via Argo to the same object.

How to read this chapter

Stack, manifests, verify.

Rollouts Lab Stack

Rollouts Lab Stack

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Docker, kind, Argo CD, Argo Rollouts.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

Manifests and Verify

Manifests and Verify

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Services, rollout manifests, GitOps deployment, verifying state.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

First Canary

Theory

Theory: A Step is a Promise About Traffic or Replicas, Then a Pause

Canary steps are a state machine: setWeight/setCanaryScale, pause, analysis. Promotion advances; abort rewinds to stable.

Invariants

1. Pause without analysis is a human gate.
2. Inspection (`kubectl argo rollouts get`) reads that state machine.
3. Replica-based weight approximates traffic if pods are homogeneous.

```
setWeight 20 → pause → setWeight 50 → analysis → setWeight 100
```

What can break

- No pause and no analysis (just a slow RollingUpdate).

How to read this chapter

Steps, promote/abort/inspect.

Canary Strategy and Steps

Canary Strategy and Steps

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Canary strategy and steps; traffic progression and pauses.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

Promote, Abort, Inspect

Promote, Abort, Inspect

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Promotion, abortion, and rollout inspection.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

Istio Canaries

Theory

Theory: Traffic Split is a Mesh API, Not a Replica Count

Istio VirtualService weights send a percentage of requests to canary subsets. Replica count can stay equal while traffic is 1%.

Invariants

1. DestinationRule subsets must match pod labels.
2. Replica-based routing request-based routing under uneven load.
3. `setCanaryScale` exists because HPA + replica canaries fight (later chapter).

```
Gateway → VirtualService  90% stable subset
                          10% canary subset
```

What can break

- Subsets that do not match Rollout's pod labels.

How to read this chapter

Istio objects, weights, replica vs traffic.

Istio Traffic Management

Istio Traffic Management

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Gateway, VirtualService, DestinationRule.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

Subsets and Weighted Traffic

Subsets and Weighted Traffic

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Stable and canary subsets; weighted canary traffic.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

Replica-Based vs Traffic-Based

Replica-Based vs Traffic-Based

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Replica-based vs traffic-based routing.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

Analysis and Prometheus

Theory

Theory: Analysis is a Predicate on Metrics Over a Window

AnalysisTemplate defines queries and success/failure conditions. Inline analysis blocks a step; background analysis can abort later.

Invariants

1. Prometheus is an oracle, not the rollout controller.
2. Failure conditions should be monotonic enough to trust.
3. Automatic abort is the point; otherwise it is a dashboard.

```
step → AnalysisRun → Prom query → success | fail → next | abort
```

What can break

- Success condition that is true on no data.

How to read this chapter

Templates, conditions, inline vs background.

AnalysisTemplate and AnalysisRun

AnalysisTemplate and AnalysisRun

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- AnalysisTemplate and AnalysisRun; Prometheus queries.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

Success, Failure, Aborts

Success, Failure, Aborts

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Success and failure conditions; automatic aborts.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

Inline vs Background Analysis

Inline vs Background Analysis

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Inline vs background analysis.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

Experiments

Theory

Theory: An Experiment is a Bounded Extra Version

Experiments run a template for a duration, optionally with analysis, without becoming stable. They are for A/B or soak tests beside the main canary.

Invariants

1. Failed experiments should not promote.
2. They consume cluster capacity (extra pods).
3. They are not a substitute for canary steps.

```
stable  +  canary (rollout)  +  experiment pods (temporary)
```

What can break

- Leaving experiments running (cost, drift).

How to read this chapter

Side by side, analysis, vs AnalysisTemplates.

Running Versions Side by Side

Running Versions Side by Side

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Running versions side by side; experiment templates.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

Analysis During Experiments

Analysis During Experiments

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Analysis during experiments; lifecycle and failed experiments.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

Experiments vs AnalysisTemplates

Experiments vs AnalysisTemplates

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Experiments vs AnalysisTemplates.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

HPA and Canary

Theory

Theory: HPA Owns Replicas; Canary Steps Also Own Replicas

Two controllers cannot both own `spec.replicas` without a rule. `setCanaryScale` and traffic-based splits exist to stop HPA from shrinking the canary out of existence.

Invariants

1. Replica-based canary + HPA on the Rollout is the subtle bug.
2. Traffic-based canary lets HPA scale the whole set more safely.
3. Metrics-server must exist for HPA to do anything.

```
HPA  --replicas-->  Rollout  <-- canary step replica math
```

What can break

- HPA on both stable and canary Deployments independently.

How to read this chapter

The problem, metrics, `setCanaryScale`.

HPA with Replica-Based Canaries

HPA with Replica-Based Canaries

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- A subtle problem: HPA with replica-based canaries.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

Metrics Server and Canary Scaling

Metrics Server and Canary Scaling

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Metrics server, CPU load, and canary scaling.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

setCanaryScale

setCanaryScale

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Istio routing and `setCanaryScale`.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

Blue-Green

Theory

Theory: Blue-Green is Two Stacks and One Cutover

Active and preview Services point at old vs new ReplicaSets. Promotion swaps who is active. Analysis can run on preview before the swap.

Invariants

1. Capacity is ~2× during the preview window.
2. Rollback is swapping Services back, not a Git revert alone (Git should still record the desired Rollout spec).
3. StatefulSets are a later limitation chapter.

```
Service active  →  RS blue
Service preview →  RS green     promote: swap active
```

What can break

- Preview service accidentally public without auth.

How to read this chapter

Services, analysis, promote/rollback.

Active and Preview Services

Active and Preview Services

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Active and preview services; the blue-green lifecycle.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

Promotion Analysis

Promotion Analysis

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Pre-promotion and post-promotion analysis.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

Promote and Rollback

Promote and Rollback

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Manual and automatic promotion; rollback behavior.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

Rollout Notifications

Theory

Theory: Notifications are Observers of Rollout Events

Triggers fire on events (step, abort, promote). Templates format Slack messages. Scope is namespace or cluster.

Invariants

1. Mis-scoped notifications spam or miss.
2. Secrets for Slack tokens are cluster secrets, not Git plaintext.
3. Troubleshooting is "did the event fire" then "did the webhook accept."

```
Rollout event → trigger → template → Slack webhook
```

What can break

- One cluster-scoped subscription for every namespace.

How to read this chapter

Architecture, Slack config, troubleshooting.

Notification Architecture

Notification Architecture

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Notification architecture and scope (namespace vs cluster).

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

Triggers, Templates, Slack

Triggers, Templates, Slack

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Triggers, templates, subscriptions, and Slack configuration.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

Troubleshooting Notifications

Troubleshooting Notifications

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Troubleshooting notifications.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

Migrating to Rollouts

Theory

Theory: workloadRef Lets a Rollout Adopt a Deployment

Migration is changing the controller that owns ReplicaSets without changing the pod spec. `workloadRef` points at an existing Deployment. StatefulSets often cannot follow this path.

Invariants

1. Git (Argo) must stop managing the old Deployment's replica field.
2. Aborts during migration need a rehearsed rollback.
3. Stateful identity (ordinals, PVCs) does not map to canary ReplicaSets cleanly.

```
Deployment  --workloadRef-->  Rollout takes over RS
Git desired: Rollout, not Deployment replicas
```

What can break

- Both Deployment and Rollout controllers racing on the same RS.

How to read this chapter

workloadRef, conversion, canary migration, StatefulSet limits.

Migration and workloadRef

Migration and workloadRef

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Migration strategies and `workloadRef`.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

Direct Conversion

Direct Conversion

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Direct conversion and Argo CD synchronization.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

Canary Migration

Canary Migration

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Canary migration, aborts, retries, scaling down the old Deployment.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

StatefulSet Limitations

StatefulSet Limitations

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- StatefulSet limitations.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

Capstone

Theory

Theory: One Loop from Commit to Careful Traffic

Git commit → forge CI → image digest in Git → Argo CD sync → Rollout steps → analysis → stable. That is the book's end-to-end mental model.

Invariants

1. Every stage is a name over content-addressed bytes (Git hash, image digest).
2. Humans change Git; controllers change clusters.
3. Larger environments add more clusters, ApplicationSets, and stricter analysis — not a different physics.

```
commit → CI → digest in Git → Argo → Rollout → traffic → stable
```

What can break

- A snowflake kubectl at any stage (breaks the loop).

How to read this chapter

End-to-end model, larger environments.

A Working Mental Model

A Working Mental Model

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- A working mental model, end to end.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

Larger Environments

Larger Environments

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Progressive delivery in larger environments.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

Appendices

Understanding Questions and Answers

Understanding Questions and Answers

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Four focused questions with detailed answers after each part.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

Git Competency Checklists

Git Competency Checklists

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Level 1: 6 essential skills everyone needs.
- Level 2: 5 power skills (reflog, interactive rebase, cherry-pick, bisect, Conventional Commits).
- Level 3: 5 advanced specialist topics.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

Real-World Scenario Playbook

Real-World Scenario Playbook

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Sanjay's deleted branch panic → recovered with reflog.
- Ajay's messy history → cleaned with interactive rebase.
- Maya's critical hotfix → deployed with cherry-pick.
- The team's mysterious production bug → fixed with bisect.
- Professional commit messages.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

Lab Environment Reference

Lab Environment Reference

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Docker, kind/K3s, kubectl, Helm, Argo CD/Rollouts setup.
- Heredoc and shell conventions.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.

Command Quick Reference and Glossary

Command Quick Reference and Glossary

This page is a stub. Prose, labs, and diagrams come in a later writing pass.

Outline

- Command quick reference and glossary.

> **Note**
>
> Scaffolding for the frozen TOC. Linux is the primary host. See `content/_planning/SYLLABUS.md`.